

CONTENTS

What is inside

TECHNOLOGY · ORIENTATION	The map before the territory	1
TECHNOLOGY · HOW SOFTWARE IS BUILT	From idea to app store	4
TECHNOLOGY · THE INTERNET	What happens when you open an app	13
TECHNOLOGY · SYSTEM DESIGN	Building for a million people at 10 am	19
TECHNOLOGY · SYSTEM DESIGN IN PRACTICE	Design it like a PM: three worked walkthroughs	29
TECHNOLOGY · CLOUD	Renting the computer instead of buying it	36
TECHNOLOGY · DATA AND MACHINE LEARNING	Teaching computers by example	40
TECHNOLOGY · LARGE LANGUAGE MODELS	The world's most well-read autocomplete	46
TECHNOLOGY · RAG, TOOLS AND MCP	Giving the model a library card and a pair of hands	51
TECHNOLOGY · MEASURING WHAT WORKS	A/B tests: letting users settle the argument	58
TECHNOLOGY · PLATFORM ECONOMICS	Why the big get bigger	66
TECHNOLOGY · EMERGING TECH	Blockchain: a ledger nobody can quietly rewrite	70
TECHNOLOGY · BUILDING WITHOUT CODE	Low-code, no-code and vibe coding	78

TECHNOLOGY · ORIENTATION

The map before the territory

What actually happens when you tap "Place order", and the one picture of a software product that every later chapter hangs on

It is 8:47 pm on a Friday in Bengaluru. You open Zomato, search "biryani", pick a restaurant, pay by UPI and watch a little rider icon crawl towards your building. The whole thing takes ninety seconds of your attention. Behind it, a few dozen pieces of software in several data centres, your bank, NPCI, the restaurant's tablet and the rider's phone have all talked to each other, most of them more than once.

You will never be asked to build any of this. You will be asked, every week, to make decisions that depend on it: "Can we promise delivery in 20 minutes?", "Why did checkout break during the sale?", "Is it cheaper to build or buy search?", "Should the AI assistant be allowed to issue refunds?" This Part gives you enough of a mental model to ask good questions, understand the answers, and hold your own in an interview when the conversation turns technical.

A restaurant is a good model of a software product

Engineers use strange words for ordinary ideas. Almost every idea in system design has a twin in a busy restaurant, and once you see the twin, the word stops being scary.

The restaurant model of a software product

Each everyday role maps to a real component you will meet in this Part

Everyday world	In technology
You, the diner, reading the menu and ordering	Client (the app or browser) The part the user sees and touches. It asks for things; it does not cook.
The waiter who takes your order to the kitchen	API A fixed way of asking for something and getting an answer back. The menu is its documentation.
The kitchen that cooks	Server / backend Computers that run the business rules: price the cart, check stock, assign a rider.
Separate stations: tandoor, desserts, billing	Microservices Small independent services, each owning one job, each deployable on its own.
The storeroom and the order register	Database Where the truth is kept: menus, users, orders, payments.
Popular dishes kept warm at the pass	Cache Copies of frequently asked-for data kept close so answers come back instantly.
The host who seats guests across sections	Load balancer Spreads incoming requests across many identical servers so none is swamped.
The dispatch counter where packed orders wait	Message queue Holds work until the next step is free, so spikes do not crush the kitchen.
Franchise kiosks near every metro station	CDN Copies of images and videos stored near users so they load fast.
The guard at the gate checking IDs	Authentication and authorisation Who are you, and what are you allowed to do?

Keep this table in your head. When an engineer says "the cache hit rate dropped so the database fell over", you can translate: the warm dishes ran out, so every order went back to the storeroom, and the storeroom could not cope.

The layered view: nine questions every product answers

A second way to see a product is as a stack of layers. Each layer answers one question. A problem in any layer shows up to the user as the same thing: the app is slow, wrong or down. Part of a PM's job during an incident or a design review is to work out which layer the question really belongs to.

The layered view of a modern software product

Top row is closest to the user; each layer answers one question

Layer	What it does	Everyday example
Experience	How do users interact? Screens, buttons, feedback	<i>The Zomato app, the website, a WhatsApp chatbot</i>
Edge & delivery	How is content made fast everywhere?	<i>Food photos served from a CDN server in your city</i>
Security & access	Who are you and what may you do?	<i>OTP login; a restaurant can edit only its own menu</i>
Integration (APIs)	How do parts talk cleanly?	<i>App asks the order service to create an order</i>
Application logic	What business rules run?	<i>Surge fee, coupon rules, which rider to assign</i>
Async & real-time	How are spikes and live events handled?	<i>Order events queued; rider location pushed live</i>
Data	Where is truth stored, and read fast?	<i>Orders in a database, menus in a cache, search index</i>
Delivery & runtime	Where does code run and scale?	<i>Containers on AWS that add machines at dinner time</i>
Reliability & observability	How is health measured and protected?	<i>Alert fires when payment success drops below 98%</i>

Source: Adapted and extended from IIM Ahmedabad, Product Mindset 2026-27

You will hear specific product names attached to these layers. You do not need to know how to use any of them, only which layer they belong to, so you can place them when they come up.

Layer	Names you will hear	What the name tells you
Experience	React, Next.js, Swift (iOS), Kotlin (Android), Flutter	Which toolkit the screens are built with
Integration	REST, GraphQL, gRPC, OpenAPI/Swagger, Postman, Kong, Apigee	How services expose and document their APIs
Application logic	Java/Spring Boot, Node.js, Go, Python/Django	Language and framework the backend is written in
Data	PostgreSQL, MySQL, MongoDB, Cassandra, Redis, Elasticsearch	Relational database, document store, wide-column store, cache, search index
Async & real-time	Kafka, RabbitMQ, Amazon SQS, WebSockets	Queues and streams for events; live two-way connections
Delivery & runtime	Docker, Kubernetes, AWS Lambda	Packaging code into containers; running and auto-scaling them
Reliability	SLOs, error budgets, Prometheus, Grafana, Datadog	How health is defined, measured and graphed
Security	OAuth 2.0, JWT, IAM, WAF, Vault	Login standards, access tokens, permissions, firewalls, secret storage
Edge	Cloudflare, Akamai, Fastly, CloudFront	Content delivery networks and edge computing

The one idea to carry through this Part

Every technical choice is a trade between **speed for the user**, **cost for the company**, **reliability**, and **how fast the team can ship**. Engineers can tell you the options and their prices. Only the PM can say which trade the user and the business want. That is why PMs need to understand the technology, not to build it.

How to use the rest of this Part

Each concept follows the same pattern: an everyday example first, then how it works with a diagram, then the terms, then where it breaks, then why a PM cares, then how it comes up in interviews.

What is in Part III

Chapters build on each other; system design is the longest

How software is built SDLC, Waterfall, Agile, Scrum, Kanban, design thinking, UX heuristics, journey maps	The internet and APIs DNS, IP, TCP/UDP, HTTP/HTTPS, CDNs, REST, webhooks, WebSockets, rate limits	System design Databases, caching, scaling, queues, microservices, CAP, reliability, security, releases
Design it at a PM level UPI payment, food-delivery tracking, WhatsApp-style messaging, worked end to end	Cloud IaaS, PaaS, SaaS, deployment and pricing models, data residency, cloud bills	Data and ML Types of learning, training, neural networks, recommenders, when to use AI
LLMs Tokens, transformers, temperature, model types, prompting	RAG, MCP and agents Grounding models in your data, connecting tools, agent loops and their risks	AI product quality A/B tests, evals, precision and recall, guardrails, human-in-the-loop, cost per task
Platform economics Network effects, two-sided pricing, multihoming, switching costs	Blockchain, IoT, AR/VR What each is for, how it works, and when not to use it	Low-code, trust, brand lift LCNC and vibe coding, security-resilience-governance, measuring campaigns

Think first

Before reading on, write down what you think happens between tapping "Pay ₹349" and seeing "Order placed". List every system you think is involved. You will redo this exercise in the system-design chapter and compare.

Check yourself

1. In the restaurant model, what are the "waiter", the "dispatch counter" and the "dishes kept warm at the pass"?
2. Name the nine layers of a software product and the question each one answers.
3. An engineer says "we moved the menu reads to Redis". Which layer changed, and what should the user notice?
4. Which four things does almost every technical choice trade off against each other?
5. Kafka, Kubernetes and Cloudflare: place each in its layer.

TECHNOLOGY • HOW SOFTWARE IS BUILT

From idea to app store

The life cycle every feature goes through, why most teams moved from Waterfall to Agile, and how Scrum and Kanban actually run week to week

Think about how a house gets built. Someone decides what the family needs (three bedrooms, a ramp for grandmother). An architect draws plans. Masons build. An inspector checks the wiring. The family moves in, and for twenty years someone fixes leaks. Nobody pours the foundation before the plan is agreed, because changing a foundation later is ruinously expensive.

Software goes through the same stages. The difference is that software is far easier to change after it is "built", and that single fact explains most of the arguments about how teams should work.

The Software Development Life Cycle (SDLC)

The **SDLC** is the sequence of stages every piece of software passes through, from "someone has an idea" to "it is running and being looked after". Take a new one-tap checkout for an e-commerce app.

The seven stages of the SDLC, using a new checkout flow

Every feature passes through these stages; methods differ only in how often and in what size

- 1 Requirements**
Decide what the checkout must do: saved UPI handle, one tap, works on 3G, handles failed payments.
Watch out: PM owns this. Vague requirements are the top cause of rework.
- 2 Analysis**
Check feasibility, dependencies and risks: does the payment partner support saved mandates? What about fraud?
Watch out: Engineering and PM together; find the blockers now.
- 3 Design**
UX designs the screens; engineers design the system: which services, what data, which APIs.
Watch out: Cheapest place to change your mind.
- 4 Development**
Engineers write and review the code, usually in small pieces merged daily.
Watch out: PM answers questions fast so nobody guesses.
- 5 Testing**
QA and automated tests check it works: happy path, failures, old phones, slow networks.
Watch out: Defects found here cost more than defects found in design.
- 6 Deployment**
Release to users, often gradually: 1% first, then 10%, then everyone.
Watch out: Watch the metrics; have a rollback plan.
- 7 Maintenance**
Fix bugs, update for new OS versions, improve based on data.
Watch out: Most of a product's lifetime cost lives here.

Words to know

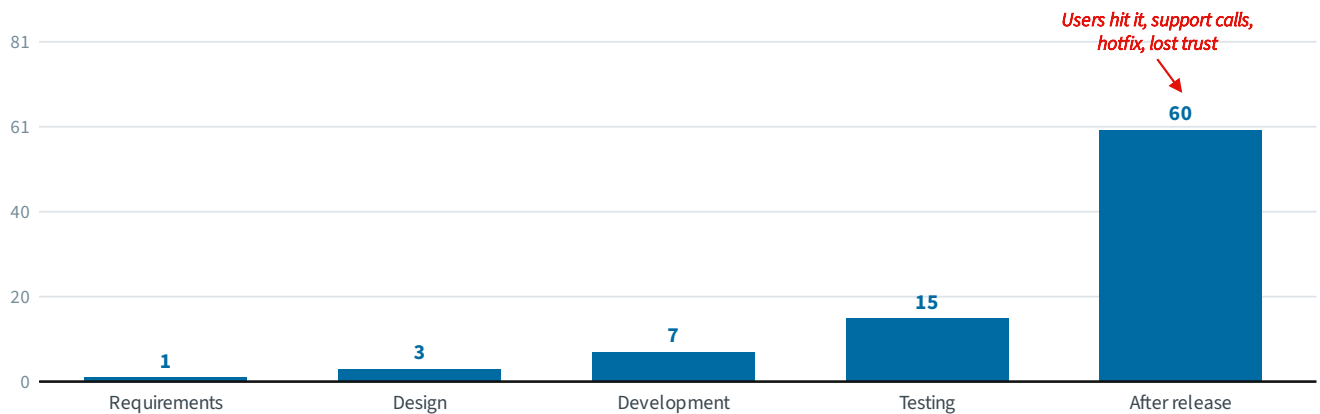
Requirement / spec / PRD: the written description of what to build and why. **QA:** quality assurance, the testing function.

Deployment / release: putting new code in front of users. **Rework:** redoing work because something was misunderstood or changed. **Technical debt:** shortcuts taken to ship faster that make later changes slower, like skipping a proper foundation to finish sooner.

Why a PM cares. Knowing the stages tells you when each decision must be made and who must be in the room. Lock the "what" before engineers start writing code, involve legal before design if payments or personal data are involved, and bring customer support in before launch, not after. The later a mistake is found, the more it costs to fix.

The later you find a mistake, the more it costs

Relative cost of fixing the same defect, by the stage where it is found (illustrative)

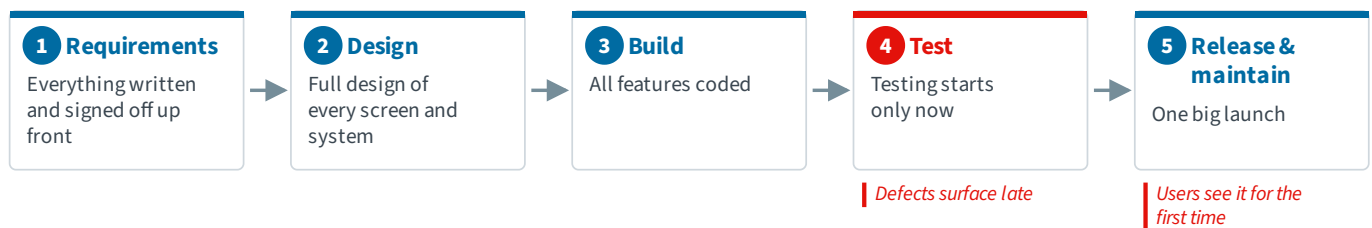


Source: Illustrative; the pattern is widely reported in software engineering studies, exact multiples vary

Waterfall: finish each stage before starting the next

Waterfall runs the SDLC once, in strict order. Requirements are fully written and signed off, then the whole design is done, then all the code, then all the testing, then one big release. Water only flows downhill; going back up is hard.

Waterfall: one pass, in strict sequence



Waterfall is not stupid. It fits work where requirements truly are fixed and mistakes after release are catastrophic or impossible to patch: a satellite, the software in a pacemaker, a bank's migration to a new core system with a regulatory deadline, a fixed-price government contract. ISRO cannot push a hotfix to a lander that is already on the Moon.

Its weaknesses show up whenever the world moves during the project:

- **Low adaptability.** A problem found in testing may send you back to design, and everything after it has to be redone.
- **Late delivery.** Users see nothing until the end, so competitors can ship first and you learn nothing from real users for months.
- **Risk piled up at the end.** Because testing comes last, the project looks fine until suddenly it does not.

Agile: build a little, show it, learn, repeat

Agile runs the whole SDLC in small loops, usually one to four weeks long, and ships working software at the end of each loop. Instead of one giant bet, you place many small ones and adjust after each.

The idea was written down in the *Agile Manifesto* (2001) by seventeen software developers. It values:

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

The items on the right still matter; the items on the left matter more. The principles that follow from this: deliver valuable software early and continuously; welcome changing requirements, even late; release frequently (every few weeks, often daily in practice today); keep technical quality high because messy code slows every later change; maximise the work *not* done; and reflect regularly on how to improve.

A useful picture: Waterfall builds a car wheel by wheel and hands you the car at the end. Agile hands you a skateboard, then a scooter, then a bicycle, then a car. Each version is usable, and you learn from how people use it. (This image comes from designer Henrik Kniberg.)

Waterfall versus Agile, delivering "get me across town"

Waterfall	Agile
● Month 1 A wheel. Useless alone.	● Week 2 A skateboard. Crude, but it moves.
● Month 3 A chassis. Still useless.	● Week 6 A scooter. Feedback: needs to carry bags.
● Month 6 A car, finally. If the user wanted a scooter, too late.	● Month 4 A car with a big boot, shaped by real use.

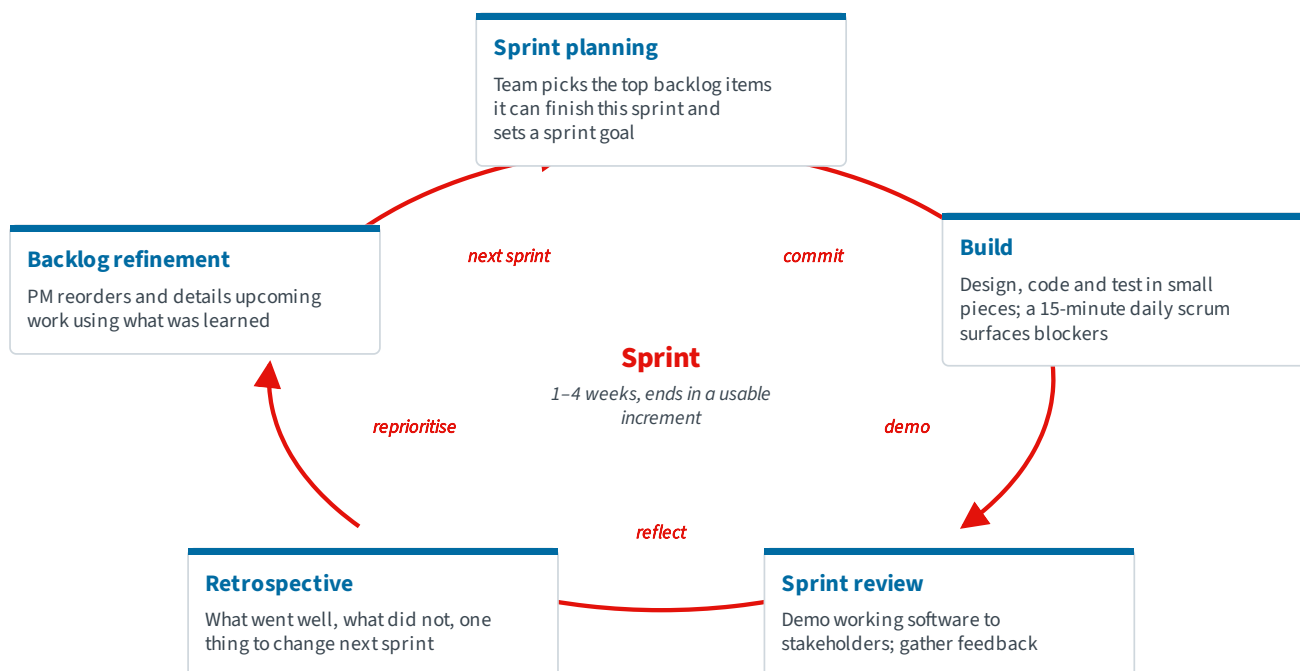
Agile lowers risk by putting something usable in users' hands early and learning from it.

Scrum: Agile in fixed-length sprints

Scrum is the most common way teams practise Agile. Work happens in **sprints**, fixed time-boxes of one to four weeks (two is typical in Indian startups).

One Scrum sprint

A two-week loop that repeats all year



Roles. The **Product Owner** decides what is built and in what order, and owns the backlog. The **Scrum Master** protects the process and removes blockers; they are a coach, not a manager. The **Developers** (engineers, designers, testers) decide *how* to build it and commit to what they can finish. In most Indian product companies the PM plays the Product Owner role, while the engineering manager or a senior engineer covers what a Scrum Master would do.

Scrum and backlog vocabulary

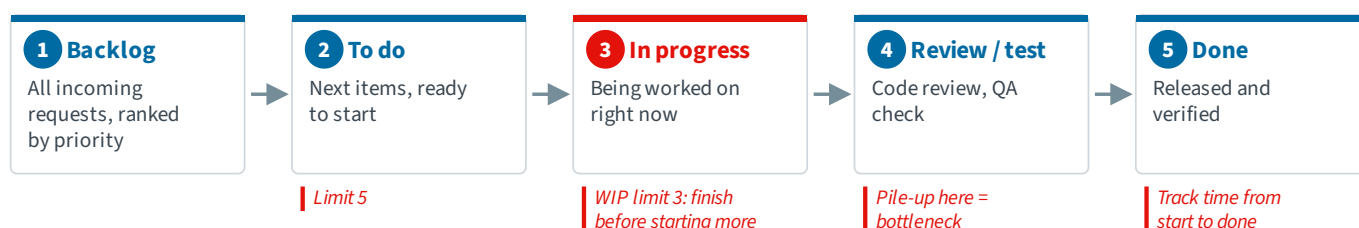
Product backlog: the ranked list of everything the product might do. **Sprint backlog:** the slice committed for this sprint. **User story:** a requirement written from the user's side, "As a first-time user, I want to pay with UPI without typing my ID, so that checkout is faster". **Acceptance criteria:** the checks that prove a story is done ("works when the UPI app is not installed"). **Story points:** a relative estimate of effort, not hours. **Velocity:** points completed per sprint, used for forecasting. **Definition of Done:** the shared bar every item must meet (tested, reviewed, documented). **Epic:** a big piece of work split into many stories.

Kanban: a continuous flow with limits

Kanban comes from Toyota's factories, where a card (*kanban* means "signboard" in Japanese) signalled that a station needed more parts. In software, every task is a card on a board, moving left to right. There are no sprints: work is pulled whenever capacity frees up.

A Kanban board for a payments support team

The WIP limit on each column is the rule that makes Kanban work



The key rule is the **work-in-progress (WIP) limit**: only so many cards may sit in a column. If "In progress" is full, nobody starts something new; they help finish what is stuck. This exposes bottlenecks (cards piling up in "Review" means you need more reviewers) and shortens **cycle time**, the time a task takes from start to done. Kanban suits teams whose work arrives unpredictably: support engineering, bug triage, operations, a trust-and-safety queue.

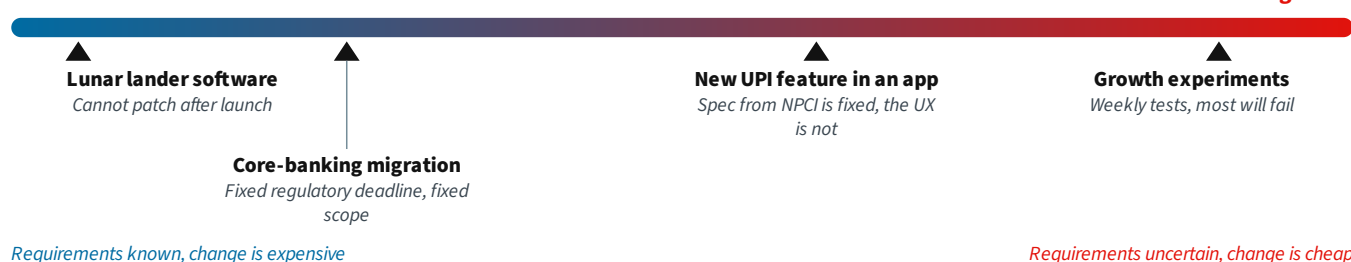
Aspect	Waterfall	Agile (Scrum / Kanban)
Flow	Linear, one pass through each stage	Iterative, many small loops
Flexibility	Changes are costly and disruptive	Changes welcomed, even late
Delivery	One release at the end	Frequent small releases
Customer involvement	At the start and the end	Continuous feedback
Risk	Concentrated at the end	Spread out, found early
Testing	After all building is done	Inside every sprint, often automated
Documentation	Heavy and detailed	Just enough to be clear
Best for	Fixed, well-understood scope; hardware; regulated migrations	Evolving requirements; consumer apps; anything with uncertainty

Where real projects sit

Most companies mix both: Agile for product work, Waterfall-like gates for regulated or hardware pieces

Waterfall fits

Agile fits



Contrasting case. Agile done badly becomes "no plan and constant churn": the backlog is reshuffled daily, stories are half-written, and the team never finishes anything. Agile does not mean no documentation or no deadlines. It means short planning horizons with real commitments inside each one.

The one-line rule

Waterfall bets that you know what to build; Agile bets that you will learn what to build. Choose based on how certain the requirements are and how expensive a change after release would be.

In the interview. "Agile or Waterfall for X?" is really a question about uncertainty and cost of change. A strong line: *"For the app's new onboarding I'd run two-week sprints and ship behind a feature flag, because we don't yet know what users will respond to. For the RBI-mandated change to how we store card data, the scope is fixed by regulation and the deadline is hard, so I'd plan it end to end up front, with Agile-style checkpoints inside."* Trap: saying "Agile is always better". Also be ready for "What does a PM do in sprint planning?" (brings a ranked, well-written backlog and the *why*; negotiates scope, not estimates) and "How do you handle an urgent request mid-sprint?" (assess impact; swap something out rather than silently adding; protect the sprint goal unless the issue is severe).

TECHNOLOGY · UX DESIGN

Design thinking: find the right problem before the right screen

How good product teams understand users, frame problems, and test ideas cheaply before engineers write code

PhonePe, Google Pay and BHIM all run on the same UPI rails, set by the same NPCI rules. Yet in April 2026 PhonePe alone processed about 47% of UPI transactions (startupfeed.in, April 2026), while NPCI's own BHIM app remains a small player. Same technology, same price (free), very different adoption. A large part of the difference is **user experience (UX)**: how easy, clear and trustworthy the product feels at every step, from scanning a QR code to seeing "Payment successful" with a satisfying tick.

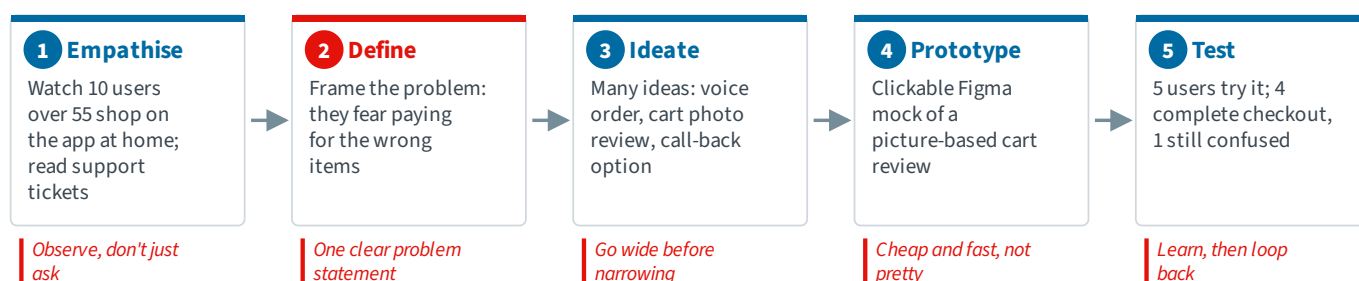
UX design is the practice of shaping a product so it is useful, usable and pleasant for the people who use it. The most widely taught approach to it is **design thinking**.

Design thinking in five stages

Design thinking is a user-centred, iterative way of solving problems. Its five stages (Stanford d.school's version) are **Empathise, Define, Ideate, Prototype, Test**, sometimes abbreviated EDIPT. "Iterative" matters: teams loop back when a test reveals that the problem was framed wrongly.

The five stages of design thinking

Worked example: a grocery app sees older users abandon checkout



Empathise: understand the user in their context

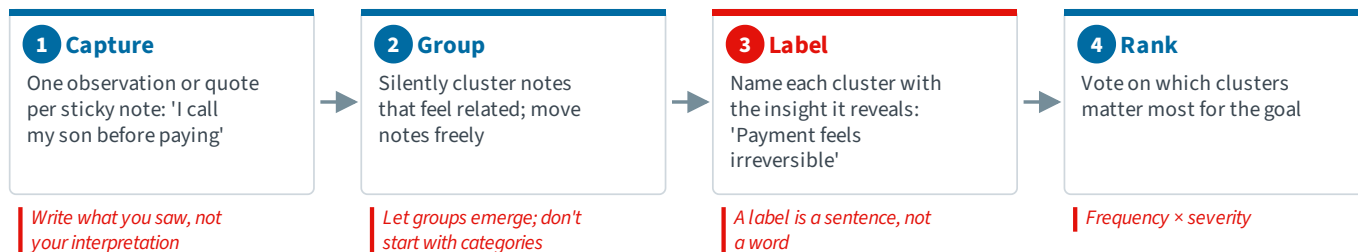
Start with secondary research (market reports, app reviews, competitor apps) and a **competitive analysis** to spot gaps. Then go to the users. Interviews tell you what people *say*; observation and usability studies show what they *do*; surveys tell you *how many* feel something; analytics tell you *where* they drop off. The richest insights usually come from watching someone use the product in their real setting: a kirana owner updating stock on a cheap phone while serving customers will not behave like a tester in an air-conditioned office.

Define: turn a pile of notes into one sharp problem

After twenty interviews you may have three hundred observations. The **KJ method** (named after Japanese anthropologist Jiro Kawakita) is a structured form of **affinity mapping** that turns them into themes.

Affinity mapping with the KJ method

From 300 sticky notes to five insights



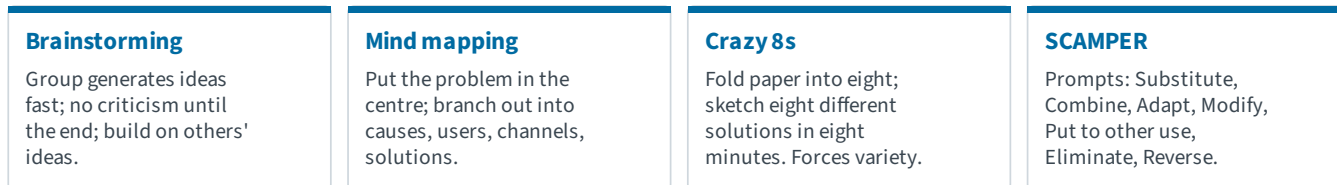
From the insights you build **personas**: fictional but research-based portraits of key user types, with demographics, goals, pain points, behaviours and context of use. A **primary persona** is the main target user ("Asha, 58, retired teacher in Nagpur, uses WhatsApp daily, shops online twice a month with her son's help"). A **secondary persona** has additional needs you should also serve (her son, who sets up the account). A **negative persona** is someone you are deliberately *not* designing for (a bulk buyer running a small shop), which is as useful as the primary one because it stops scope creep.

Then write a **problem statement** from the user's point of view: *[User] needs a way to [need] because [insight]*. For example: "A first-time online shopper over 55 needs a way to check exactly what she is paying for, because she fears money cannot be recovered once it leaves her account." Teams often reframe it as a "How might we..." question to open up ideation: "How might we make paying feel reversible?"

Ideate: go wide, then narrow

Generate many ideas without judging them first. Common techniques:

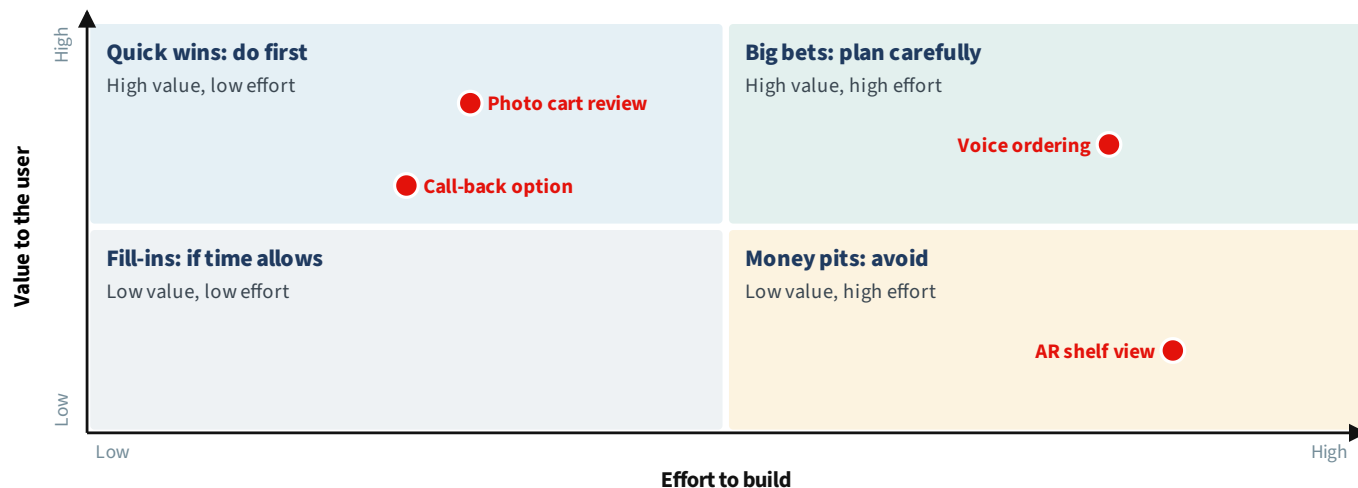
Four ideation techniques



Then prioritise. A simple way is to plot ideas on user value versus effort and start with the high-value, low-effort corner.

Prioritising ideas by value and effort

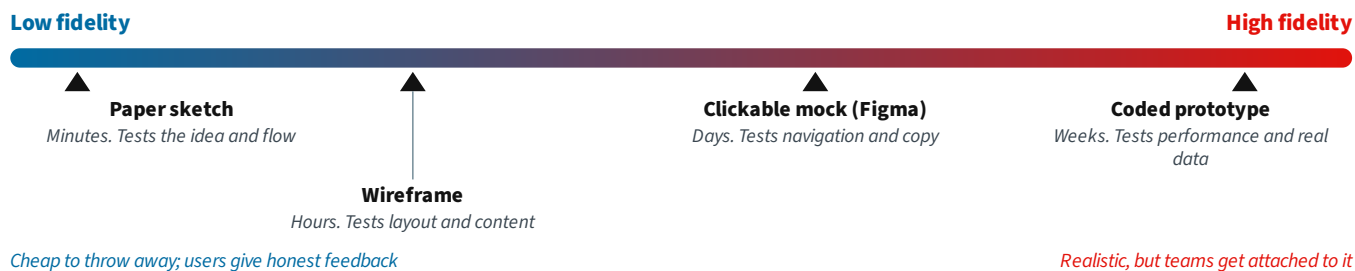
Grocery checkout ideas from the example above



Prototype: make it real enough to learn from

A **prototype** is a cheap, tangible version of an idea, built to test an assumption before paying for real engineering. **Fidelity** is how close it looks and behaves to the final product.

Prototype fidelity: match it to the question you are asking



Test: watch people use it

In a usability test you give users a task ("buy these three items and pay") and watch silently, asking them to think aloud. Five users typically uncover most of the big problems in a design (Jakob Nielsen's widely cited rule of thumb). Use A/B or multivariate tests once the product is live and you have traffic (see the chapter on AI product quality for how A/B testing works).

A faster check, done by experts rather than users, is a **heuristic evaluation** against **Nielsen's 10 usability heuristics**. Learn these with examples; they are the vocabulary of UX critique in interviews.

Nielsen's 10 usability heuristics, with examples you know

1. Visibility of status

Tell users what is happening. UPI apps show 'Processing, do not press back'.

2. Match the real world

Use the user's words and icons: ₹, 'Cash on delivery', vernacular options.

3. User control

Easy exits and undo. Gmail's 'Undo send'; cancel order within 60 seconds.

4. Consistency

Same word for same thing. Never mix 'Add to cart' and 'Add to basket'.

5. Error prevention

Stop mistakes before they happen: UPI shows the payee's bank name before you pay.

6. Recognition over recall

Show options instead of making users remember: recent payees, autocomplete.

7. Flexibility & efficiency

Shortcuts for experts: 'Reorder' button, UPI Lite for small payments without PIN.

8. Minimalist design

Show only what matters now. Google's home page; a one-field login.

9. Help recover from errors

Say what went wrong and what to do: 'Payment failed, ₹349 will be refunded in 48 hours'.

10. Help and documentation

Short, searchable, in context: a '?' next to the confusing field.

Source: Jakob Nielsen, Nielsen Norman Group (1994, updated 2020)

The three lenses every product must pass

Great products are **desirable** (users want it), **feasible** (we can build it with our technology and team) and **viable** (it makes business sense). Design thinking protects desirability; engineering checks feasibility; the business case checks viability. A PM is responsible for the overlap.

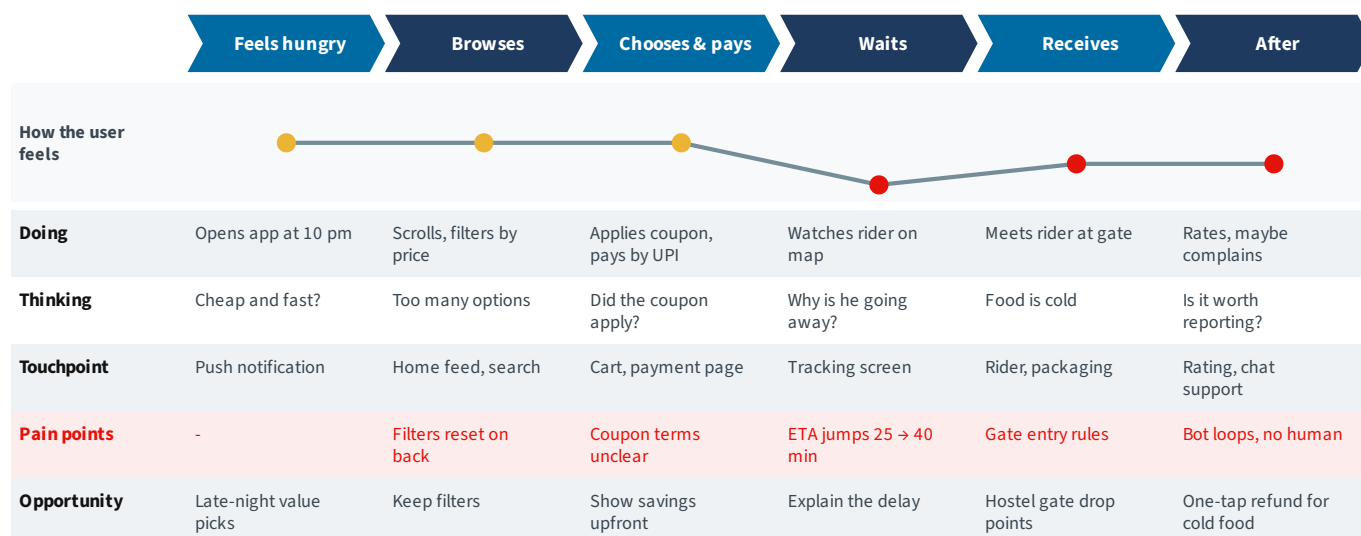
Customer journey maps: the whole experience, not one screen

A delivery app can have a beautiful checkout and still lose the customer, because the order arrived 25 minutes late and support took two days to reply. A **customer journey map** is a timeline of everything one type of customer does, thinks and feels while trying to

reach one goal, across every channel. It stops teams from optimising a single screen while the real pain sits elsewhere.

Journey map: a hostel student ordering dinner on a food app

Current state; the feeling curve shows where the experience breaks



A good map has six parts: the **actor** (one persona), the **scenario and goal**, the **phases**, the **actions and touchpoints**, the **thoughts and emotions**, and the **pain points and opportunities**. It is built from evidence, not imagination: interviews give goals and emotions; product analytics give drop-offs; support tickets and reviews give recurring friction and the customer's own words; observation shows workarounds nobody mentions; operational data (actual versus promised ETA) shows where the service itself fails.

Two kinds exist. A **current-state map** documents today's experience and is used to diagnose friction and prioritise fixes. A **future-state map** describes the experience you intend to build and is used to align teams around a redesign. The practical rule: **one actor and one scenario per map**. A student, a restaurant owner and a rider have different journeys; a map that tries to cover everyone guides nobody.

Why a PM cares. Journey maps show where friction is frequent and severe, give product, design, operations, marketing and support one shared story, and let you attach a metric to each stage (search-to-menu conversion, payment success rate, actual-minus-promised ETA, first-response time).

Contrasting case. Design thinking is overkill when the problem is already well understood and measurable, such as speeding up a slow API, or when regulation dictates the flow, as with a KYC step. Then you optimise, rather than re-empathise.

In the interview. Design questions ("Design a pharmacy app for elderly users", "Improve Google Maps for two-wheeler riders") are secretly design-thinking questions. Structure the answer the same way: pick one persona and say why; name the goal; walk the journey and find the biggest pain; propose two or three solutions and prioritise; say how you would prototype and what metric would prove it worked. Strong line: *"Before solutioning, I want to pick one user. Elderly users who order refills for chronic conditions have the most frequent, highest-stakes journey, so I'll design for them first and treat caregivers as a secondary persona."* Name a heuristic when critiquing a screen ("this violates recognition over recall") and it signals real UX literacy.

Check yourself

1. Name the seven SDLC stages in order and, for each, one thing the PM does.
2. Give one project where Waterfall is the right choice and explain why in terms of certainty and cost of change.
3. What is a WIP limit, and what does a pile of cards in the "Review" column tell you?
4. Who decides *what* gets built in Scrum, and who decides *how*?
5. Walk through the KJ method's steps. Why should labels be sentences?
6. Pick three of Nielsen's heuristics and give an Indian app example of each, one good and one violated.
7. Why should a journey map cover only one actor and one scenario?
8. (*From the orientation chapter*) In the restaurant model, which component is the "waiter", and which layer of the stack does it belong to?

TECHNOLOGY • THE INTERNET

What happens when you open an app

Addresses, phone books, registered post and sealed envelopes: how a request actually travels, and how apps talk to each other through APIs

Open JioHotstar during an India–Pakistan match. Within a second or two the home screen appears, the score updates, and a live stream starts playing in HD. During the ICC Men's T20 World Cup final in March 2026, JioHotstar reported a peak of **72.5 million concurrent streams** (Reliance Industries / JioStar, 2026). Every one of those phones had to find the right servers, prove the connection was secure, and pull a steady flow of video, all at once.

None of this is magic. It rests on a handful of ideas that are easy to picture once you have the right everyday analogy. This chapter walks one request from your thumb to a server and back.

Clients and servers

A **client** is the thing that asks; a **server** is the thing that answers. Your Swiggy app, your browser and your smart TV are clients. Swiggy's computers in a data centre in Mumbai are servers. A server is not a special kind of machine, just a computer whose job is to wait for requests and answer them, all day, for many clients at once.

Engineers split the work into the **frontend** (what runs on the client: screens, buttons, animations) and the **backend** (what runs on servers: business rules, data, payments). A **native app** is installed from the Play Store or App Store and runs directly on the phone; a **web app** runs inside the browser; a **progressive web app (PWA)** is a website that behaves like an app (installable, works partly offline). Flipkart Lite, a PWA launched in 2015, was built for exactly this reason: many Indian users had low-storage phones and patchy data.

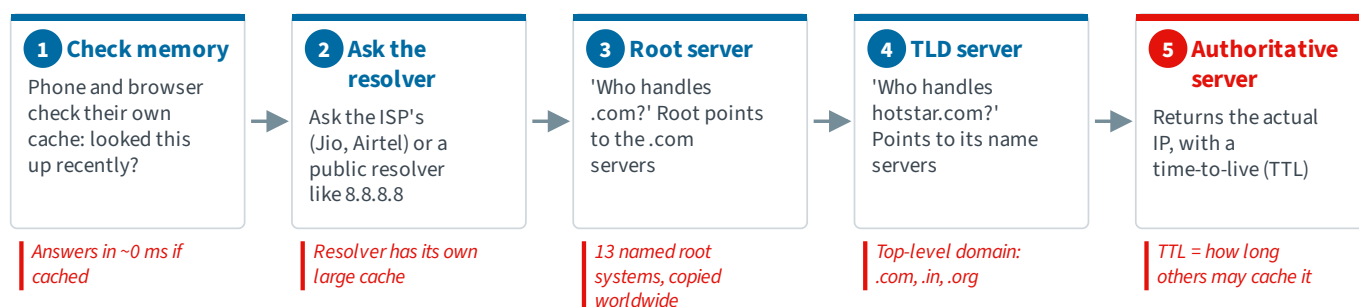
Step 1: finding the server (IP addresses and DNS)

Every device on the internet has an **IP address**, a number that works like a postal address: `142.250.183.14` is one of Google's. The older format, **IPv4**, has about 4.3 billion possible addresses, which the world ran out of. The newer **IPv6** has enough for every grain of sand many times over; India, led by Jio's network, is one of the heaviest users of IPv6 in the world. Within a machine, a **port** number is like the flat number inside the building: web traffic goes to port 443 (HTTPS) or 80 (plain HTTP).

People cannot remember numbers, so we use names like `hotstar.com`. The **Domain Name System (DNS)** is the internet's contacts app: it turns a name into an IP address.

How DNS finds hotstar.com

Asked once, then remembered: most lookups are answered from a cache in step 1 or 2



Why a PM cares. DNS is invisible until it breaks, and then everything breaks. On 19–20 October 2025 a race condition in Amazon's automation left the DNS record for DynamoDB (a core AWS database) in its main US region **empty**. Programs could not find the database even though the database itself was fine; the failure cascaded into EC2, load balancers, Lambda and more, and parts of AWS were disrupted for over 14 hours, taking many consumer apps down with it (AWS post-event summary, Oct 2025). DNS also matters when you migrate or fail over: a short TTL lets you switch traffic quickly, a long one means some users keep going to the old address for hours.

Step 2: sending the data (packets, TCP and UDP)

Data does not travel as one piece. It is chopped into small **packets** (typically about 1,500 bytes each), each labelled with the source and destination address, and each may take a different route through the network. At the other end they are put back together. The rules for this are the **TCP/IP** family of protocols.

Two rules sit on top of IP, and they make opposite trade-offs.

TCP versus UDP

Registered post versus a live radio commentary

TCP: reliable, ordered	UDP: fast, no guarantees
● Analogy Sending a book page by page by registered post. Every page is numbered and acknowledged; missing pages are resent.	● Analogy Live cricket commentary on radio. If you miss a ball, the commentator does not repeat it; he moves on.
● How Starts with a handshake; confirms each packet arrived; resends losses; delivers in order.	● How No handshake, no acknowledgement, no resending. Just send.
● Cost Extra round trips and waiting when a packet is lost.	● Cost Some data is lost or arrives out of order; the app must cope.
● Used for Web pages, APIs, payments, email, chat messages, file downloads.	● Used for Video calls, online gaming, live streams, DNS lookups, and QUIC, the base of HTTP/3.

Choose TCP when every byte must arrive; choose UDP when late data is worse than missing data.

A video call on Google Meet is the classic UDP case: a frame that arrives half a second late is useless, so it is better to skip it and show the next one. A UPI payment request is the opposite: every byte matters, so it runs over TCP.

Step 3: asking for something (HTTP)

With a connection in place, the client asks for what it wants using **HTTP** (HyperText Transfer Protocol). HTTP is a simple request-and-response conversation: the client sends a **request** with a method, an address, some headers and sometimes a body; the server replies with a **status code**, headers and a body.

The **method** says what you want to do: **GET** fetches something, **POST** creates something, **PUT/PATCH** updates it, **DELETE** removes it. The **status code** says how it went. You will see these in bug reports and dashboards, so learn the families.

Code	Meaning	Everyday reading
200 OK / 201 Created	It worked	"Here is your menu" / "Order created"
301 / 302	Moved	"That page lives at a new address now"
304 Not Modified	Use your copy	"Nothing changed since you last asked" (saves data)
400 Bad Request	Client sent something malformed	App bug or bad input
401 Unauthorized / 403 Forbidden	Not logged in / not allowed	Session expired; trying to see someone else's order
404 Not Found	No such thing	Deleted product, broken link
429 Too Many Requests	Slow down	You hit the rate limit
500 Internal Server Error	Server broke	Bug on our side
502 / 503 / 504	Gateway error / unavailable / timeout	Overloaded or a service behind it is down

Rule of thumb: **4xx means the client's side**, **5xx means the server's side**. A dashboard showing 5xx errors spiking during a sale is a backend capacity problem; a spike in 401s right after a release is probably a login bug in the new app version.

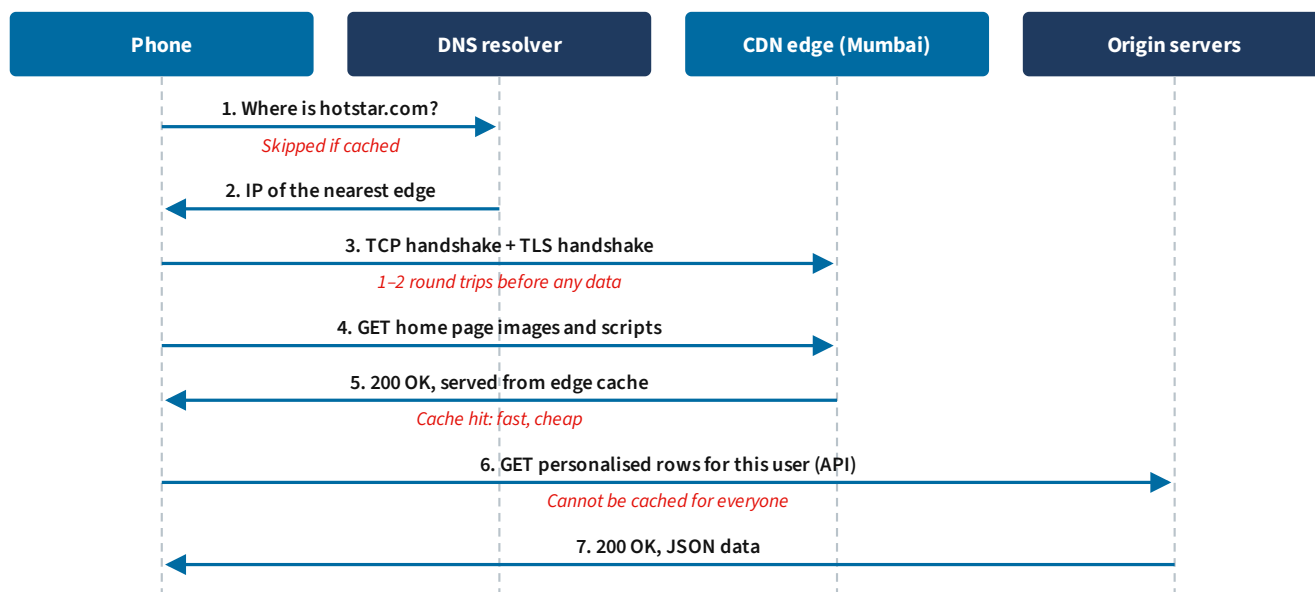
HTTP is **stateless**: each request stands alone, and the server does not remember the previous one. So how does Amazon know you are logged in? The client attaches a **cookie** or a **token** (a signed pass) to every request, like showing your gym membership card at the door every time rather than expecting the guard to remember your face.

Step 4: keeping it private (HTTPS and TLS)

Plain HTTP travels as readable text; anyone on the same café Wi-Fi could read it. **HTTPS** is HTTP inside a **TLS** (Transport Layer Security) tunnel. TLS does three jobs: it **encrypts** the data so only the two ends can read it; it **authenticates** the server with a **certificate**, a digital ID card signed by a trusted certificate authority, so you know you are really talking to your bank and not an impostor; and it **checks integrity**, so nobody can alter the data in transit. The padlock in the browser means all three are in place.

One page load, end to end

Each arrow crossing the network costs at least one round trip; a round trip Mumbai–US East is ~200 ms, Mumbai–Chennai ~25 ms (illustrative)



What HTTPS does *not* do: it protects data **in transit**, not data sitting on the server ("at rest"), and it does not stop a server that is itself hacked or dishonest. Encryption at rest, access controls and audits cover those (see the security section of the system-design chapter).

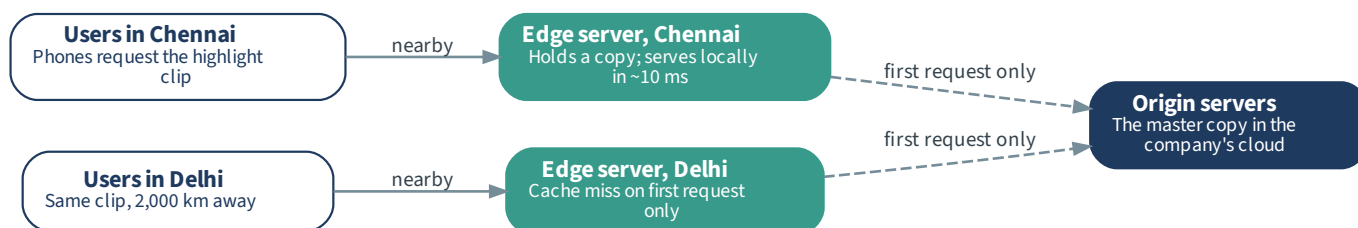
Step 5: bringing content closer (CDNs)

The speed of light in optical fibre is about 200,000 km per second. Mumbai to a data centre in Virginia is roughly 13,000 km each way, so physics alone sets a floor of about 130 ms for a round trip, and real networks roughly double it. A page that needs ten round trips would feel sluggish. You cannot make light faster, so you move the content closer.

A **Content Delivery Network (CDN)** is a network of servers spread across cities, each keeping copies of content. The user is routed to the nearest one, the **edge**. Only if the edge does not have the content (a **cache miss**) does it fetch it from the company's main servers, the **origin**, and keep a copy for the next person.

How a CDN serves a cricket highlight to users across India

Popular files are fetched from the origin once, then served locally thousands of times



Netflix built its own CDN, **Open Connect**, and places its caching boxes inside ISPs' networks for free. As early as 2016 it said close to 90% of its traffic was delivered over direct connections between Open Connect and residential ISPs (Netflix, March 2016). When you press play in Pune, the film may be coming from a Netflix box in your ISP's building, not from the US.

Why a PM cares. CDNs cut **latency** (faster pages mean more conversions), cut **cost** (serving from the edge is cheaper than sending everything from the origin, and cloud providers charge for data leaving their network), and absorb **spikes** (a viral sale banner is served from hundreds of edges, not one server). The trade-off is **freshness**: cached copies can be stale. Prices and stock should not be cached for long; logos and product photos can be cached for days. Deciding what is safe to cache, for how long, is partly a product decision.

The one-line rule

A request is a trip: find the address (DNS), open a reliable line (TCP), secure it (TLS), ask (HTTP), and fetch from the nearest copy (CDN). Every round trip costs time, so good systems remove trips and move content closer to the user.

In the interview. "What happens when you type a URL and press enter?" is a classic warm-up, even for PMs. Walk the five steps above in plain words, then add one product insight: *"The first visit is slow because of DNS, TCP and TLS set-up, so for a flash sale I'd pre-warm the CDN with banner images and ask whether we can keep connections alive between app launches."* Other forms: "Why is our app slow for users in Tier-3 towns?" (distance to servers, few CDN edges, 3G networks, heavy images; measure by region and network type), "Should our live quiz use TCP or UDP?" (answers must arrive reliably: TCP, or WebSockets over TCP).

TECHNOLOGY · APIS

APIs: how software talks to software

The waiter, the menu and the doorbell: REST, webhooks, WebSockets, rate limits, and why an API is itself a product

When you book an Ola ride, the map comes from a mapping provider, the payment goes through a payment gateway to NPCI's UPI switch, the OTP arrives by an SMS provider, and the driver's phone gets a push notification through Google's or Apple's servers. None of these companies share code. They talk through **APIs**.

An **API (Application Programming Interface)** is an agreed way for one program to ask another for something. In the restaurant model, the API is the **waiter** and its documentation is the **menu**: you can only order what is listed, in the way it is listed, and you do not need to know how the kitchen works. The kitchen can change its stoves, recipes or staff, and as long as the menu stays the same, diners notice nothing. That separation is the whole point.

APIs can be **private** (a company's own app talking to its own backend), **partner** (Swiggy's restaurant partners' billing systems sending menu updates) or **public** (Google Maps, Razorpay, OpenAI). India's digital public infrastructure is largely a set of public API standards: **UPI** for payments, **Aadhaar eKYC** for identity, **DigiLocker** for documents, the **Account Aggregator** framework for sharing financial data with consent, and **ONDC** for open commerce. Any licensed player can plug in, which is why dozens of apps can offer UPI.

REST: the most common API style

Most APIs you will meet follow a style called **REST**. The idea: everything is a **resource** with an address (a noun), and you act on it with the standard HTTP methods (verbs). Data usually travels as **JSON**, a simple text format of labelled values.

What the app wants	REST request	Server replies
List biryani places in Bengaluru	GET /restaurants?city=blr&dish=biryani	200 + list of restaurants
See one restaurant's menu	GET /restaurants/4521/menu	200 + menu items
Place an order	POST /orders with cart in the body	201 Created + order ID
Change delivery instructions	PATCH /orders/98812	200 + updated order
Cancel the order	DELETE /orders/98812 (or a cancel action)	200, or 409 if already picked up

A reply might look like this:

```
{ "order_id": 98812, "status": "PREPARING", "eta_minutes": 28, "rider": { "name": "Ravi",
"lat": 12.9352, "lng": 77.6245 } }
```

REST is popular because it is simple, readable and easy to debug. Its weakness is **over-fetching** (getting fifty fields when the screen needs three, which wastes mobile data) and **under-fetching** (needing five calls to build one screen). Two alternatives address this:

- **GraphQL** (created at Facebook) lets the client ask for exactly the fields it needs in one request. Good for complex, data-heavy screens.
- **gRPC** (created at Google) sends compact binary messages over HTTP/2. Faster and lighter, but not human-readable; mostly used for service-to-service calls inside a company's backend.

How does the app hear about changes?

HTTP is "ask and answer": the server cannot speak unless spoken to. But a food-delivery app needs to know when the restaurant accepts the order, and a chat app needs to show a new message instantly. There are four ways to solve this, and choosing between them is a real product trade-off between freshness, battery, and server cost.

Four ways to learn that something changed

From the child asking 'are we there yet?' to the driver who calls you when he arrives

Polling

Client asks every few seconds: 'Any update?' Simple, but wasteful: most answers are 'no', and it drains battery and servers.

Long polling

Client asks, server holds the question open until there is news or a timeout. Fewer empty answers; still one request per update.

WebSockets

One connection stays open and both sides can talk any time. Best for chat, live scores, rider tracking, collaborative editing.

Webhooks

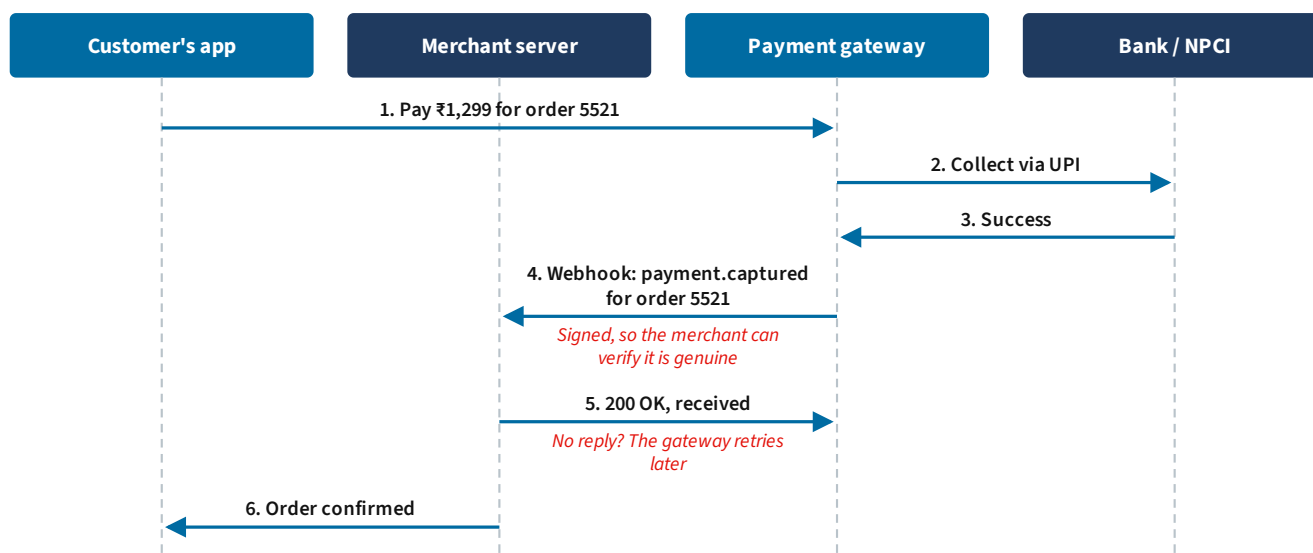
Server-to-server doorbell: 'Call this URL when X happens.' Razorpay calls the merchant's server when a payment succeeds.

Push notifications are a fifth path for when the app is closed: your server sends the message to Apple's (APNs) or Google's (FCM) push service, which wakes the phone. You do not control delivery timing or guarantees; the OS does.

A **webhook** deserves a closer look because it appears in almost every payments and SaaS integration.

A payment webhook

The merchant's site does not keep asking 'did they pay?'; the gateway rings the doorbell



Because webhooks are retried, the same message may arrive twice. The merchant must therefore process it **idempotently**: doing it twice must have the same effect as doing it once (do not ship two parcels). Idempotency comes up again in the UPI walkthrough in the system-design chapter.

Rate limits: the queue manager at the counter

Every API has a capacity, and some callers are greedy or malicious. A **rate limit** caps how many requests one caller may make in a time window, for example 100 per minute per user. Past the limit, the API replies **429 Too Many Requests**. A common mechanism is the **token bucket**: each caller has a bucket that refills with tokens at a steady rate; each request spends a token; an empty bucket means wait. This allows short bursts while capping the average.

IRCTC is the Indian textbook case. When Tatkal bookings open at 10 am, genuine users compete with bots and touts. IRCTC has combined rate limiting and bot detection with identity checks: Tatkal bookings on its website and app need Aadhaar-authenticated accounts from 1 July 2025, with Aadhaar OTP added from 15 July 2025 (PIB, June 2025); it deactivated over 3 crore suspicious user IDs and reports its anti-bot measures cut malicious bot traffic by 55–65% (Business Today, Sep 2026).

API vocabulary

Endpoint: one address an API answers at (/orders). **Request / response**: the question and the answer. **JSON**: the common text format for data. **API key / token**: the caller's identity pass. **Rate limit**: maximum calls per time window; 429 when exceeded. **API gateway**: the front door that checks keys, applies rate limits and routes calls to the right internal service. **Versioning**: keeping /v1/ working while /v2/ launches, so old apps do not break. **SDK**: a ready-made code library that wraps an API for developers. **Idempotent**: safe to repeat; doing it twice equals doing it once. **Latency**: time for one call; **throughput**: calls handled per second.

Contrasting case. Tight coupling through APIs creates dependency risk. If your app calls a third-party address-lookup API on the checkout page and that provider goes down, your checkout goes down. Good design adds a **timeout** (stop waiting after 800 ms), a **fallback** (let the user type the address manually) and sometimes a second provider.

Why a PM cares.

- **APIs can be the product**. Stripe, Razorpay, Twilio and the AI model providers sell APIs; their "UX" is documentation, SDKs, error messages and sandbox testing. Developer experience decides adoption.
- **Old app versions live for years**. Many users never update. Any change to an API must stay **backward compatible** or be versioned, or you break the app on their phones. Ask "which app versions does this affect?"
- **Partner and platform strategy runs on APIs**. Opening an API invites an ecosystem (Shopify's apps, UPI's third-party apps) but also creates obligations: uptime promises, support, security reviews and rate-limit policy.
- **Rate limits are a pricing tool**. Free tier: 1,000 calls a day; paid tier: more. Many AI and SaaS products price exactly this way.

The one-line rule

An API is a promise: "ask me this way and I will answer that way". Keep the promise stable, limit who can call it how often, and plan for the day the other side does not answer.

In the interview. "Explain an API to a 10-year-old" (the waiter), "How would Zomato show live order status?" (WebSockets while the app is open, push notifications when it is closed, polling as a fallback on poor networks), "A partner wants to integrate with our platform; what do you consider?" (use cases, authentication, rate limits, versioning, SLA, pricing, abuse, data privacy). A strong line: "I'd treat the partner API as a product with its own users, developers, and its own success metric: time to first successful call."

Check yourself

1. Trace a request from typing `hotstar.com` to seeing the page, naming the five steps.
2. Why did an empty DNS record take down services whose databases were perfectly healthy in the October 2025 AWS outage?
3. Give one product that should use UDP and one that must use TCP, and explain why.
4. What is the difference between a 4xx and a 5xx error? Which one points to a capacity problem?
5. What does HTTPS protect, and what does it not protect?
6. Polling, WebSockets, webhooks: which would you use for (a) live rider tracking, (b) telling a merchant a payment succeeded, (c) a weather widget that refreshes hourly?
7. Why must webhook handling be idempotent?
8. (From the SDLC chapter) At which SDLC stage should you decide whether an API change must stay backward compatible, and who needs to be involved?

TECHNOLOGY • SYSTEM DESIGN

Building for a million people at 10 am

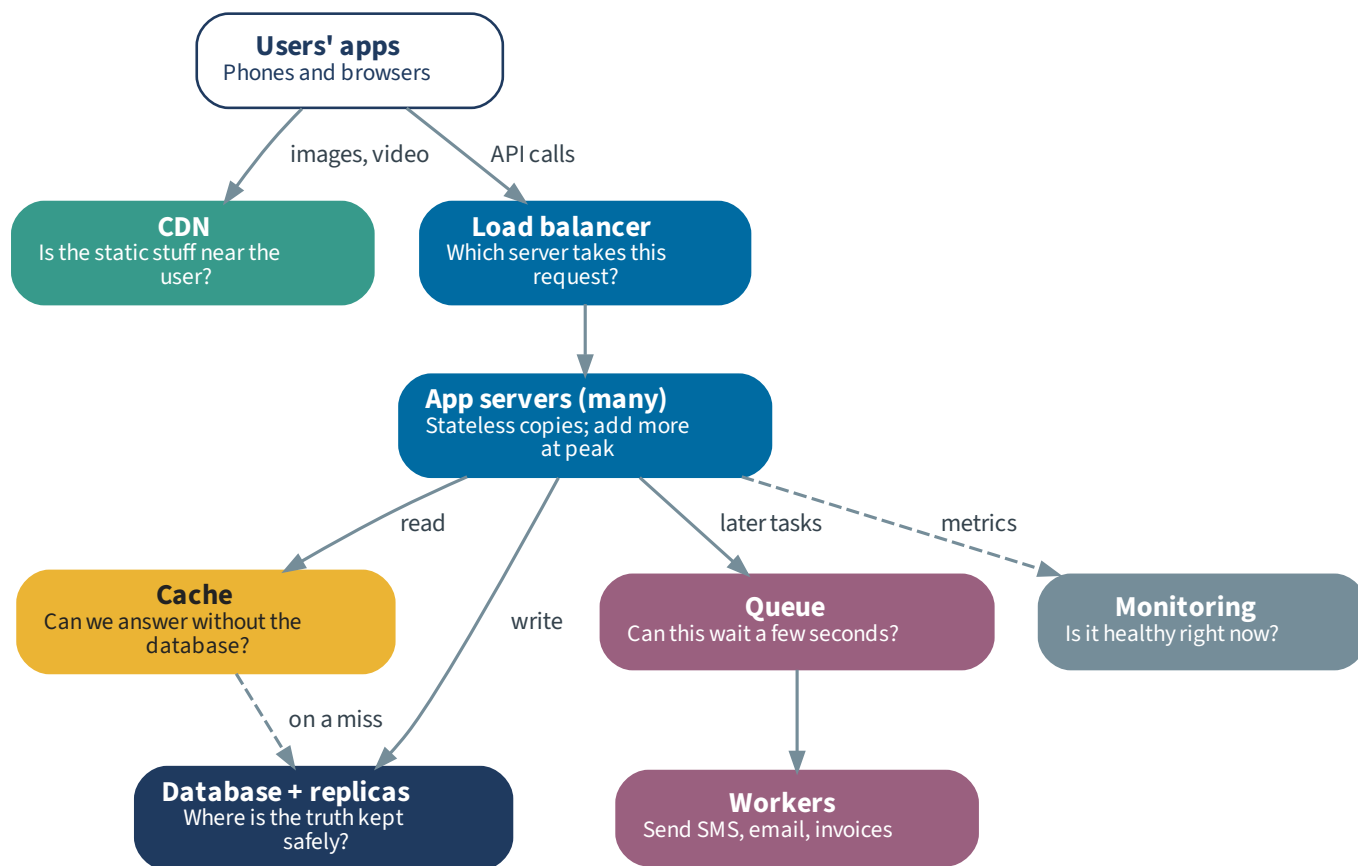
The building blocks engineers use to make products fast, reliable and affordable at scale, explained with the trade-offs a PM has to call

At 9:59 am, IRCTC is quiet. At 10:00 am, Tatkal booking opens for AC classes and a large share of the day's most wanted seats are fought over in the next few minutes. IRCTC's own data shows the highest booking volume consistently falls in the first 15 minutes, and on 7 September 2026 it set a record of **20,06,353 tickets in a single day**, after roughly doubling capacity with new servers (Business Today, Sep 2026). A system that is perfectly comfortable at 9:59 has to survive a wall of traffic one minute later, without selling the same berth twice.

That is the whole of system design in one scene: **many users, spiky demand, data that must stay correct, and a budget**. This chapter explains the building blocks engineers use, one at a time, with the question each one answers and the trade-off each one forces. The next chapter puts them together in three worked designs.

The building blocks, and the question each one answers

Keep this map in mind; each block gets its own section below



How fast is fast? An intuition for latency

Before the blocks, one intuition. Computers do some things unimaginably fast and others painfully slowly, and good design is mostly about avoiding the slow things. Stretch time so that one nanosecond becomes one second, and the differences become human.

If one nanosecond were one second

Typical operation times (order of magnitude), rescaled to human time

Real operation	Stretched time
Reading from the CPU's own cache: 1 ns	1 second Glancing at a note in your hand
Reading from memory (RAM): ~100 ns	About 2 minutes Walking to the next room for a file. Caches like Redis live here.
Reading from an SSD: ~100 µs	About a day Ordering the file from another city. A database read that misses memory.
Round trip inside one data centre: ~0.5 ms	About 6 days One service calling another. Ten such calls add up.
Seek on an old spinning disk: ~10 ms	About 4 months Why databases work hard to avoid disk reads
Round trip Mumbai to US East: ~200 ms	About 6 years Why servers and CDNs must be near Indian users

Source: Based on 'latency numbers every programmer should know' (Jeff Dean, Peter Norvig), updated orders of magnitude; illustrative

On the user's side, the classic research-based thresholds still hold: about **0.1 second** feels instant, about **1 second** keeps the user's flow of thought, and past about **10 seconds** you lose their attention (Jakob Nielsen, *Usability Engineering*, 1993). Google reported in 2016 that 53% of mobile site visits were abandoned when a page took longer than three seconds to load.

Latency terms you will hear in reviews

Latency: time to answer one request. **Throughput:** requests handled per second (RPS or QPS; TPS for transactions). **p50 / p95 / p99:** the latency that 50%, 95% or 99% of requests beat. An average hides pain: if p50 is 200 ms but p99 is 4 seconds, one user in a hundred waits four seconds, and your heaviest users make the most requests, so they hit it most often. **Latency budget:** the total time allowed for a screen, split across the services it calls.

Why a PM cares. You set the budget. "Search results in under 300 ms at p95 on 4G" is a requirement engineers can design for; "make it fast" is not. Every extra service call, round trip or heavy image spends from that budget.

Databases: where the truth lives

A restaurant keeps two kinds of records. The **order register** must be exact: every bill numbered, every payment matched, nothing lost. The **ideas board** in the kitchen, with notes on what customers liked, can be messy and change shape every week. Databases come in the same two broad kinds.

Relational (SQL) databases store data in tables with fixed columns, like a set of linked spreadsheets: a **users** table, an **orders** table, a **restaurants** table, joined by IDs. You query them with **SQL** (Structured Query Language). Examples: PostgreSQL, MySQL, Oracle. Their superpower is correctness, summed up as **ACID**:

ACID, explained with a ₹500 bank transfer

Atomicity

All or nothing. Debit Asha ₹500 and credit Ravi ₹500 both happen, or neither does. No money vanishes halfway.

Consistency

Rules always hold. A balance can never go below zero if the rules say so; a broken rule cancels the transaction.

Isolation

Simultaneous transactions don't trip over each other. Two people booking the last berth: only one wins.

Durability

Once confirmed, it stays confirmed, even if the power fails one second later.

Non-relational (NoSQL) databases relax the fixed structure, and often some consistency, in exchange for flexibility and easy scaling across many machines. They come in several shapes:

The main kinds of NoSQL store, and what they are good at

Key-value

Look up a value by a key, extremely fast. Sessions, carts, counters. Redis, DynamoDB.

Document

Store flexible JSON-like records; fields can vary. Product catalogues, user profiles. MongoDB.

Wide-column

Huge volumes of writes spread across many machines. Chat messages, event logs, IoT readings. Cassandra.

Graph

Data that is mostly relationships. Social graphs, fraud rings, 'people you may know'. Neo4j.

Search index

Full-text search with typo tolerance and ranking. Product search. Elasticsearch, OpenSearch.

In-memory

Everything in RAM for sub-millisecond reads; used as a cache or for live leaderboards. Redis, Memcached.

	SQL (relational)	NoSQL (non-relational)
Shape	Tables with a fixed schema	Flexible: key-value, documents, columns, graphs
Strength	Strong consistency, transactions, complex queries and joins	Scale-out across many machines, fast simple lookups, flexible fields
Weakness	Harder to spread across many machines; schema changes need planning	Weaker guarantees; complex queries and joins are harder
Use it for	Payments, orders, inventory, bookings, anything involving money	Feeds, chat history, catalogues with varied fields, sessions, analytics events

Real products use several at once, each for what it does best. That is called **polyglot persistence**. A food-delivery app might keep orders and payments in PostgreSQL, restaurant menus in a document store, the search box on Elasticsearch, live rider locations in

Redis, and every tap event in a data warehouse for analytics.

Indexes: the index at the back of the book

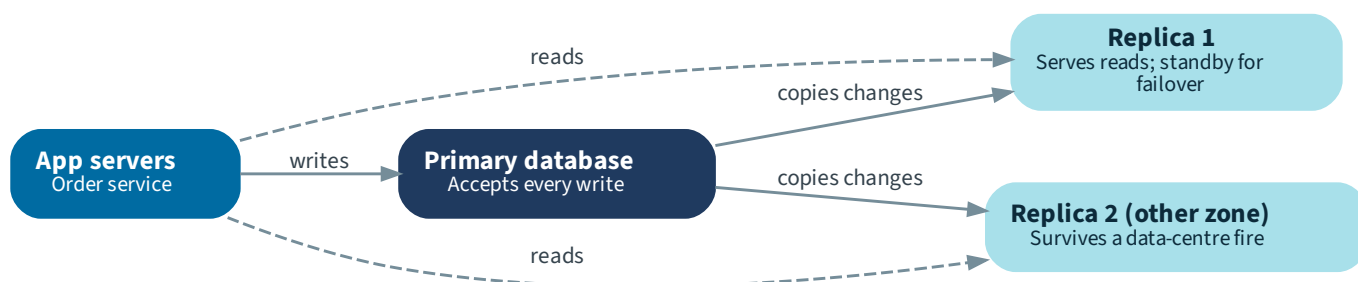
Finding every order by customer 55821 in a table of a billion orders by reading every row would take minutes. An **index** is like the index at the back of a textbook: a sorted list that points straight to the right pages. Queries on indexed columns become thousands of times faster. The price: every new row must also update the index, so writes get a little slower and storage grows. A surprising number of "the app got slow" incidents end with "someone added a query on a column without an index".

Replication: photocopies in every branch

A single database is a **single point of failure**: if that machine dies, the business stops. **Replication** keeps live copies on other machines. Usually one **primary** accepts all writes and streams changes to several **replicas**, which serve reads. If the primary fails, a replica is promoted.

Primary-replica replication

Writes go to one place; reads are spread across copies



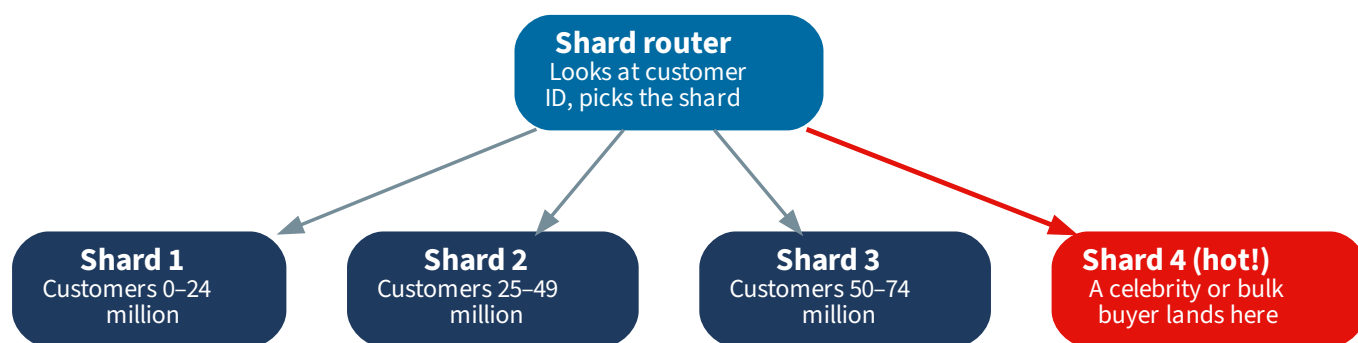
The catch is **replication lag**. Copies take a few milliseconds to seconds to catch up. That is why you sometimes change your delivery address, go back a screen, and still see the old one for a moment: your write went to the primary, your read came from a replica that had not caught up. Good products hide this ("read your own writes" from the primary for a few seconds after a change).

Sharding: splitting the phone directory into volumes

Replication makes copies of the *same* data. When the data itself is too large, or there are too many writes for one machine, you **shard** (or partition) it: split the data into pieces, each on its own machine. Like splitting a city's phone directory into volumes A–F, G–M, N–S and T–Z.

Sharding orders by customer ID

A router sends each request to the machine that holds that customer's data



The design choice is the **shard key**. Shard by customer and "show this customer's orders" is fast, but "all orders for restaurant X" must ask every shard. Pick badly and you get a **hot shard**: one machine overloaded while others idle, like the queue for the volume containing "Sharma" and "Singh". Sharding is powerful and painful; teams postpone it until they must.

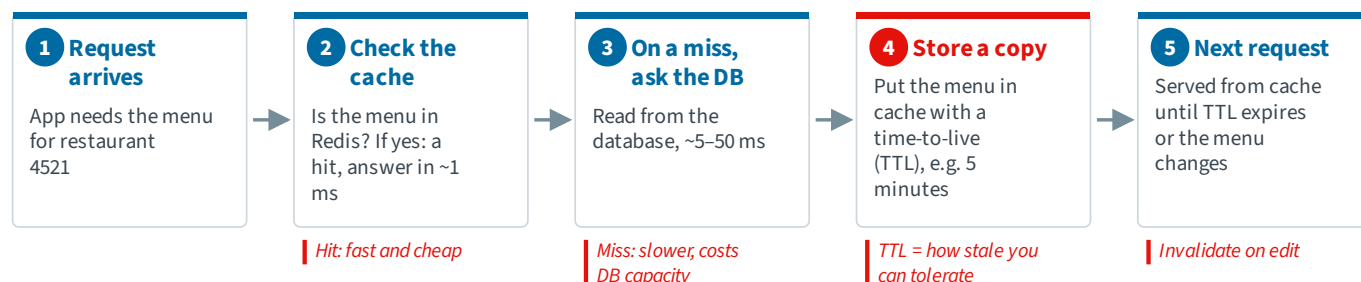
Why a PM cares. Database choices shape which features are cheap and which are expensive. "Show all orders across all restaurants for a city, live" might be trivial or might need a new analytics pipeline, depending on how the data is stored. Ask before you promise it.

Caching: keep the popular dishes warm

A busy dhaba does not cook dal from scratch for every order. It keeps a big pot hot, because most customers order it. A **cache** keeps copies of frequently requested data in fast memory so most requests never reach the slower database.

How a read-through cache answers a request

The goal is a high hit rate; 95% is common for menus and product pages



Caches exist at every layer: in your browser, on the CDN edge, in the app servers' memory, in a shared store like Redis, and inside the database itself. The hard part is **invalidation**: making sure a changed price or an out-of-stock item is not served from a stale copy. An old engineering joke says there are only two hard things in computer science, cache invalidation and naming things.

A second danger is the **stampede** or **thundering herd**. If a popular cache entry expires at the exact moment a million people ask for it, all million requests fall through to the database at once. Hotstar's engineers faced this during cricket: a big wicket sends millions of viewers back to the home screen at the same second. Their defences included multi-level caching, adding random **jitter** to retries and **exponential back-off** in the app, so requests spread out instead of arriving in one wave (systemdesign.one, on Hotstar's 2019 IPL scaling).

Why a PM cares. Caching is the cheapest performance win, but it trades **freshness for speed**. You decide the tolerance: a restaurant rating can be 10 minutes stale; stock in a 10-minute grocery app should be seconds fresh; a payment status must never come from a stale cache.

Load balancing and scaling

When one server cannot handle the traffic, you have two options, and the difference is one of the most-asked PM interview concepts.

Vertical versus horizontal scaling

Vertical: scale up	Horizontal: scale out
Analogy Replace your delivery van with a bigger truck.	Analogy Add more vans, and a dispatcher to share the orders.
How Buy a bigger machine: more CPU, more memory.	How Run many identical, ordinary servers behind a load balancer.
Good Simple; no code changes; fine for databases up to a point.	Good Nearly unlimited; if one dies, others carry on; add or remove servers by the hour.
Limits There is a biggest machine; it is expensive; still one point of failure; upgrading often needs downtime.	Limits Servers must be stateless; data layer gets harder; more moving parts.

Modern consumer apps scale app servers horizontally and scale databases carefully, often both ways.

A **load balancer** is the dispatcher, or the host at a busy restaurant seating guests across sections. It sits in front of the servers, sends each request to one that has capacity (in turn, **round robin**; to the one with the fewest active connections; or by the user's region), runs **health checks** every few seconds, and stops sending traffic to any server that fails them. For this to work, app servers must be **stateless**: they keep nothing about the user in their own memory between requests (sessions live in a shared cache or a token), so any server can handle any request.

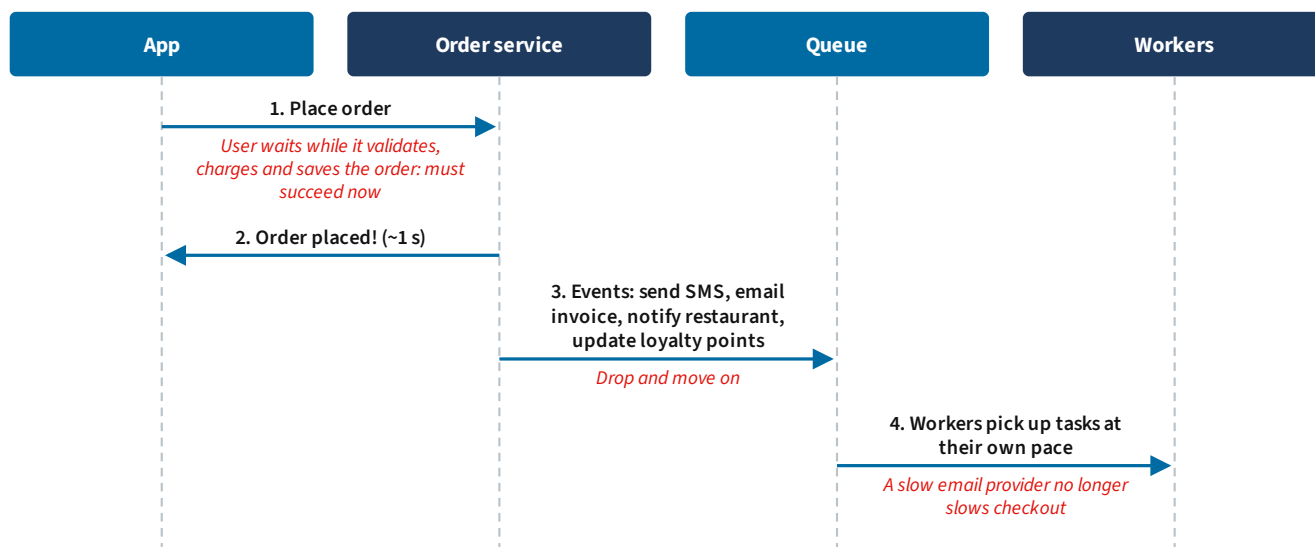
Auto-scaling adds servers when load rises and removes them when it falls, so you pay for dinner-time capacity only at dinner time. But new servers take time to start (tens of seconds to minutes). For a spike that arrives in seconds, that is too slow. Hotstar learned this during IPL: rather than rely on auto-scaling, it **pre-warmed** capacity ahead of big matches based on forecasts from past traffic, and built its own scaling logic because the standard tools reacted too slowly (systemdesign.one, 2023). IRCTC faces the same at 10 am daily. Predictable spikes should be scaled for *in advance*.

Queues and asynchronous processing: the token system

At a busy bank branch you take a token and sit down. The counter serves tokens in order at the speed it can manage, and the crowd does not crush the clerk. A **message queue** does this for software. One part of the system (the **producer**) drops a task in the queue and moves on; another part (the **consumer** or worker) picks tasks up when it has capacity.

What happens synchronously versus asynchronously when you place an order

The user waits only for the part that must be instant; everything else is queued



Queues give you three things. **Speed for the user**, because slow tasks happen after the response. **Spike absorption**, because a burst of 50,000 orders waits in the queue instead of overwhelming the invoice service. **Resilience**, because if the SMS provider is down, messages wait and are retried rather than lost; tasks that fail repeatedly go to a **dead-letter queue** for humans to inspect. The price is that things become **eventually** done rather than instantly done, and the user experience must say so honestly: "Your refund has been initiated", "Booking under process".

A related tool is an **event stream** such as Kafka: a durable log of everything that happened ("order placed", "rider assigned", "payment failed") that many services can read independently. Analytics, fraud detection and notifications can all react to the same events without the order service knowing they exist.

Monolith or microservices?

A **monolith** is one big application that does everything: login, catalogue, cart, payments, notifications, all in one codebase and deployed as one unit. **Microservices** split it into many small services, each owning one job and its own data, talking through APIs or events.

Monolith versus microservices

One big kitchen versus a food court of specialised counters

Monolith	Microservices
● Good Simple to build, test and deploy early on; one codebase to understand; fast calls inside one program.	● Good Teams own services end to end and ship independently; scale only the busy parts; a failure can be contained.
● Bad As the team grows, everyone steps on each other; one bug can crash everything; must scale the whole thing even if only search is busy.	● Bad Many network calls (latency, failures); harder debugging; data spread across services; needs strong tooling and on-call culture.
● Fits Startups, small teams, a new product finding its fit.	● Fits Large organisations with many teams and uneven load, like Swiggy, Flipkart, Uber.

Most companies should start as a well-organised monolith and split out services when team size or scaling pain demands it.

A useful observation called **Conway's law** says systems end up mirroring the communication structure of the organisation that builds them. Microservices are as much an organisational choice (small, autonomous teams) as a technical one. When a PM hears "that needs changes in four services owned by three teams", that is the cost of microservices showing up in the roadmap.

Consistency versus availability (CAP in plain words)

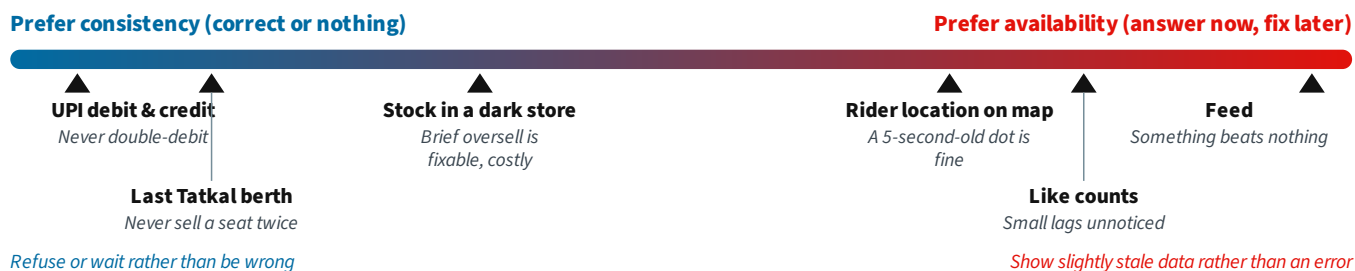
Imagine a bank with two branches, in Mumbai and Pune, keeping copies of the same ledger and phoning each other after every transaction. One day the phone line is cut. A customer walks into the Pune branch and wants to withdraw ₹10,000. The Pune clerk has two choices:

- **Refuse until the line is back**, because the Mumbai branch may have already paid this customer. The ledger stays correct, but the customer is turned away. That is choosing **consistency**.
- **Pay out anyway** and reconcile later. The customer is served, but the two ledgers may now disagree. That is choosing **availability**.

The **CAP theorem** (Eric Brewer, 2000) formalises this: when a distributed system suffers a network **partition** (P), which in real networks will happen, it must choose between **consistency** (C, everyone sees the same, latest data) and **availability** (A, every request gets an answer). You cannot have both during the split. Even without failures there is a quieter version of the same trade: waiting for all copies to agree adds latency.

Which way should each feature lean?

The same app makes different choices for different data



Strong consistency means every read sees the latest write. **Eventual consistency** means copies will agree soon, typically within seconds, if no new changes arrive. Most large products are eventually consistent almost everywhere and strongly consistent exactly where money, inventory or safety is involved.

The one-line rule

When the network splits, choose between being right and being responsive. Money and scarce inventory choose right; feeds, counts and locations choose responsive. Naming which data belongs where is a product call.

Reliability: designing for things to break

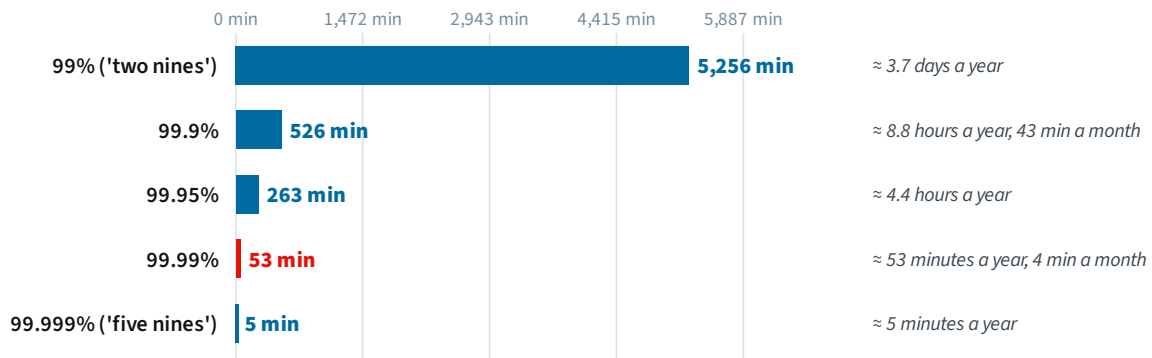
Every car has a spare tyre, and planes have two engines, because parts fail. At scale, something is *always* failing: a disk, a server, a network link, a whole data centre. Reliable systems assume this.

Redundancy means having more than one of everything that matters: several app servers, replicated databases, servers in more than one **availability zone** (separate data centres in the same region), and for critical systems, more than one region. The enemy is a **single point of failure (SPOF)**, any component whose loss stops everything.

Availability is usually stated in "nines". Each extra nine cuts allowed downtime by ten times, and typically costs much more to achieve.

What each level of availability actually allows

Maximum downtime per year



Source: Arithmetic: $(1 - \text{availability}) \times 525,600$ minutes

Three terms are often confused:

SLI, SLO, SLA: measure, target, promise

Example: a payments API



The gap between 100% and the SLO is the **error budget**. At 99.95%, you may "spend" about 22 minutes of failure a month. While budget remains, teams ship features quickly; when it is spent, they freeze risky launches and fix reliability. This turns the eternal "features versus stability" argument into a number both PM and engineers agreed on in advance. It comes from Google's Site Reliability Engineering (SRE) practice.

Graceful degradation means that when part of the system is struggling, the product gets simpler instead of going dark. Hotstar's "panic mode" switched off non-essential features such as personalised recommendations during extreme load so the stream itself kept playing, and streams drop to a lower bitrate rather than stop. A food app under stress can hide "similar restaurants" and keep checkout working. The PM's job is to decide in advance what is **core** (must never break) and what is **nice to have** (can be switched off).

A few more defences you will hear about:

- **Timeouts:** stop waiting for a slow service after a fixed time instead of hanging forever.
- **Retries with exponential back-off and jitter:** try again after 1 s, then 2 s, then 4 s, with a little randomness. Retrying immediately and in unison creates a **retry storm** that finishes off a struggling service.
- **Circuit breaker:** like the fuse in your house. If a dependency keeps failing, stop calling it for a while and use a fallback, so one broken service does not drag others down.
- **Disaster recovery:** backups and a second region, measured by **RPO** (how much data you can afford to lose, e.g. the last 5 minutes) and **RTO** (how long until you are back, e.g. 1 hour).

Observability: the dashboard of the car

You would not drive at night without a speedometer, fuel gauge and warning lights. **Observability** is how engineers see inside a running system.

The three pillars of observability, and the four golden signals

Metrics Numbers over time: requests per second, error rate, p95 latency, CPU. Cheap to store; power dashboards and alerts.	Logs A diary line for each event: 'order 98812 payment failed: bank timeout'. Detailed; used to investigate.	Traces One request's journey across every service it touched, with time spent in each. Finds the slow link.	Latency How long requests take (watch p95/p99).
Traffic How much demand there is.	Errors How many requests fail.	Saturation How full the system is: CPU, memory, queue length.	

The four golden signals come from Google's SRE book. On top of these, product teams add **business health metrics** that often catch problems before technical ones do: orders per minute, payment success rate, login success rate, compared with the same minute last week. If orders drop 30% at 8 pm on a Friday, something is broken even if every server looks green.

When something does break, companies run an **incident** process: detect, declare severity, communicate (status page, support scripts), mitigate (roll back, switch off the feature, fail over), then write a **blameless postmortem** that asks "what about the system allowed this?" rather than "who did it?". PMs usually own customer communication and the decision on what to switch off.

Security basics every PM should know

Security failures are product failures: they break the user's trust in one day. You need five ideas.

Authentication (AuthN) versus authorisation (AuthZ). A hotel front desk checks your ID (authentication: *who are you?*) and gives you a key card that opens only your room and the gym (authorisation: *what may you do?*). Login is authentication; "this restaurant owner can edit only their own menu" is authorisation. Stronger authentication uses a second factor (OTP, authenticator app) or **passkeys**, which replace passwords with cryptographic keys stored on your phone. "**Sign in with Google**" uses the **OAuth** standard: like giving a valet a key that starts the car but cannot open the boot, the app gets limited, revocable access without ever seeing your Google password.

Encryption in transit and at rest. HTTPS protects data moving across the network; encryption at rest protects data stored on disks and backups. Passwords are never stored at all, only a one-way **hash** of them, so even a stolen database does not reveal them. In India, RBI's card-on-file **tokenisation** rules (effective October 2022) mean merchants store a token in place of your actual card number.

The most common holes. The OWASP Top 10, maintained by the Open Worldwide Application Security Project, lists the most frequent web-application risks. In plain words:

The web's most common security holes, in plain words

Broken access control Change order ID 98812 to 98813 in the URL and see a stranger's address. The most common serious flaw.	Injection Typing code into a search box that the server runs against the database.	Security misconfiguration A storage bucket of customer KYC files left publicly readable.
Vulnerable components Using an old open-source library with a known hole.	Authentication failures No limit on OTP attempts, so bots can guess all 10,000 codes.	Cryptographic failures Sensitive data sent or stored without proper encryption.

Source: Based on OWASP Top 10 categories; examples illustrative

Abuse at the front door. A DDoS (distributed denial-of-service) attack floods you with junk traffic from thousands of machines; CDNs and specialised services absorb it. **Rate limiting** and **bot detection** stop credential stuffing (trying leaked passwords), OTP

bombing and scalping (the IRCTC Tatkal problem). A **WAF** (web application firewall) filters malicious requests.

Least privilege and data minimisation. Give every person and service the minimum access needed, and do not collect data you do not need. India's **Digital Personal Data Protection Act, 2023**, with its Rules notified in November 2025 and obligations phasing in over roughly 18 months, makes consent, purpose limitation, breach reporting and deletion legal requirements, not just good practice.

Why a PM cares. Security is a trade against friction. An OTP on every login is secure and annoying; no OTP is smooth and dangerous. Good products use **risk-based** friction: step-up checks only for a new device, a large transfer or a changed bank account.

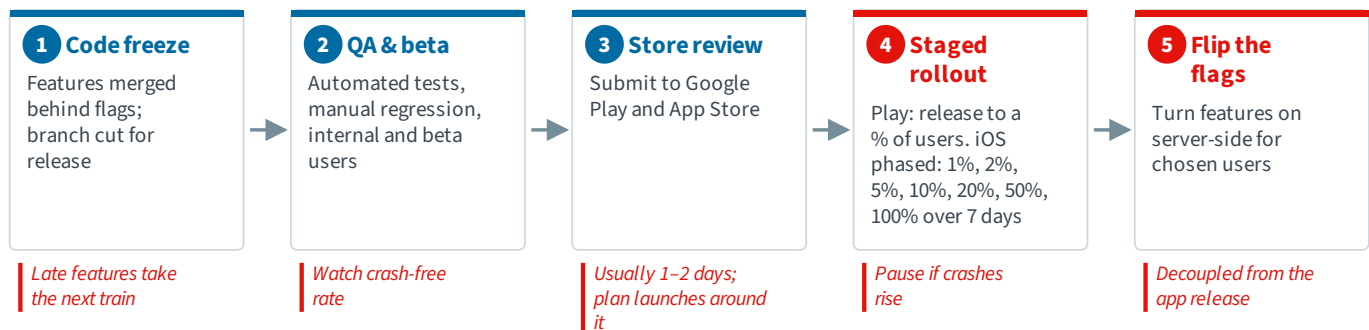
Shipping safely: releases and feature flags

Back-end code on your own servers can be changed several times a day. Mobile apps cannot. A new Android or iOS version must be built, reviewed by Google or Apple (usually within a day or two, occasionally longer), downloaded by users, and some users will not update for months. A bug in the app on people's phones cannot simply be rolled back.

Teams therefore ship mobile apps on a **release train**: a fixed cadence (often weekly or fortnightly) where whatever is ready boards the train, and whatever is not waits for the next one, instead of delaying the whole release.

A typical fortnightly mobile release train

Illustrative cadence; each stage is a gate where a PM can stop the train



The most powerful tool here is the **feature flag**: a switch in the code, controlled from a server, that decides whether a feature is on or off for a given user. With flags you can:

- **Decouple deploy from release**: the code ships in the app on Monday, the feature turns on for users on Thursday when marketing is ready.
- **Roll out gradually**: 1% of users, then 10%, then 50%, watching metrics at each step; this is also how A/B tests are run.
- **Kill switch**: turn off a misbehaving feature in seconds, without a new app release.
- **Target**: enable for one city, beta users, or employees first (**dogfooding**).

On the server side, the equivalents are **canary releases** (send 1–5% of traffic to the new version and compare its error rate with the old one), **blue-green deployments** (run old and new side by side and switch traffic over, with instant switch-back) and **CI/CD** (continuous integration and delivery: every code change is automatically built and tested, and can be deployed within minutes).

The cautionary tale is **CrowdStrike, 19 July 2024**. A faulty content update to its security software was pushed to all customers at once, crashing about **8.5 million Windows machines** worldwide (Microsoft's estimate), grounding flights and disrupting hospitals and banks, including in India. A staged rollout to a small percentage first would have caught it within minutes. The lesson for PMs: **every change should reach a small group before it reaches everyone**, including "just a config update".

The one-line rule for releases

Separate *shipping code* from *turning features on*. Put every meaningful change behind a flag, roll it out gradually while watching a small set of health metrics, and know in advance which metric would make you switch it off.

In the interview

System-design questions for PMs are not coding tests. Interviewers want to see that you can reason about **scale, failure and trade-offs** and translate them into product decisions. Strong candidates:

- **Ask about scale and shape of demand first.** "How many users, how spiky? IPL final or a normal Tuesday?" Scale changes the answer.
- **Separate the must-be-correct from the can-be-stale.** "Seat inventory needs strong consistency; the seat map can be cached for a few seconds and re-checked on booking."
- **Name the failure modes and the degraded experience.** "If the recommendation service is down, we show trending items, not an error."
- **Tie every technical choice to a user or business outcome.** "Moving images to a CDN should cut p95 page load in Tier-2 cities and reduce our bandwidth bill."

Typical prompts: "How would you prepare IRCTC for Tatkal?" (pre-scale, virtual waiting room with a queue and honest wait times, rate limits and identity checks against bots, cache static data, strong consistency on seat allocation, graceful degradation of non-booking features); "Our app went down during the sale, what do you do?" (incident process: mitigate first, communicate, then root cause and postmortem; measure revenue impact); "Should we move to microservices?" (depends on team size and scaling pain; the cost is operational complexity).

Check yourself

1. Rescale these to human time and explain why caches matter: a memory read, an SSD read, a Mumbai-US round trip.
2. Why can the average latency look fine while users complain? Which number would you watch instead?
3. Give one feature that belongs in a SQL database and one that suits NoSQL, and say why.
4. What is replication lag, and how might a user notice it?
5. Explain a hot shard using a phone directory.
6. Why did Hotstar pre-warm servers instead of relying on auto-scaling?
7. Place these on the consistency-availability scale: wallet balance, "people also bought", seat count on a flight, typing indicator in chat.
8. Define SLI, SLO and SLA for an OTP-login service, and say what an error budget lets the team do.
9. What does a feature flag let you do that an app-store release cannot? What went wrong in the CrowdStrike incident?
10. (From the internet chapter) A dashboard shows a spike in 503 errors during a sale. Is this a client-side or server-side problem, and which building block would you check first?

TECHNOLOGY • SYSTEM DESIGN IN PRACTICE

Design it like a PM: three worked walkthroughs

A repeatable method for "design X" questions, applied to a UPI payment, a food order with live tracking, and WhatsApp-style messaging

Knowing the building blocks is half the job. The other half is putting them together under pressure: in a design review where the engineering lead asks what matters most, or in an interview that begins "Walk me through how UPI works" or "How would you design live order tracking?" This chapter gives you one method and uses it three times.

A method for any "design X" question

A PM-level system design answer is not a list of technologies. It is a chain of reasoning that starts with the user and ends with trade-offs and metrics.

Eight steps for a PM-level system design answer

Spend most of your time on steps 1–3 and 6–8; that is where PM judgement shows

- 1 **Clarify the user and the goal**
Who is using it, for what, in what context? One primary flow first.
Watch out: Interviewers reward narrowing the scope out loud.
- 2 **List functional requirements**
What must it do? Pay, track, send, notify. Mark must-haves versus later.
- 3 **Set non-functional requirements**
Scale, spikes, latency, consistency, availability, security, cost, regulation.
Watch out: This is where the design is decided.
- 4 **Estimate on the back of an envelope**
 $\text{Users} \times \text{actions} \div \text{seconds} = \text{requests per second}$; $\text{peak} = \text{average} \times \text{a spike factor}$.
Watch out: Round numbers; state assumptions.
- 5 **Draw the main boxes**
Clients, gateway, core services, data stores, queues, third parties.
Watch out: Five to eight boxes, not thirty.
- 6 **Walk the critical flow**
Follow one request end to end with a sequence diagram.
Watch out: Say what the user sees at each step.
- 7 **Break it on purpose**
What if a service is slow, a message is lost, the network drops, traffic triples?
Watch out: Name the degraded experience.
- 8 **Trade-offs, metrics, questions**
Which trade-offs you chose and why; how you would measure success and health.
Watch out: End with what you'd ask the engineers.

The one-line rule

Start with the user, decide what must be correct and what can be approximate, draw only the boxes you need, and spend your remaining time on what happens when things go wrong.

WALKTHROUGH 1 · PAYMENTS

How a UPI payment works, and what can go wrong

Scanning a QR code at a kirana store, traced through five systems in a few seconds

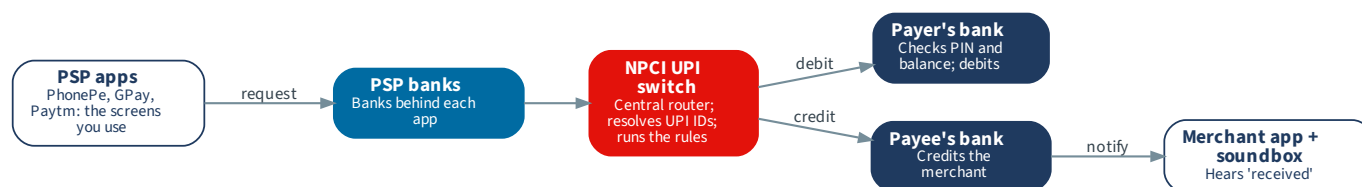
You scan the QR code at a kirana store in Indore, type ₹240, enter your PIN, and within a couple of seconds you see a green tick while the shop's soundbox announces "₹240 received". India now does this at a staggering scale: **24.51 billion UPI transactions worth ₹29.82 lakh crore in August 2026** alone (NPCI data via Business Standard, Sep 2026). That averages about **9,150 transactions every second**, around the clock, with festival and month-start peaks well above that.

Requirements. Functionally: pay to a UPI ID, phone number or QR; authenticate with a PIN; show a clear final status; reverse failed payments; notify both sides. Non-functionally: **never debit twice and never lose money** (strong consistency), a few seconds end to end, available 24×7, interoperable across hundreds of banks and dozens of apps, and secure enough that a stolen phone alone is not enough to pay.

The design idea. UPI is a **hub-and-spoke** network. Apps like PhonePe, Google Pay or Paytm are **PSP apps** (Payment Service Providers) that sit on top of partner banks. None of them hold your money. At the centre sits **NPCI's UPI switch**, which routes every message between the payer's bank and the payee's bank. This central switch is what makes any app work with any bank.

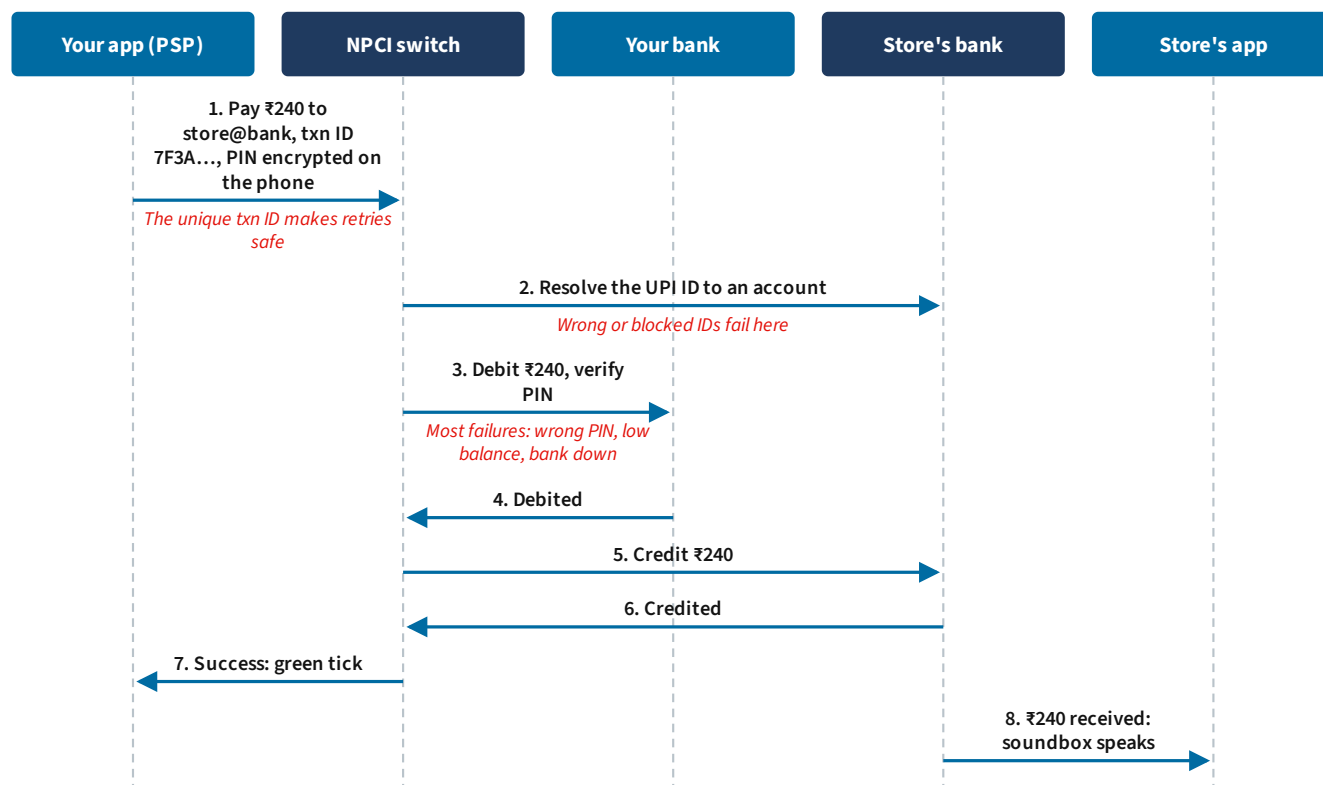
UPI is a hub-and-spoke network

Apps never hold the money; banks do. NPCI routes the messages between them



One UPI payment, step by step

A scan-and-pay at a store; the whole exchange normally completes in seconds



Source: Simplified from NPCI's published UPI flow

Notice where the PIN goes. It is encrypted on your phone and checked by **your bank**, not by the app or by NPCI. That is why no UPI app can ever "see" your PIN, and why a customer-care caller asking for it is always a fraudster.

Breaking it on purpose

Payments are hardest not when everything works, but in the grey zone where something *might* have happened.

- **Timeout after the debit.** Your bank debited you, but the reply got lost on the way back. The app does not know the outcome, so it shows "Pending" rather than guessing. It then asks NPCI using the **Check Transaction Status API**. If the credit to the store failed, the debit is **reversed automatically**. RBI's turnaround-time rules require failed UPI debits to be reversed by the next day (T+1), with compensation of ₹100 per day of delay beyond that (RBI circular, September 2019).
- **Retry without double-charging.** Users tap Pay twice; networks resend messages. Every payment carries a **unique transaction ID**, and every system treats a repeated ID as the same payment. This is **idempotency** in action.
- **Retry storms.** In the UPI outages of March–April 2025, one problem was that banks could query transaction status without limit; when one hop slowed down, banks bombarded the system with status checks, making the slowdown worse. NPCI's April 2025 circular capped these calls: the first check only after 90 seconds (later 45–60 seconds), and a maximum of three checks in the first two hours (Business Standard, April 2025). It also cut API response times from June 2025: the pay request limit fell from 30 to 15 seconds and status checks from 30 to 10 seconds (Business Today, June 2025).
- **A bank is down.** Some banks' core systems slow down at month-start and during festivals. UPI Lite, an on-device balance for small payments without a PIN, was designed partly to take small, frequent payments off banks' core systems.
- **Fraud.** Collect-request scams ("enter your PIN to receive money") exploit the UX, not the technology. Warnings on collect requests, limits for new devices and cooling periods for first-time large transfers are product defences.

UPI's key design choices and what each one buys

Central switch (NPCI)

Buys interoperability: any app, any bank. Costs: a system-wide single point that must be engineered for extreme reliability.

Strong consistency

Buys trust: money is never created or lost. Costs: some transactions sit in 'pending' until confirmed.

Unique transaction IDs

Buys safe retries: duplicate messages never double-debit. Costs: every system must store and check IDs.

PIN checked by the bank

Buys security: apps never see credentials. Costs: bank systems are in the critical path for every payment.

Strict timeouts and retry caps

Buys stability under stress. Costs: some payments fail fast instead of waiting.

Status-first UX

Buys calm users: 'Pending, don't pay again' prevents double payment. Costs: design and support effort.

Questions a PM should ask the engineers

What share of our failures are bank-side versus our side, by bank? What does the user see while status is unknown, and for how long? How do we stop users paying twice when a payment is pending? Are our retries within NPCI's limits? What is our p95 time to a final status? Which support scripts change if timeouts change?

WALKTHROUGH 2 · LOGISTICS

A food-delivery order with live tracking

From "Place order" to the rider at your gate, and why the moving dot is harder than it looks

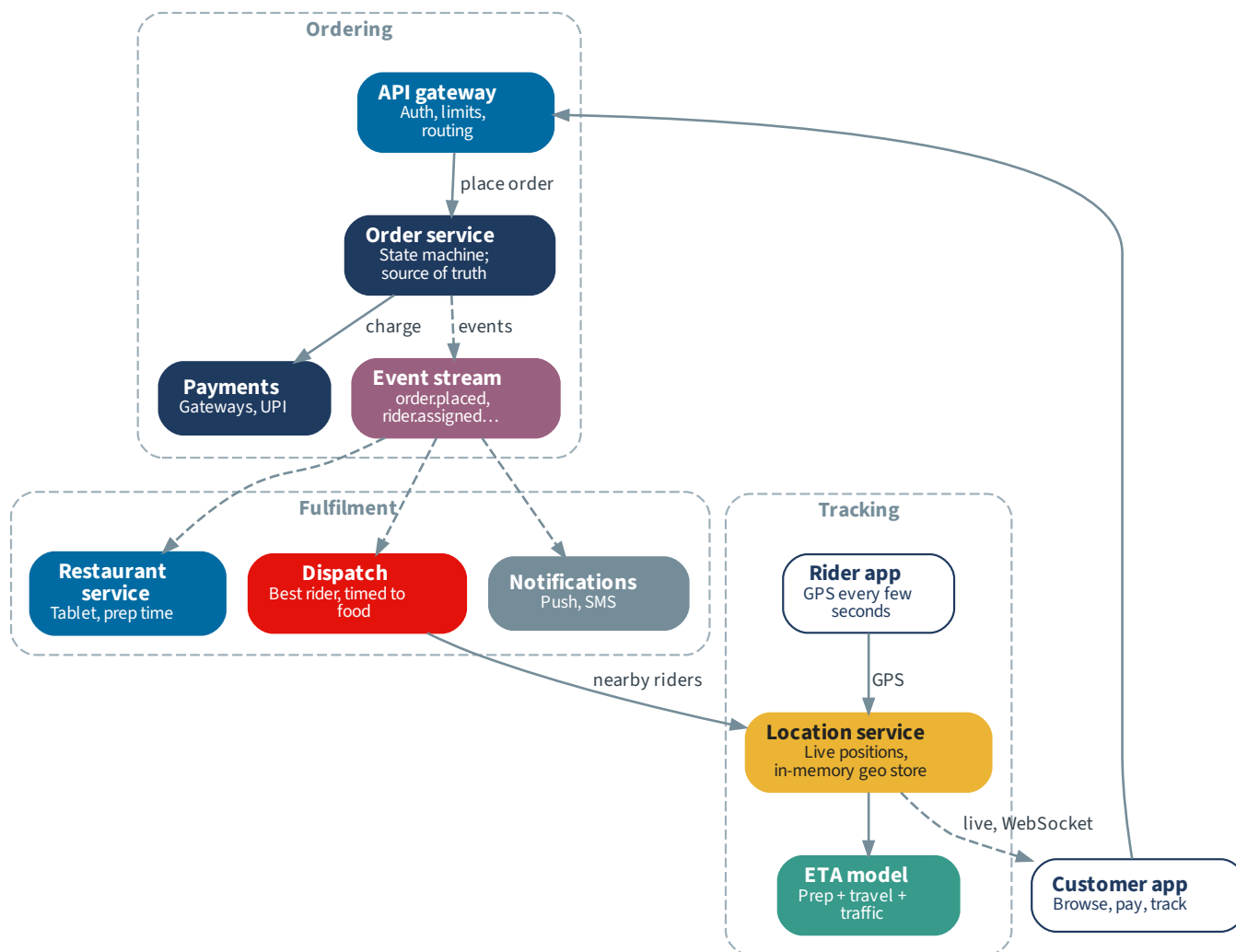
It is 8:30 pm in Hyderabad. You order from a restaurant 4 km away. Over the next 35 minutes the app tells you the restaurant accepted, the food is being prepared, a rider named Suresh is assigned, he has picked up the order, and a little bike icon moves along the roads towards you with an ETA that updates. At the same moment tens of thousands of other orders in the city are doing the same thing.

Requirements. Functionally: browse and order; pay; send the order to the restaurant; assign a rider at the right moment; show live status, rider location and ETA; allow chat or call with masked numbers; handle cancellations and refunds. Non-functionally: dinner-time peaks several times the afternoon load; order state and payment must be **exactly right**; rider location can be **a few seconds stale**; ETA should be accurate and honest; the rider app must not drain battery or data.

Back of the envelope (illustrative). Suppose 2 lakh riders are active at the national dinner peak and each phone sends its GPS position every 4 seconds. That is $2,00,000 \div 4 = 50,000$ **location updates per second**, just from riders, before any customer opens the tracking screen. This one number explains why location is handled by a separate, specialised service and not by the orders database.

The main services behind a food order

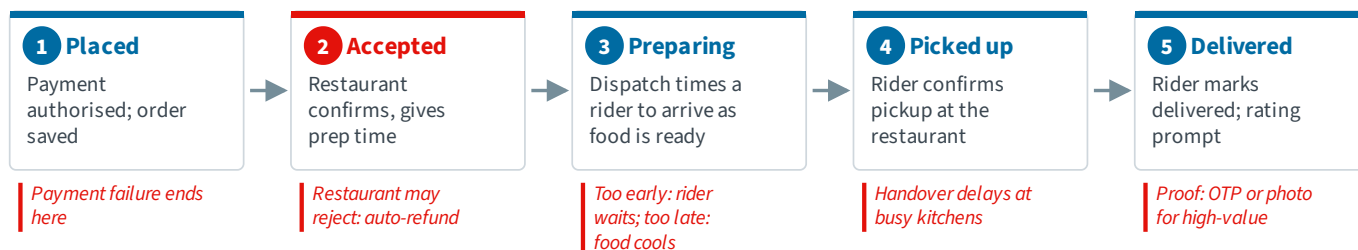
Solid arrows: requests; dashed: events and live streams



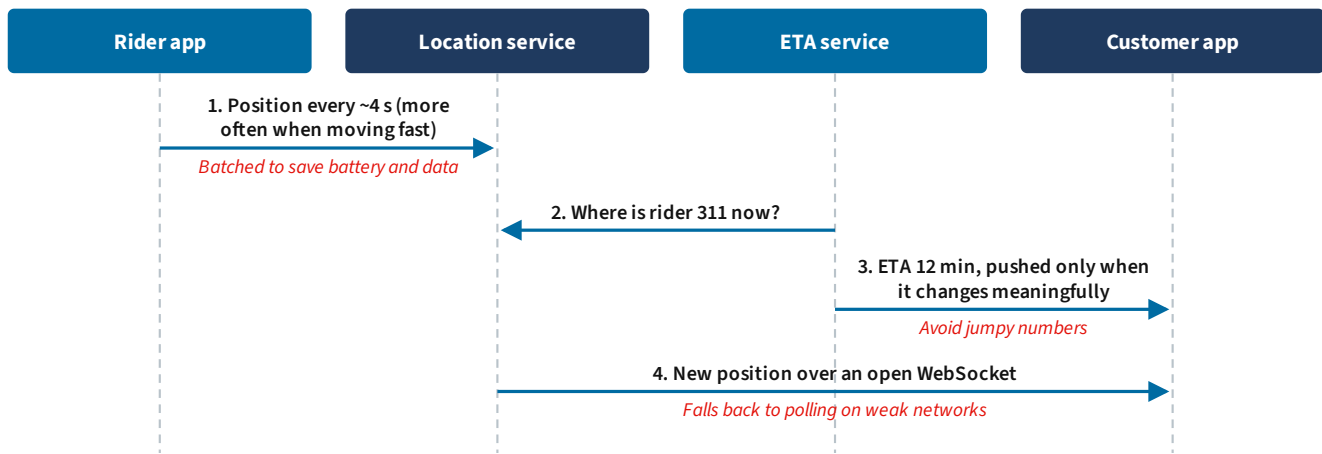
The order is a state machine. The order service is the single source of truth for where each order is, and it only allows legal moves: an order cannot be "delivered" before it is "picked up". Every change is published as an event so other services react without being called directly.

The order state machine

Each arrow is an event other services listen to; the red step is where most cancellations happen



Live tracking, step by step



The trade-offs a PM owns here

- **Freshness versus battery and cost.** GPS every second looks smooth but drains the rider's phone and multiplies server load. Every 4–5 seconds, with the app **animating** the icon smoothly between points and **snapping** it to the road map, looks just as good.
- **ETA accuracy versus honesty.** A precise ETA that keeps jumping (25, then 40, then 32 minutes) erodes trust more than a slightly padded, stable one. Good ETA models combine restaurant prep time, rider travel time, traffic and the "last 100 metres" (finding the gate, climbing stairs). Google Maps showed how much better models help: DeepMind's graph neural networks improved Maps' ETA accuracy by up to 50% in cities such as Sydney, Tokyo and Jakarta (DeepMind, September 2020).
- **Batching versus speed.** Giving one rider two nearby orders cuts cost per delivery but adds minutes to one of them. The dispatch algorithm encodes a business choice.
- **Consistency where it matters.** Order and payment states are strongly consistent; the dot on the map is eventually consistent. Never show "Delivered" based on a location guess.
- **Graceful degradation.** If the location service is overloaded, show the last known status with a timestamp ("Rider picked up your order, 8:52 pm") rather than a frozen map.

Questions a PM should ask the engineers

What is our p95 gap between promised and actual delivery time, by city and restaurant type? How often do riders wait more than five minutes at the restaurant? What does the customer see when location updates stop? How much battery does the rider app use per hour? What happens to in-flight orders if the dispatch service restarts?

WALKTHROUGH 3 · MESSAGING

How a WhatsApp message gets delivered

One grey tick, two grey ticks, two blue ticks: a lesson in store-and-forward, push, and end-to-end encryption

In October 2020, Mark Zuckerberg said WhatsApp was delivering roughly **100 billion messages a day** (TechCrunch, Oct 2020). That averages over a million messages every second. Each one must arrive once, in order, even if the recipient's phone is switched off in a train tunnel, and WhatsApp's servers must not be able to read any of them.

The ticks you see are the system's state, shown honestly to the user:

What the ticks actually mean

One grey tick

The message reached WhatsApp's server. Your phone's job is done; the server now holds it.

Two grey ticks

The message was delivered to the recipient's phone (every device, in a group: every member).

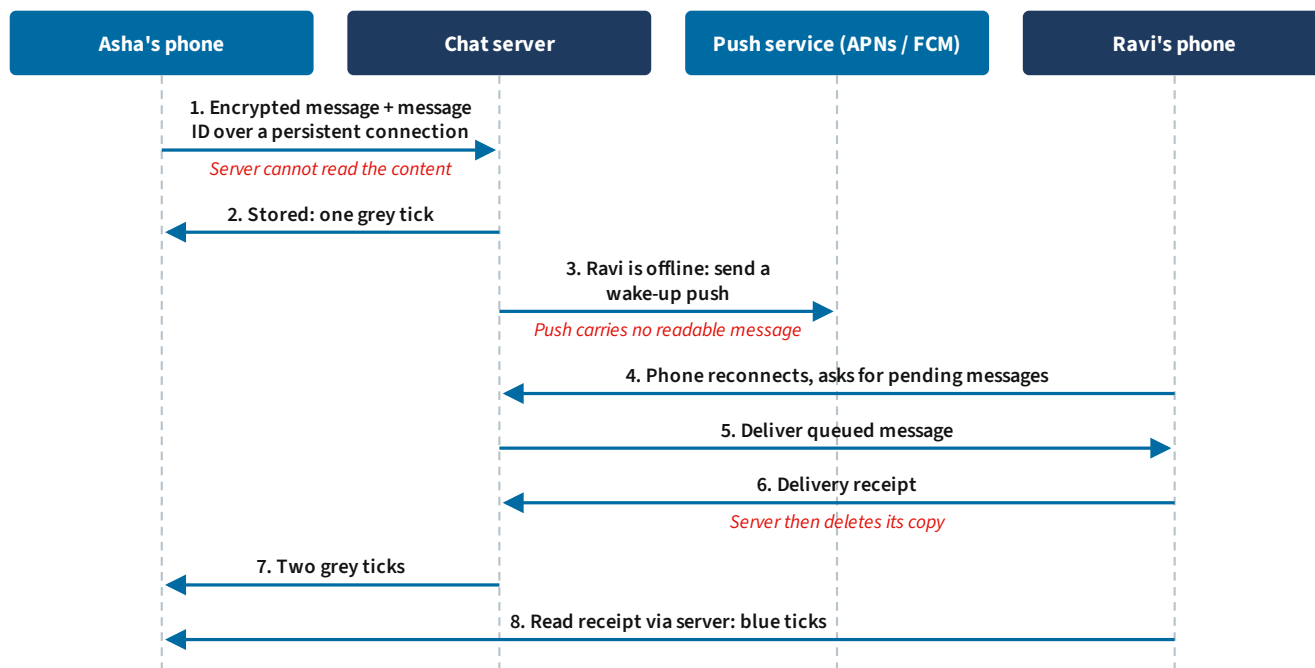
Two blue ticks

The recipient opened the chat. Read receipts can be turned off, in which case ticks stay grey.

Requirements. Deliver every message exactly once and in order; work when the recipient is offline; feel instant when both are online; support groups, media and multiple devices; **end-to-end encryption** so only sender and recipient can read content; work on

cheap phones and weak networks.

Sending a message to a friend who is offline, then online



The key ideas.

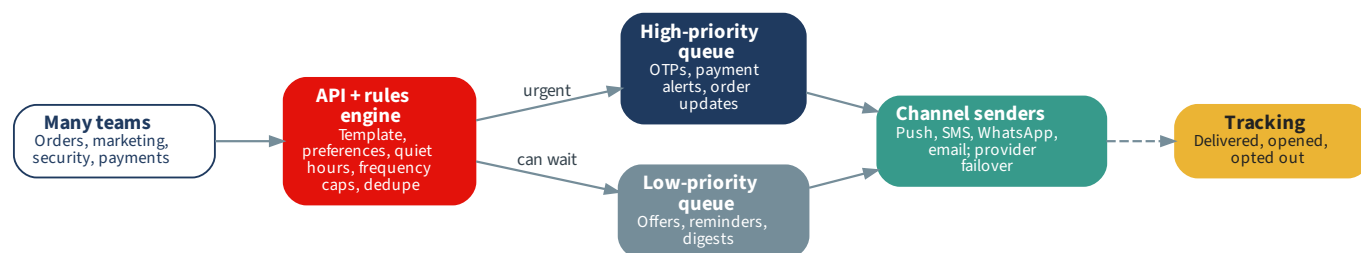
- **Persistent connections.** While the app is open, the phone keeps one long-lived connection to a chat server (like a phone line left open), so messages arrive instantly in both directions without polling.
- **Store and forward.** If the recipient is offline, the server holds the encrypted message in a queue and delivers it when they reconnect, then deletes it. WhatsApp says undelivered messages are deleted from its servers after 30 days.
- **Push to wake up.** When the app is in the background, the server asks Apple's or Google's push service to wake it; the app then fetches the message itself.
- **Message IDs and acknowledgements.** Each message has an ID; each hop acknowledges it. Resends with the same ID are ignored, so network retries never create duplicates.
- **End-to-end encryption.** Since 2016, WhatsApp uses the Signal protocol: messages are locked on the sender's phone with keys that only the recipients' devices hold. The server moves sealed envelopes it cannot open. This is a product promise with consequences: the company cannot read chats to moderate them or restore them without the user's own backup.
- **Groups fan out.** A message to a 200-member group is sent once to the server and then delivered to 200 devices. Group size limits exist partly to keep this fan-out, and misinformation spread, under control.
- **Media travels separately.** A photo is encrypted and uploaded once to storage; the message carries only a link and the key to open it.

A related design: the notification system

Almost every company eventually builds a central **notification service**, because dozens of teams want to message users and nobody wants each team spamming separately.

A central notification service

One front door for every team's messages, with rules that protect the user



The design choices are mostly product choices: **separate queues** so a marketing blast of 5 crore messages never delays an OTP; **frequency caps** ("no more than three promotional pushes a day"); **quiet hours**; **deduplication** so users do not get the same alert on three channels; **channel fallback** (push first, SMS if the push is not delivered within a minute for critical messages); and **provider failover** when an SMS vendor is down. The metric that matters is not "messages sent" but **useful messages opened** and **opt-out rate**.

Questions a PM should ask the engineers

What is our delivery latency for OTPs at p95, and does it change during a marketing campaign? What happens to messages for users who are offline for a week? How do we prevent duplicate notifications across channels? What is the opt-out rate by notification type? If we wanted end-to-end encryption, which features (search, moderation, backup) would change?

In the interview

Interviewers use these three systems because every Indian candidate uses them daily. A strong answer to "How does UPI work?" covers the actors (PSP app, PSP bank, NPCI, issuing and beneficiary banks), the flow, where the PIN is checked, what "pending" means, and one failure mode with its fix. A strong answer to "Design live tracking" starts with the user's anxiety ("where is my food and when will it come?"), estimates location traffic, separates the exact order state from the approximate location, and ends with ETA honesty. A strong answer to "Why does WhatsApp show two ticks?" explains store-and-forward and receipts, then adds a product insight: *"The ticks are a trust feature. They turn an invisible system state into something the user can see, which cuts 'did you get my message?' anxiety."*

Traps: listing technologies without explaining why; ignoring failure; promising strong consistency and instant speed everywhere; forgetting the other side of the marketplace (restaurant, rider, merchant).

Check yourself

1. Name the eight steps of the PM system-design method. Which steps show the most PM judgement?
2. Draw the UPI flow from memory with five actors. Where is the PIN verified, and why does that matter?
3. A payment is stuck at "Pending". What is happening in the system, and what should the app tell the user?
4. How did unlimited status checks make the 2025 UPI outages worse, and what did NPCI change?
5. Estimate location updates per second for 1 lakh active riders sending GPS every 5 seconds.
6. Which data in a food order must be strongly consistent, and which can be eventually consistent?
7. Explain the three WhatsApp tick states and the store-and-forward idea.
8. (From the system-design chapter) Why must an OTP never wait behind a marketing message, and which building block solves this?

TECHNOLOGY • CLOUD

Renting the computer instead of buying it

What "the cloud" really is, the IaaS–PaaS–SaaS ladder, how cloud is bought and sold, and the decisions it pushes onto a PM's desk

In 2005, a Bengaluru startup that wanted to launch a website had to buy servers, rent space in a data centre, sign a bandwidth contract and hire someone to replace failed disks at 2 am. Capacity took weeks to add, so you bought for your biggest imagined day and paid for idle machines the rest of the year. Today the same team opens an account with AWS, Microsoft Azure or Google Cloud, picks a data centre in Mumbai or Hyderabad, and is serving users within the hour. When the IPL final brings ten times the traffic, they rent ten times the machines for the evening and hand them back at midnight.

That shift is **cloud computing**: the on-demand delivery of computing resources (servers, storage, databases, networking, AI models) over the internet, paid for as you use them, instead of owned.

The electricity analogy

A century ago, large factories ran their own power plants. Then the grid arrived, and a factory simply plugged in and paid per unit. Nobody today builds a power plant to run an office. Cloud does the same for computing.

Cloud is computing as a utility

Everyday world	In technology
Owning a generator for your house	On-premises servers You buy the capacity up front (capital expense), size it for the peak, and maintain it yourself.
Paying the electricity bill by the unit	Pay-as-you-go cloud No upfront cost; pay for what you use (operating expense); capacity is someone else's problem.
The grid never 'runs out' at your house	Elasticity Scale up in minutes for a spike and back down afterwards, without buying anything.
Leaving the AC on in an empty room	Idle cloud resources The meter keeps running. Forgotten servers are the number-one cause of surprise cloud bills.

The ladder of service models: IaaS, PaaS, SaaS

Cloud comes in layers, depending on how much the provider manages for you. The classic analogy is getting pizza.

Pizza as a service

The further right, the more is done for you, and the less you control

On-premises: make it at home You buy the oven, flour, toppings; you cook and clean. You manage everything: hardware to app.	IaaS: take-and-bake You get the kitchen and oven (virtual machines, storage, network); you install and run everything on top. AWS EC2, Azure VMs.	PaaS: pizza delivered The platform runs the servers, operating system and scaling; you just bring your code and data. Google App Engine, Heroku, managed databases.	SaaS: eat at the restaurant Ready-to-use software through a browser; you just use it. Gmail, Slack, Salesforce, Zoho, Freshdesk.
---	---	---	--

	IaaS	PaaS	SaaS
You get	Raw infrastructure: virtual machines, storage, networks	A platform to build, deploy and run apps	Finished software
Provider manages	Hardware, data centre	Hardware + operating system + runtime + scaling	Everything
You manage	OS, runtime, app, data, security settings	Your app and data	Your users, settings and data
Best for	Custom infrastructure, special needs, large scale	Shipping fast with a small team	Standard business needs (email, CRM, HR, support)
PM focus	Flexibility, cost control	Speed of development	Adoption, integrations, vendor fit
Example scenario	Enterprise building its own platform	Startup MVP	Internal collaboration tool

The rule of thumb: **moving down the ladder toward IaaS gives you more control and more engineering responsibility; moving up toward SaaS gives you speed and less control.**

Two newer rungs matter too. **Containers** (Docker) package an app with everything it needs so it runs the same everywhere, and **Kubernetes** runs thousands of containers and restarts or adds them automatically. **Serverless** or **functions-as-a-service** (AWS Lambda) goes further: you upload a small piece of code that runs only when triggered, and you pay per execution, down to the

millisecond. It is excellent for spiky, occasional work (resize an image when it is uploaded) and can get expensive for constant heavy work.

Where the cloud lives: deployment models

Public, private and hybrid cloud

Public cloud

Shared infrastructure owned by AWS, Azure, Google, used over the internet. Scalable and cheap to start; less control; compliance needs care. Most startups.

Private cloud

Infrastructure dedicated to one organisation, on its own premises or hosted. High control and security; expensive and slower to scale. Some banks, defence, government.

Hybrid cloud

A mix: sensitive systems (core banking) on private, customer-facing apps on public, connected. Balance of control and scale; more complex to run.

Multi-cloud

Using two or more public providers, to avoid lock-in, meet local rules or use each one's best services. More resilient, much harder to manage.

How cloud is bought, and how cloud products are sold

Two different pricing questions often get mixed up. As a **buyer** of cloud, a company chooses how to pay AWS. As a **seller** of a SaaS product, it chooses how to charge its own customers.

Buying cloud capacity: three ways to pay for the same server

Discounts are the providers' published maximums; actual savings vary

On-demand

Pay by the second or hour, no commitment. Most flexible, highest unit price. Good for new or unpredictable workloads.

Reserved / committed

Commit to 1 or 3 years of usage for a large discount (up to about 72%). Good for the steady base load you are sure of.

Spot / preemptible

Spare capacity at up to about 90% off, but the provider can take it back with two minutes' notice. Good for batch jobs that can pause.

Source: AWS pricing pages (Savings Plans up to 72%, Spot up to 90% off On-Demand), 2026

A mature company mixes them: reserved capacity for the base load that runs all day, on-demand for daily peaks, spot for model training and overnight analytics.

As a **seller**, the common SaaS models are:

Model	How it works	PM view
Pay-as-you-go / usage	Charge per use: per API call, per GB, per message	Fair and scales with value; revenue fluctuates; customers fear bill shock
Subscription	Fixed monthly or annual fee	Predictable for both sides; heavy users may cost more than they pay
Tiered	Plans with rising limits and features (Free, Pro, Enterprise)	Serves segments; upgrade path is the growth engine
Freemium	Basic free forever, premium paid	Cheap acquisition; lives or dies on free-to-paid conversion
Reserved / committed	Customer commits to a volume for a discount	Locks in revenue; good for enterprise deals

Benefits and the catches

Benefit	The catch on the other side
Cost efficiency: no upfront capital; pay for use (capex → opex)	Cost overruns: idle servers, oversized machines and data-transfer (egress) charges produce bills nobody expected

Benefit	The catch on the other side
Scalability and elasticity: add capacity in minutes	Vendor lock-in: deep use of one provider's special services makes switching expensive
Speed to market: infrastructure in minutes, not months	Security is shared: the provider secures the cloud; you must secure what you put <i>in</i> it (the "shared responsibility model")
Global reach: regions close to users worldwide	Data residency: some laws require data to stay in the country
Reliability: built-in backups, multiple zones, failover	Provider outages still happen: the AWS US-East outage of October 2025 lasted over 14 hours for some services

Data residency in India is a concrete example. Since RBI's 2018 directive on storage of payment system data, payment companies must store payment data only in India. The Digital Personal Data Protection Act, 2023 and its 2025 Rules add obligations on consent and cross-border transfers of personal data. A PM launching a fintech or health product must ask early: which data can leave India, and which regions are we allowed to use?

Contrasting case. Cloud is not always cheaper. At very large, steady scale, owning hardware can cost less. 37signals, the company behind Basecamp and HEY, publicly moved off public cloud in 2023 and reported large savings, because its load was predictable and its team could run servers. Cloud wins on flexibility and speed; ownership can win on steady-state cost.

The one-line rule

Cloud turns computing into a variable cost you can dial up and down. That makes it fast and flexible, but it also means every feature has a running cost per user that someone must watch.

Why a PM cares.

- **Cloud cost is unit economics.** "Cloud cost per order" or "per active user" belongs on the same dashboard as contribution margin. A feature that stores every video forever or calls an expensive AI model on every page view has a real, recurring cost.
- **Build on the right rung.** For a new product, favour managed services (PaaS, SaaS) to ship fast, and move lower only when scale or control demands it. Do not build what you can rent unless it is your differentiator: **compete on unique value, not on parity** in infrastructure.
- **Design for change.** Loose coupling between modules keeps you able to swap a provider or service later and makes it easier to optimise the expensive part without touching the rest.
- **Plan for residency, latency and compliance from day one,** and choose your own revenue model (subscription, usage, tiered) so that revenue rises with the costs usage creates.

In the interview. "Should our startup build on AWS or our own servers?" (cloud: no capex, speed, elasticity; revisit at steady massive scale), "Explain IaaS, PaaS and SaaS with examples" (pizza analogy, then one Indian example each: a bank running VMs on Azure, a startup deploying on a managed platform, Zoho or Freshworks as SaaS), "Our cloud bill doubled; what do you do?" (break it down by service and feature; look for idle and oversized resources, egress, logging, AI calls; move steady load to committed pricing; add per-team cost dashboards and budgets). A strong line: *"I'd treat cloud cost as a product metric, not a finance surprise, and track cost per order next to conversion."*

Check yourself

1. Explain cloud computing using the electricity analogy, including what "idle resources" means.
2. Map on-premises, IaaS, PaaS and SaaS to the pizza analogy, and give one real example of each.
3. When would you choose reserved, on-demand and spot capacity for the same product?
4. What is the shared responsibility model? Give one thing the provider secures and one the customer must secure.
5. Why might an Indian payments company be unable to use a Singapore data centre for its payment data?
6. Give one situation in which owning servers beats public cloud.
7. (From the system-design chapter) How does cloud elasticity relate to horizontal scaling, and why did Hotstar still pre-warm capacity?

TECHNOLOGY · DATA AND MACHINE LEARNING

Teaching computers by example

What AI, machine learning and deep learning actually mean, how a model learns, how recommendations work, and how a PM decides whether a problem needs ML at all

Suppose you are the PM for fraud at a payments app and you try to catch fraud with rules. "Block any payment above ₹50,000 from a new device." "Flag five payments to different people within a minute." You write fifty rules. Fraudsters notice and switch to ₹49,000 payments spread over six minutes. You write fifty more. Meanwhile genuine users who just bought a new phone are getting blocked.

Now try something different. Take ten million past transactions, each labelled "fraud" or "genuine" after investigation, and let a program **find the patterns itself**: the combinations of amount, time, device age, location, payee history and typing speed that separate the two. When fraudsters change tactics, retrain on the new examples. That is **machine learning**.

Traditional programming versus machine learning

Traditional programming	Machine learning
Input Rules written by people + data	Input Data + the right answers (labels)
Output Answers	Output Rules, learned: a model
Example IF amount > ₹50,000 AND device age < 1 day THEN block	Example Model scores each payment 0–1 for fraud risk from hundreds of signals
Good when The logic is clear, stable and must be explainable, like GST calculation or a KYC checklist.	Good when The patterns are too many, too subtle or change too fast to write by hand.

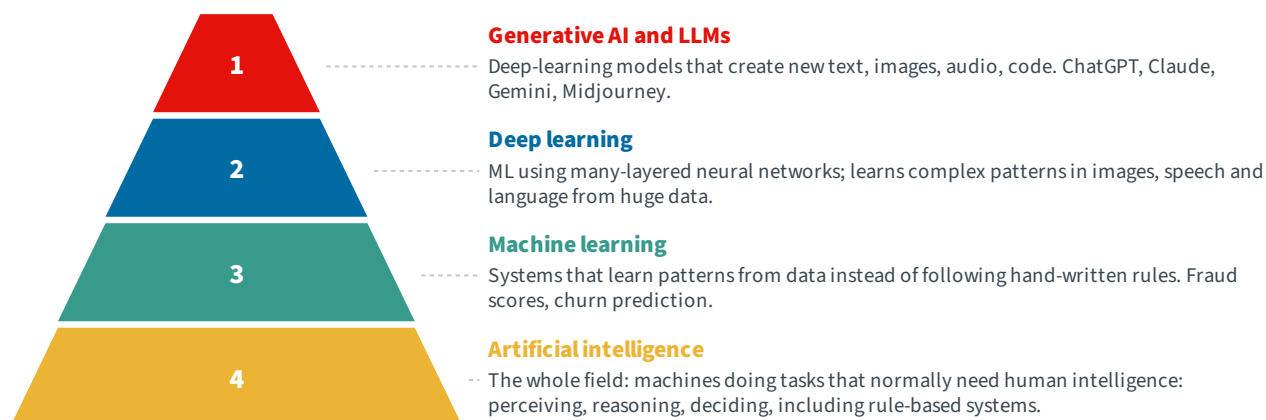
ML flips programming around: instead of writing the rules, you provide examples and the computer infers the rules.

AI, ML, deep learning, generative AI: the nesting dolls

These terms are used loosely, but they fit inside each other.

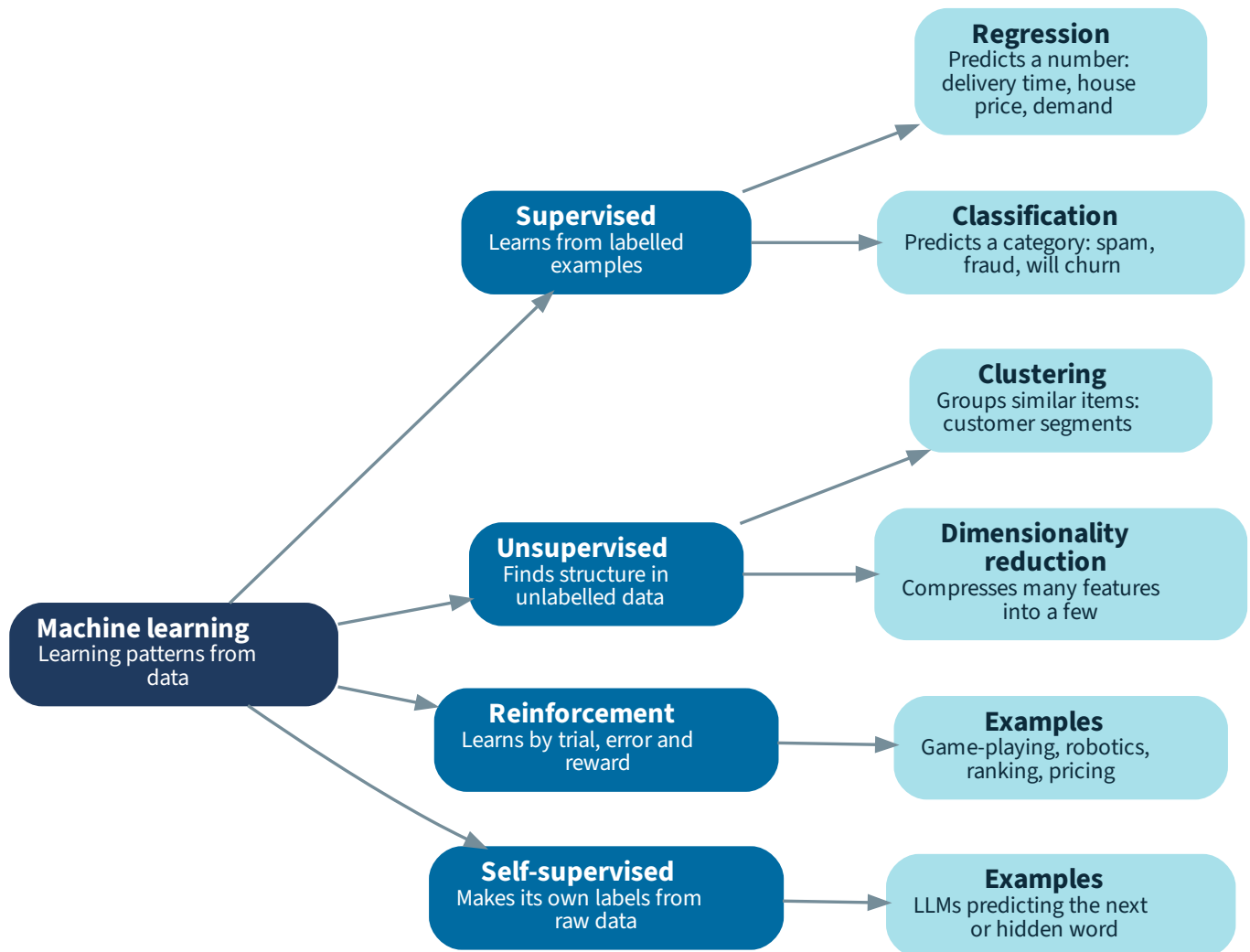
The nesting dolls of AI

Each layer is a subset of the one below it



The main ways machines learn

Types of machine learning



- **Supervised learning** is learning with an answer key. Show the model thousands of past orders with their actual delivery times and it learns to predict delivery time for a new order (**regression**, because the answer is a number). Show it emails marked spam or not and it learns to sort new emails (**classification**, because the answer is a category).
- **Unsupervised learning** has no answer key. Give it every customer's purchase history and it finds natural groups: weekend party shoppers, monthly bulk buyers, daily milk-and-bread buyers (**clustering**). Marketing then names and targets those segments.
- **Reinforcement learning** learns by acting and receiving rewards, like training a dog with treats. A game-playing program tries moves, wins or loses, and gradually learns strategy. It is also used to tune recommendations and prices over time, and, as a later chapter explains, to shape how chatbots respond.
- **Self-supervised learning** creates its own labels from raw data: hide a word in a sentence and ask the model to guess it. This is how today's large language models are pre-trained on vast amounts of text without anyone labelling it.

How a model is trained

Think of a student preparing for the CAT. She practises on thousands of questions (**training**), checks herself on mock tests she has not seen and adjusts her strategy (**validation**), and finally sits the real exam once (**testing**). If she simply memorised the answers to the practice papers she would ace them and fail the real exam. In ML that failure has a name: **overfitting**.

The life of an ML model

Most of the work, and most of the failures, are in data and monitoring, not in the algorithm

1 Define the prediction

Exactly what are we predicting, for whom, and what decision will it drive? 'Probability this order is late by 10+ minutes.'

Watch out: A vague target produces a useless model.

2 Collect and label data

Past orders with features (distance, restaurant, weather, time) and the true outcome (late or not).

Watch out: Garbage in, garbage out; biased data gives biased models.

3 Split the data

Training set (~70%) to learn; validation set (~15%) to tune; test set (~15%) touched once, at the end.

Watch out: Test data must stay unseen or scores are inflated.

4 Train and tune

The algorithm adjusts itself to fit the training data; engineers adjust settings (hyperparameters) using the validation set.

Watch out: Watch for overfitting: great on training, poor on validation.

5 Evaluate

Score on the test set with metrics that match the business cost of errors (see precision and recall later).

Watch out: Accuracy alone is often misleading.

6 Deploy

Serve predictions inside the product, often behind a feature flag and an A/B test.

Watch out: Decide the fallback if the model is down.

7 Monitor and retrain

Track live performance; retrain when the world changes (data drift).

Watch out: Covid changed shopping overnight; old models broke.

Words to know

Feature: an input signal the model uses (order distance, hour, rain). **Label:** the right answer for a training example (late: yes).

Model: the learned function that turns features into a prediction. **Training / inference:** learning from data / using the model to predict. **Hyperparameter:** a setting chosen by engineers, not learned (how deep a tree may grow). **Overfitting:**

memorising the training data, failing on new data. **Underfitting:** too simple to capture the pattern. **Data drift:** real-world data changes so the model's past lessons stop applying. **Ground truth:** the verified correct answer, often slow or costly to get (a fraud investigation can take weeks).

Three families of algorithms, and what to ask about each

You do not need to know how the algorithms work inside. You do need to know their personalities.

Family	Great for	Ask your team	Watch out for
Linear models (linear and logistic regression)	A simple, fast baseline; predictable, explainable outcomes	"Is the relationship roughly linear? How much better is anything fancier than this baseline?"	Oversimplifying; misses interactions ("late night <i>and</i> new device")
Tree-based models (decision trees, random forests, gradient-boosted trees such as XGBoost)	Tabular business data: credit scoring, fraud, churn. Often the strongest choice for spreadsheet-like data	"Can we show the business the main decision drivers?"	Overfitting if trees grow too deep
Neural networks (deep learning)	Images, speech, language, very large complex datasets	"Do we have enough data and compute? How will we explain a decision?"	The black-box problem: hard to explain a single prediction, which matters for trust, debugging and regulation

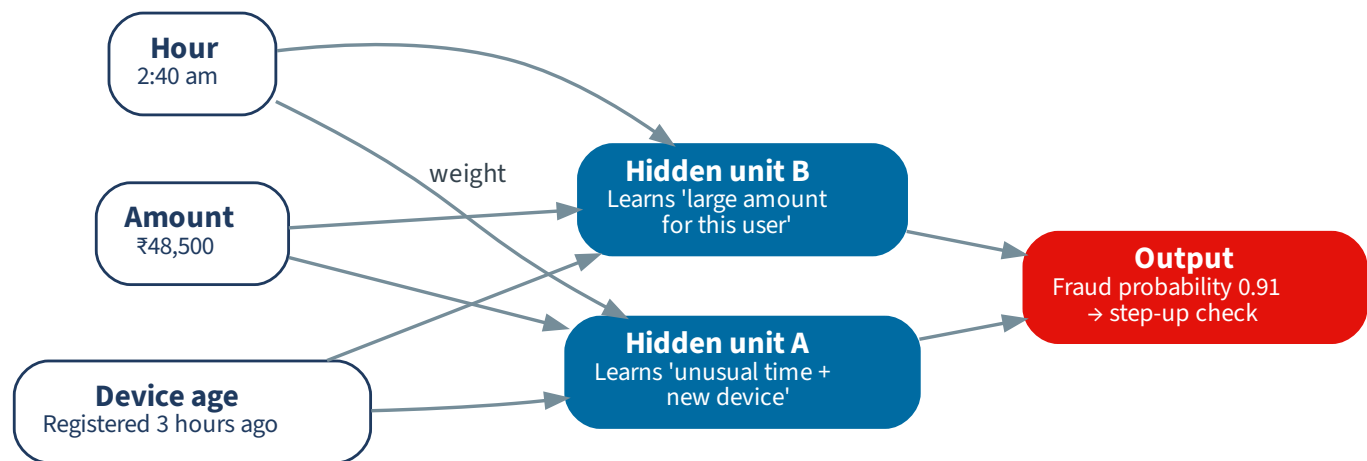
A PM who asks "what is the simple baseline, and how much does the complex model beat it by?" earns instant respect. Many teams ship a neural network where a logistic regression would have done 95% as well at a tenth of the cost.

Neural networks, without the maths

A neural network is loosely inspired by the brain: layers of simple units ("neurons") connected to each other. Picture a panel of judges in rows. The first row each look at raw inputs; each judge in the next row listens to the first row's opinions, **giving more weight to some judges than others**; the final judge announces a verdict. Training means adjusting how much each judge trusts each other judge until the verdicts match the right answers on millions of examples.

A tiny neural network scoring a payment for fraud

Real networks have millions to billions of weights; the idea is the same



The key terms: **input layer** (receives features), **hidden layers** (learn intermediate patterns nobody programmed, such as "late-night + new device"), **output layer** (the prediction), **weights** (how much each connection matters), **activation function** (decides how strongly a unit passes its signal on), and **training** (nudging all the weights, a little at a time, to reduce errors; the method is called backpropagation). Deeper networks learn more complex patterns but need more data and more computing power. Neural networks power image recognition, speech-to-text, recommendations, fraud detection and every large language model.

Recommender systems: how Netflix, YouTube and Flipkart choose for you

Open YouTube and the home page is different for every person in your family. Recommenders decide much of what gets watched, bought and listened to on large platforms. Netflix has said for years that most of what members watch comes from recommendations rather than search. Two basic ideas sit underneath.

Two ways to recommend

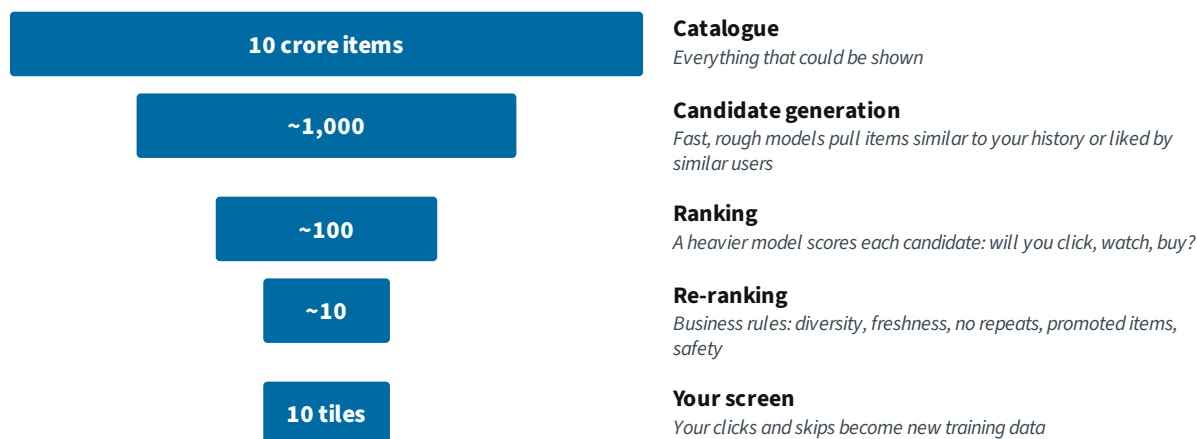
Content-based filtering	Collaborative filtering
Idea 'You liked this; here are similar items.' Uses item features: genre, cast, brand, price.	Idea 'People who behave like you also liked...' Uses only behaviour, not item features.
Example Watched three Malayalam thrillers → more Malayalam thrillers.	Example Fans of your three thrillers also loved a Korean drama → you get the drama.
Strengths Works for brand-new items with good descriptions; easy to explain.	Strengths Finds surprising, relevant items; no item metadata needed.
Weakness Stays in your lane: little discovery; needs good item data.	Weakness Cold start: new users and new items have no behaviour yet; popular items dominate.

Almost every large product uses a hybrid, plus rules for diversity, freshness and business goals.

Real systems at scale work in stages, because scoring millions of items for every user on every visit is too slow.

How a large-scale recommender works

The funnel from millions of items to the ten you see



Source: Pattern described in YouTube's published recommender paper (Covington et al., 2016); illustrative numbers

Cold start is the classic PM problem in recommenders. New users: ask for a few preferences at onboarding, use location and device signals, and show popular items. New items: use content features and give them a little exploration traffic. **Filter bubbles** are the other: optimising only for clicks shows you more of the same, which can bore users over time or amplify harmful content. That is why re-ranking adds diversity and why "what are we optimising for?" (clicks, watch time, satisfaction surveys, long-term retention) is the most important PM question about any recommender.

What ML is good at, and where it hurts

The promise and the problems of AI/ML

What it gives you	What it costs you
● Automation Handles repetitive judgement at scale: sort tickets, verify documents.	● Data quality Poor or unrepresentative data gives wrong predictions.
● Personalisation A different, better experience for each user.	● Bias and fairness Amazon dropped an experimental hiring model in 2018 after it penalised CVs mentioning 'women's' (Reuters).
● Better decisions Predictions from patterns people cannot see.	● Privacy and security Sensitive data must be protected; DPDP Act applies.
● Scale Millions of decisions a second.	● Explainability Hard to justify a loan rejection made by a black box.
● Improvement Gets better as data accumulates.	● Cost and hallucination Compute is expensive; generative models can produce confident nonsense.

Should this be an ML feature at all? A PM's checklist

Five questions before building an AI/ML feature

1. Product goal What user problem are we solving? Why does it need ML rather than a rule? How does it support the product goal?	2. Data readiness What data do we have and need? Is it accurate, representative and unbiased? Enough of it? Can we get labels?	3. MVP and integration What is the smallest AI feature that delivers value? How does it fit into the product? What happens when the model is unavailable?
4. Success metrics Model metrics (precision, recall), product metrics (conversion, CTR, retention) and the acceptable false-positive / false-negative trade-off.	5. Monitoring and trust How will users understand and trust outputs? How do we detect drift? What is the rollback or fallback plan?	

The one-line rule

Use ML when the pattern is real, the data exists, the rules would be too many or change too fast, and a wrong answer is survivable or catchable. Otherwise, write the rule.

Contrasting case. Many "AI features" are better as rules. Calculating GST, enforcing a refund policy, or routing tickets that contain an order ID needs no model. ML adds cost, unpredictability and a monitoring burden; a PM should be able to say "this does not need ML" as confidently as "this does".

TECHNOLOGY · GENERATIVE AI

From predicting to creating: generative AI

How GenAI differs from the ML above, what it can generate, and the new problems it brings

Traditional ML **predicts or classifies**: is this payment fraud, will this user churn, how long will delivery take. **Generative AI creates** new content: an email, an image, a song, a working piece of code, a video. It learns the patterns of enormous datasets well enough to produce new examples that fit them.

Traditional ML versus generative AI

Traditional ML	Generative AI
Output A prediction: a number, a category, a score.	Output New content: text, images, audio, video, code.
Data Often structured, business-specific tables.	Data Huge, mostly unstructured, multimodal datasets.
Example A fraud score of 0.91 on one payment.	Example Drafting a polite reply to an angry customer email.
Evaluation Clear right answers: was it fraud or not?	Evaluation Many acceptable answers; quality is judged against a rubric (see the chapter on evals).

GenAI is built on deep learning but changes the product problem: outputs are open-ended, so defining 'good' becomes the hard part.

Type	What it does	Popular products	PM use cases
Text	Human-like writing, summarising, answering	ChatGPT, Claude, Gemini	Support assistants, summaries, content drafts, search

Type	What it does	Popular products	PM use cases
Image	Pictures from a text prompt	Midjourney, DALL·E, Adobe Firefly, Imagen	Marketing creatives, product mock-ups, catalogue images
Code	Writes, explains and fixes code	GitHub Copilot, Cursor, Claude Code	Developer productivity, rapid prototypes
Audio	Speech and music	ElevenLabs, Suno	Voice assistants, dubbing, accessibility, vernacular content
Video	Video from text or images	Veo, Runway, Sora	Ads, product demos, training content

Common applications: conversational assistants and customer support, content creation and summarisation, enterprise knowledge assistants, coding help, marketing and design automation, and increasingly **agents** that take actions (covered two chapters from now).

The challenges are new in kind, not just in degree: **hallucinations** (fluent, confident, false statements), **bias** in generated content, **privacy and security** risks (users paste sensitive data; attackers hide instructions in documents), **high and variable compute cost** per use, and **copyright and IP** questions about training data and outputs. Each has a product answer, and the next three chapters cover them.

In the interview. Expect "What's the difference between AI, ML and deep learning?" (the nesting dolls, with one example each), "How does Netflix recommend?" (content-based versus collaborative, cold start, the candidate–ranking funnel, what it optimises), "Would you use ML for X?" (the five-question checklist, and be willing to say no), and "What could go wrong with this model?" (bias in data, drift, explainability, fallback). A strong line: *"Before choosing a model I'd ask what decision it drives and what a wrong prediction costs. For loan approvals I'd accept a slightly less accurate but explainable tree model, because we must tell rejected applicants why."*

Check yourself

1. Explain the difference between traditional programming and ML using the fraud example.
2. Place AI, ML, deep learning and generative AI in the nesting dolls, with one example each.
3. Is predicting next week's demand for onions regression, classification or clustering? What about grouping customers by shopping style?
4. What are the training, validation and test sets for? What is overfitting, in the CAT analogy?
5. When would you prefer a tree-based model over a neural network?
6. Explain the cold-start problem and two ways to reduce it for a new user.
7. Why does a large recommender use candidate generation before ranking?
8. (From the system-design chapter) A fraud model runs on every UPI payment. What should the product do if the model service times out?

TECHNOLOGY • LARGE LANGUAGE MODELS

The world's most well-read autocomplete

How LLMs turn a prompt into an answer one token at a time, what the settings do, which kind of model to pick, and how to write prompts that work

Type "Happy birth" on your phone and the keyboard suggests "day". It has seen that pattern many times. Now imagine a keyboard that has read a large share of all the public text ever written, in dozens of languages, plus code, and that looks at your entire message, not just the last word, before suggesting what comes next. Keep accepting its suggestions and it will write the whole email for you. That, at its heart, is a **large language model (LLM)**.

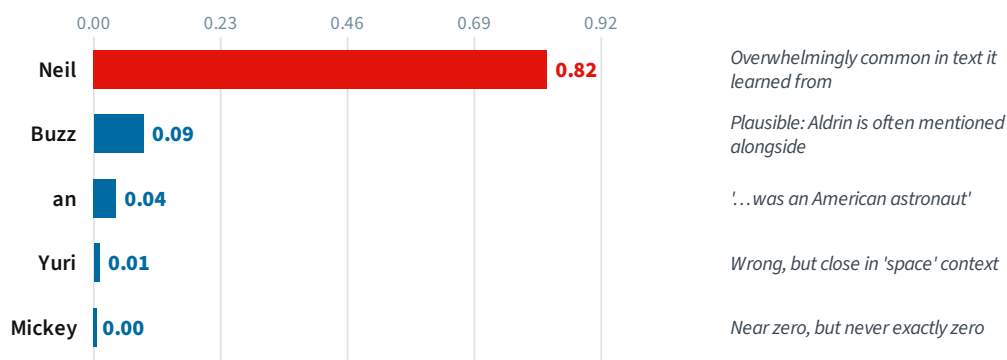
An LLM is a very large neural network trained on huge amounts of text to do one thing: **predict the next piece of text**. Everything else, answering questions, summarising contracts, writing code, translating Tamil to Hindi, emerges from doing that one thing extraordinarily well.

Prediction, one token at a time

Give a model the prompt "The first person to walk on the Moon was". It does not look up a fact. It calculates, for every possible next piece of text in its vocabulary, how likely it is to come next.

What the model 'sees' after 'The first person to walk on the Moon was'

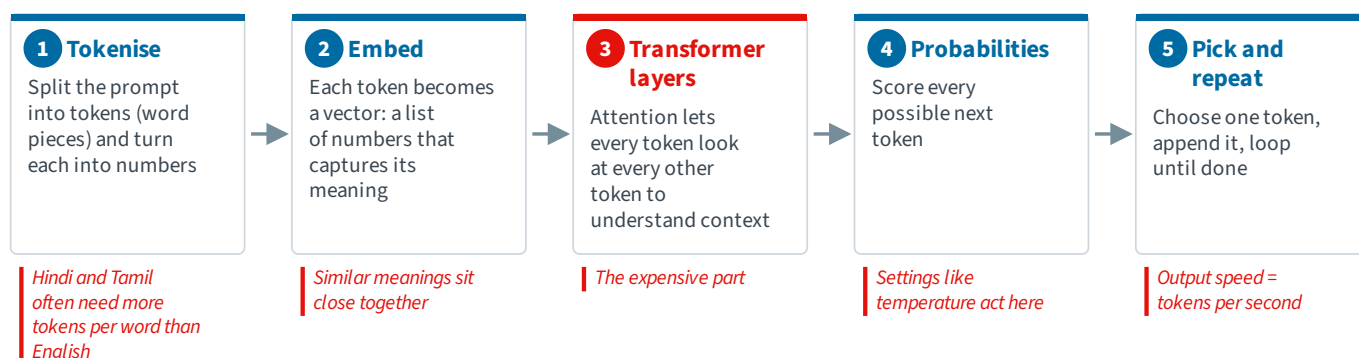
Probabilities for the next token (illustrative)



It picks a token (usually one of the likely ones), adds it to the text, and repeats the whole calculation to predict the token after that, and so on until the answer is complete.

How an LLM generates an answer

The loop runs once per token; a 300-word answer is roughly 400 loops



Tokens are the unit of everything in LLM products: pricing, speed and limits. A token is a word or a piece of a word; in English, one token is roughly three-quarters of a word on average. Many Indian-language scripts are split into more tokens per word, which quietly makes the same answer in Hindi or Telugu slower and more expensive than in English, a real product consideration for Bharat-first apps.

The **context window** is the model's working memory: the maximum number of tokens (your prompt, any documents you paste, the conversation so far, and the answer) it can consider at once. It ranges from thousands to over a million tokens depending on the model. Anything outside the window, the model simply does not see.

Embeddings deserve a moment because they return in the RAG chapter. The model turns each token, and can turn whole sentences, into a long list of numbers, like coordinates on a map of meaning. "Refund" and "money back" land close together; "refund" and "biryani" land far apart. That is how an AI system can find the right help article even when the user's words do not match the article's words.

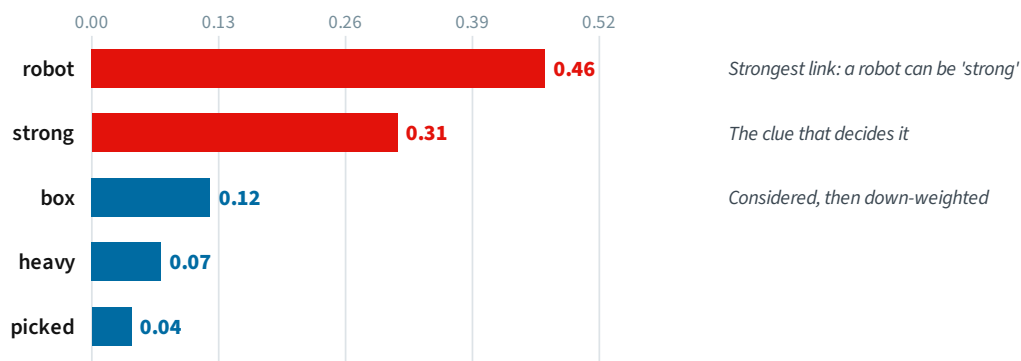
The transformer and attention

Before 2017, language models read text roughly word by word and struggled to connect words far apart. In 2017, a team at Google published *Attention Is All You Need*, introducing the **transformer**. Its key idea, **attention**, lets every word look at every other word in the passage and decide which ones matter for understanding it.

Read this: "The robot picked up the heavy box because **it** was strong." You know "it" means the robot. Change one word: "...because **it** was light." Now "it" means the box. Attention is how the model makes the same leap: when processing "it", it weighs "strong" and "robot" heavily in the first sentence and "light" and "box" in the second.

Attention, as a highlighter

Processing 'it', the model weighs other words by relevance (illustrative weights)



The "GPT" in ChatGPT stands for **Generative Pre-trained Transformer**: it generates text, it was pre-trained on a large corpus, and it is built on the transformer. Most of today's major models use this architecture.

How an LLM is made

From raw text to a helpful assistant

- 1 Pre-training**
 The model reads trillions of tokens of text and code, learning to predict the next token (self-supervised). This takes months on thousands of chips and costs very large sums.
Watch out: Result: a knowledgeable but unruly 'base model'
- 2 Instruction tuning**
 Trained on curated examples of good answers to instructions, so it follows requests instead of just continuing text.
Watch out: Result: an assistant that answers
- 3 Reinforcement learning from feedback**
 People (and increasingly automated graders) compare answers; the model is rewarded for helpful, honest, harmless ones. Reasoning models are also trained with rewards on checkable tasks like maths and code.
Watch out: Shapes tone, safety and reasoning
- 4 Deployment**
 Served through an app or an API; companies may add their own instructions, data (RAG), tools and fine-tuning.
Watch out: Knowledge frozen at a training cutoff date

Two consequences matter for products. First, the model's knowledge stops at its **training cutoff**; it does not know yesterday's news or your company's refund policy unless you give it that information in the prompt (which is what RAG does). Second, because it generates the *likeliest-sounding* continuation, it can produce fluent, confident, **false** statements: **hallucinations**. The model is optimised to be plausible, and plausible is usually but not always true.

Controlling the output: temperature, top-k and top-p

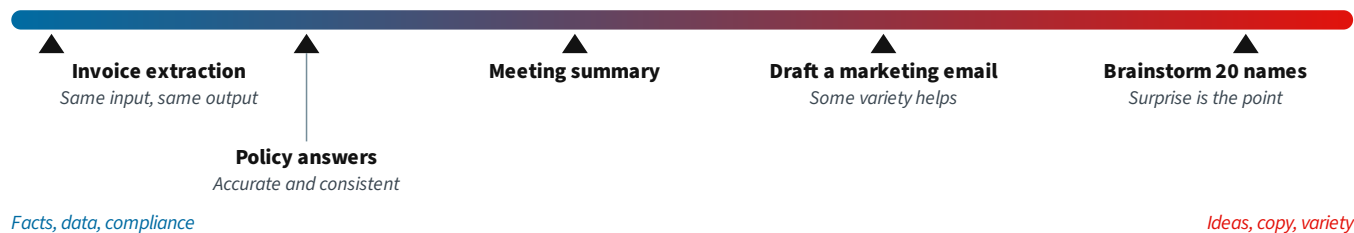
Remember that the model picks from a list of probabilities. Three settings change how it picks.

- Temperature** controls randomness. Low (for example 0–0.3) makes the model almost always pick the most likely token: consistent, predictable, a little bland. High (0.8–1.0+) flattens the probabilities so less likely tokens get picked more often: more varied and creative, and more likely to drift or make things up.
- Top-k** restricts the choice to the k most likely tokens. With $k = 1$, the model always takes the top option.
- Top-p** ("nucleus sampling") restricts the choice to the smallest set of tokens whose probabilities add up to p , say 90%. When the model is confident, that set is tiny; when it is unsure, the set is wider.

Match the temperature to the task

Low temperature: predictable

High temperature: creative



Other controls you will meet: **maximum output tokens** (caps length and cost), **stop sequences** (where to stop), a **system prompt** (standing instructions the user does not see, such as tone, rules and persona), and **structured outputs** (forcing the answer into a fixed JSON format so software can use it reliably). Some providers fix or limit sampling settings on their reasoning models; there, the prompt and the choice of model matter more than the dials.

LLM vocabulary

Token: a word or word-piece; the unit of pricing and limits. **Context window:** how many tokens the model can consider at once. **Embedding:** numbers representing meaning, used for semantic search. **Inference:** running the model to get an answer (as opposed to training it). **Latency:** often split into time-to-first-token (how soon text starts appearing) and tokens per second. **Hallucination:** a fluent but unsupported or false output. **Fine-tuning:** further training a model on your own examples to specialise it. **Open-weight model:** a model whose weights you can download and run yourself (Llama, Mistral, Qwen, Gemma, DeepSeek); **closed model:** used only through the provider's API (GPT, Claude, Gemini).

The model landscape: pick the right tool

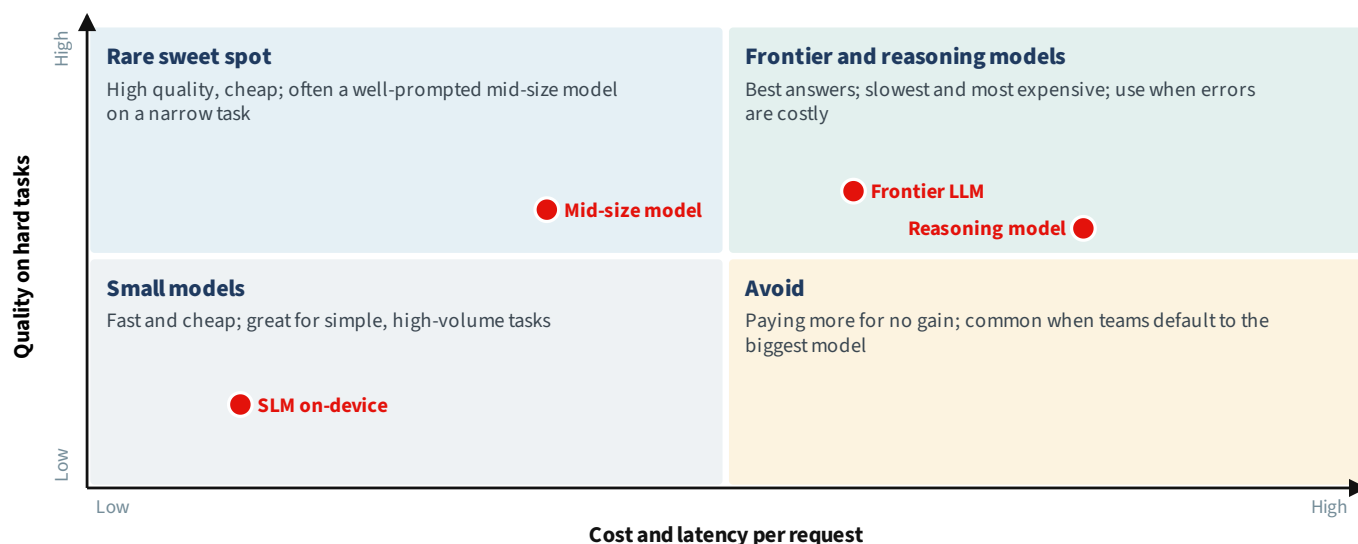
"LLM" is used as a catch-all, but product teams choose among quite different kinds of models.

Model type	What it is	Typical PM use
Frontier LLM	Largest general-purpose models; best quality, highest cost and latency	Complex assistants, research, hard writing and coding
Small language model (SLM)	Compact, fast, cheap; can run on-device or at huge volume	Classify tickets, extract fields, short summaries, autocomplete
Reasoning model	Spends extra tokens "thinking" step by step before answering	Multi-step analysis, maths, planning, tricky code; slower and costlier per answer
Multimodal model	Takes text, images, audio, sometimes video, together	Read a photo of a bill, understand a screenshot, voice conversations
Image / video generation	Creates visuals from prompts	Creatives, mock-ups, catalogue images
Speech models	Speech-to-text and text-to-speech	Voice bots, call-centre transcription, vernacular access
Domain or regional model	Tuned for a field (law, medicine, finance) or languages; for example, India's IndiaAI Mission backs home-grown models focused on Indian languages	Contract review, clinical notes, Indic-language assistants

Note one common confusion: an **AI agent** is not a type of model. It is a *system* that uses one or more models together with tools, memory and a loop of actions. Agents get their own chapter.

Choosing among them is a three-way trade between quality, latency and cost, which the chapter on AI product quality covers in detail. The practical pattern is **routing**: send easy requests to a small, fast model and escalate only hard ones to a large or reasoning model.

Model choice is a quality–speed–cost trade



Prompt engineering: directing a brilliant improviser

If an LLM is a gifted improv actor, the prompt is the script and the director's notes. The same model can give a vague, generic answer or an excellent one depending on how it is asked. **Prompt engineering** is the craft of designing inputs that reliably produce useful, correctly formatted outputs.

The same request, prompted weakly and well

Weak prompt	Strong prompt
Prompt Summarise this customer feedback.	Role and goal You are a PM analyst. Goal: help the team prioritise fixes.
What you get A generic paragraph, different every time, mixing bugs with praise, no numbers.	Context Here are 200 app reviews from the last week, between <reviews> tags.
	Task and format Group into Bug, Feature request, UX issue. For each group: count, top 3 themes, one quote. Output as a table.
	Example and rules One labelled example; 'If a review fits none, put it in Other. Do not invent quotes.'

Clear role, context, task, format, examples and rules turn a party trick into a reliable tool.

Technique	What it means	Example
Zero-, one-, few-shot	Give no, one or a few worked examples so the model copies the pattern	"Here are 3 labelled reviews. Label these 20 the same way."
Role prompting	Ask the model to act as a persona to set expertise and tone	"Act as a PM. Summarise this idea for the CEO in five bullets."
Chain-of-thought	Ask it to reason step by step before answering	"Should we launch in Southeast Asia? Think through market size, competition, cost, then decide."
ReAct (reason + act)	Alternate reasoning with tool use, such as web search or a database lookup	"Search current iPhone prices in India, compare with last quarter, then recommend."
Structured output	Demand a fixed format, often JSON, so software can parse it	"Return {intent, urgency, order_id}."
Prompt chaining	Split a big task into steps, each its own prompt	Extract facts → check facts → write summary

Practical rules that hold up: **start simple**, then add detail only where output falls short; **be explicit** about format, length, tone and audience; **say what to do** rather than only what to avoid; **show examples** of good output; **separate instructions from data** with clear markers; and **test and save**: keep prompts in version control and run them against an eval set whenever you change them or the model. Reasoning models do much of the step-by-step thinking on their own, so with them, clarity of goal and context matters more than "think step by step" tricks.

The one-line rule

An LLM predicts plausible text, not truth. Control it with the right model, the right settings and a clear prompt, and give it the facts it needs rather than trusting what it remembers.

Contrasting case. Prompting cannot fix missing knowledge or unreliable facts. If the model does not know your refund policy, the best prompt in the world will produce a confident guess. That problem needs retrieval (RAG), tools, or a narrower task, covered next.

Why a PM cares. Tokens are your unit cost: long prompts, long documents and long answers cost money on every request. Latency is time-to-first-token plus generation speed; streaming the answer word by word makes a 6-second answer feel fast. Temperature and model choice are product decisions tied to the job. And prompts are product assets that need owners, versions and tests, just like code.

In the interview. "Explain how ChatGPT works to your grandmother" (a super-autocomplete that has read a library and predicts the next word, very well), "Why does it hallucinate?" (it optimises for plausible, not true; no built-in fact-check; knowledge cutoff), "What would you set temperature to for a banking support bot?" (low, for consistency), "How would you reduce cost for an AI feature?" (smaller model for easy cases, shorter prompts, caching, limits on output length). A strong line: *"I'd treat the model as a component with a price per token and a quality score, and pick the smallest one that clears our eval bar."*

Check yourself

1. In one sentence, what does an LLM actually do? Walk through the five-step generation loop.
2. What is a token, and why can the same answer cost more in Hindi than in English?
3. Explain attention using the 'robot and box' sentence.
4. What are pre-training and reinforcement learning from feedback for?
5. What do temperature, top-k and top-p change? Pick settings for invoice extraction and for brainstorming.
6. Why is an AI agent not a 'type of model'?
7. Rewrite "Analyse our churn" as a strong prompt using role, context, task, format and rules.
8. (From the ML chapter) How is the self-supervised pre-training of an LLM different from supervised learning for fraud detection?

TECHNOLOGY · RAG, TOOLS AND MCP

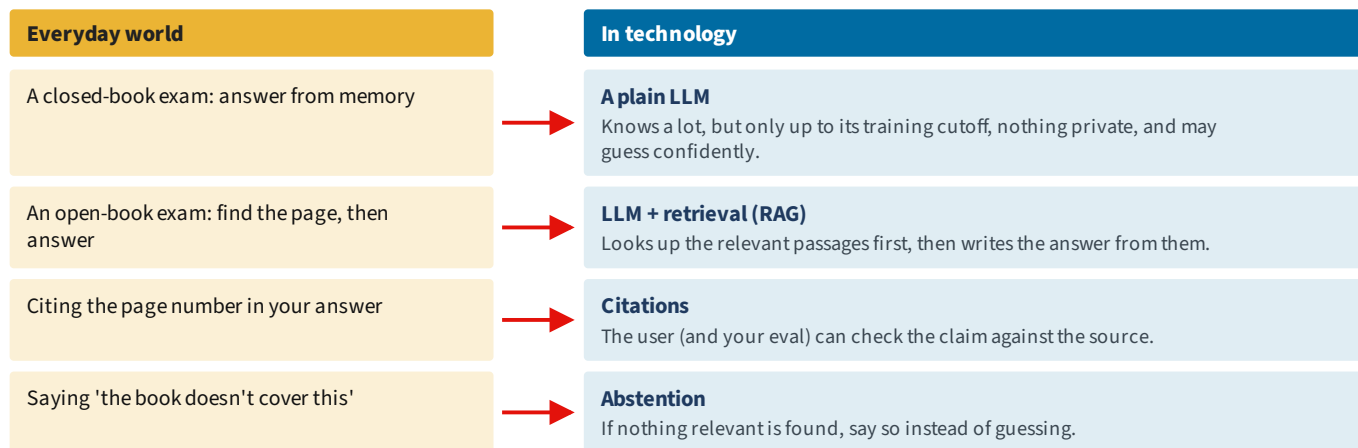
Giving the model a library card and a pair of hands

How AI products answer from your own, current data (RAG), how they take actions through tools, and why MCP became the common plug for connecting them

Ask a general-purpose chatbot, "How much checked baggage can I take on my Kochi–Dubai flight next week?" and it may give a confident, generic answer based on what airlines typically allowed when it was trained. Ask the airline's own assistant the same question and it first looks up the airline's *current* baggage policy for that route and your fare type, then answers: "30 kg on your Saver fare; see section 4.2 of the baggage policy", with a link. The model inside both could be identical. The difference is the system built around it.

That system is called **Retrieval-Augmented Generation (RAG)**. A common misconception is that RAG is a feature of the model. It is not. It is an **architecture around the model**: a retrieval step that finds relevant information and places it in the prompt, so the model generates its answer from that material rather than from memory.

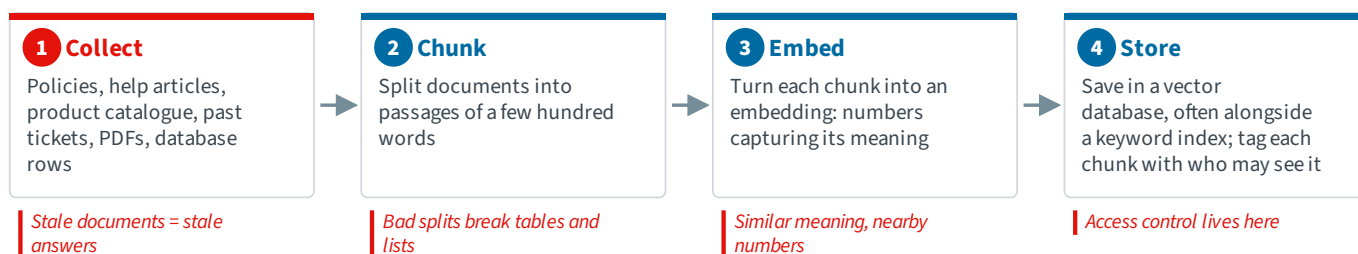
RAG is an open-book exam



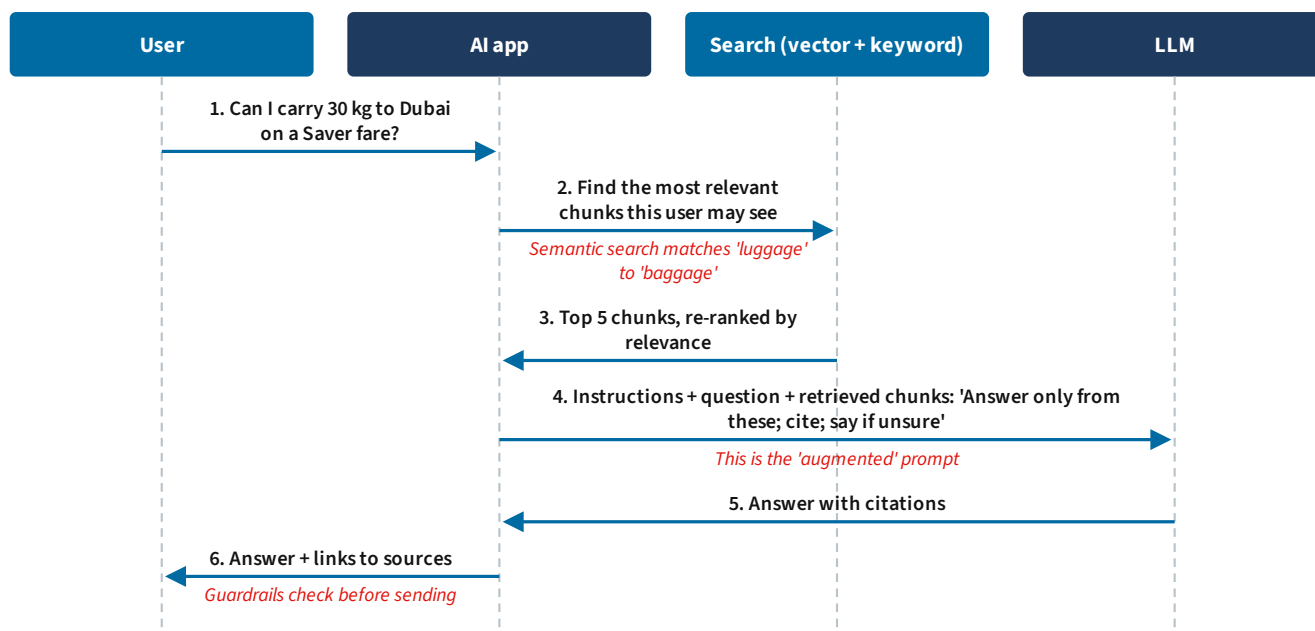
How RAG works

RAG has two phases: preparing the library ahead of time, and using it when a question arrives.

Phase 1, done in advance: building the searchable library



Phase 2, for every question: retrieve, then generate



The knowledge sources can be almost anything: web search results (how chat assistants answer questions about last night's match), internal documents (an HR policy assistant), live databases (order status), knowledge graphs, or the user's own history (past purchases for personalised suggestions).

What RAG fixes, and what it does not

RAG helps with the four biggest weaknesses of a plain model. It supplies **current** information without retraining the model. It supplies **private** information the model never saw, such as your policies and your customer's order. It **reduces hallucination** by giving the model the facts, and makes answers **checkable** through citations. And it can **respect permissions**: an employee assistant

should only retrieve documents that employee is allowed to read. It is also far cheaper and faster to update than retraining a model: change the document and the next answer changes.

But RAG only moves the problem to retrieval. **If the search returns the wrong passage, the model will confidently answer from the wrong passage.** Common failure modes:

- **Retrieval misses:** the right chunk exists but is not in the top results (vocabulary mismatch, poor chunking, a table split in half).
- **Outdated or conflicting documents:** two versions of the refund policy in the index.
- **Ignoring the context:** the model answers from memory despite being told not to; or it misses facts buried in the middle of long context.
- **Prompt injection:** a retrieved web page or uploaded document contains hidden instructions ("ignore previous instructions and..."). Retrieved text must be treated as data, never as commands.

The fixes are engineering-and-product work: hybrid search (meaning plus keywords), re-ranking, cleaning and versioning the source documents, measuring **retrieval quality separately from answer quality** in evals, and designing the model to **abstain** when retrieval comes back empty.

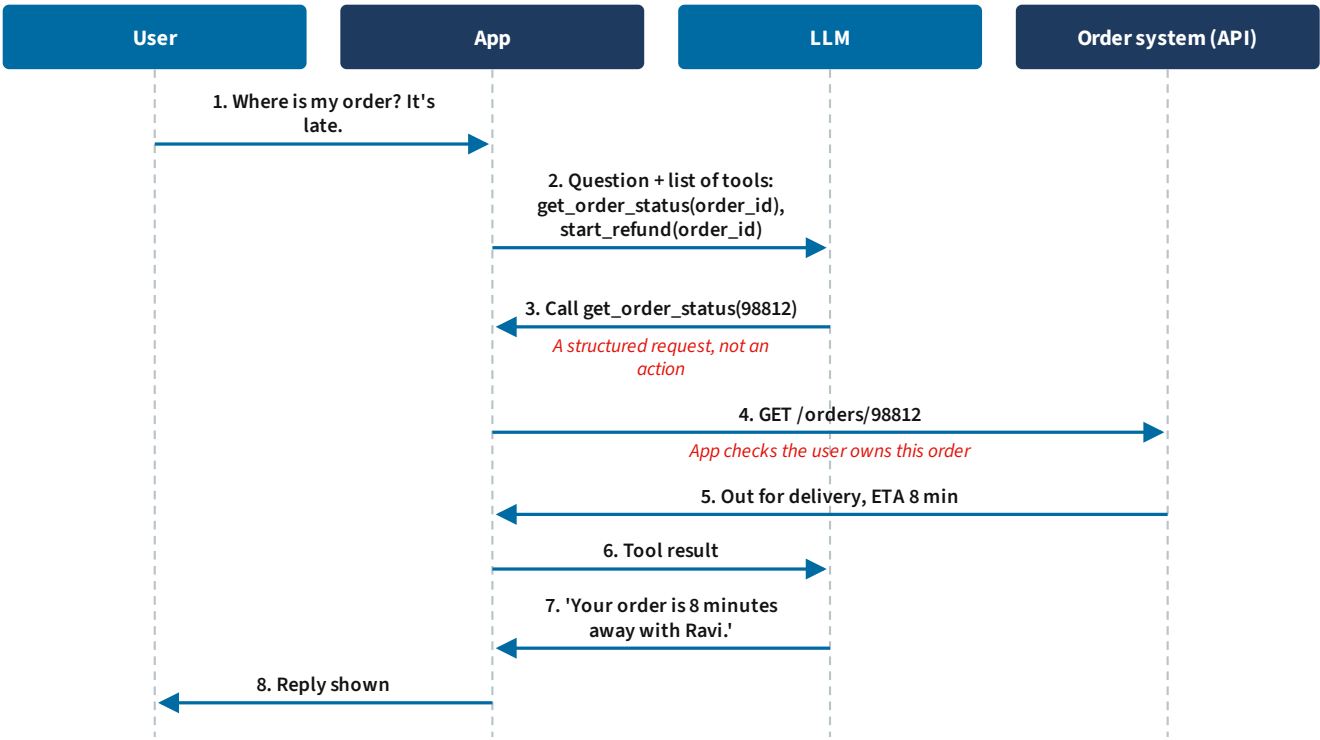
Approach	What it changes	Best for	Watch out
RAG	What the model <i>knows</i> for this answer	Facts that change or are private: policies, catalogues, orders, docs	Answer quality is capped by retrieval quality
Fine-tuning	How the model <i>behaves</i> : style, format, specialised skills	Consistent tone, a fixed output format, a narrow task done at scale	Poor way to add facts; must retrain to update
Long context	Paste everything into the prompt	Small, bounded material: one contract, one report	Cost and latency grow with every token; accuracy can drop over very long inputs
Tools / APIs	What the model can <i>look up or do</i> live	Real-time data (stock, balance) and actions (book, refund)	Needs permissions, error handling and approvals

These are not rivals. A production assistant commonly uses a fine-tuned or well-prompted model, RAG for policies, and tools for live order data, all at once.

Tool use: letting the model act

An LLM on its own can only produce text. It cannot check your order status, book a cab or send an email. **Tool use** (also called **function calling**) bridges this. The developer describes the available tools to the model (name, what it does, which inputs it needs). When the model decides a tool is needed, it replies not with prose but with a structured request. The *application*, not the model, actually runs the tool and hands the result back.

Tool use: the model asks, the application acts



That design keeps control where it belongs: the application decides which tools exist, checks permissions, and can require a human to approve risky calls.

MCP: a common plug for AI tools

Tool use created a new problem. Every AI app (ChatGPT, Claude, Copilot, your own assistant) wanted to connect to every tool (Google Drive, Jira, Slack, Salesforce, GitHub, a company database), and each connection was custom-built. Ten apps and fifty tools meant up to five hundred separate integrations, each with its own format.

The **Model Context Protocol (MCP)** is an open standard that fixes this, the way USB-C replaced a drawer full of chargers. Anthropic introduced it in November 2024; it was adopted across the industry, and in December 2025 it was donated to the Linux Foundation's newly formed **Agentic AI Foundation**, making it vendor-neutral (Linux Foundation and MCP blog, Dec 2025). A tool maker builds **one MCP server**; any AI app that speaks MCP can use it.

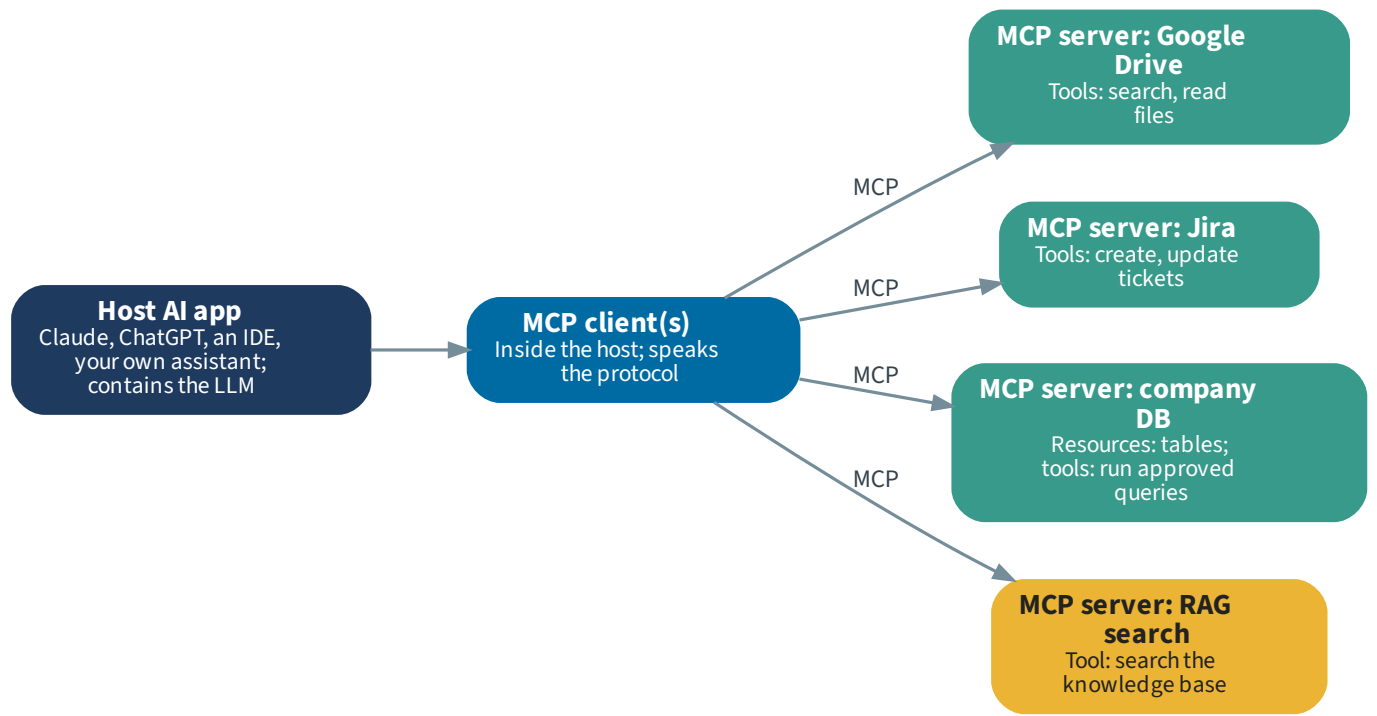
Before and after MCP

Before: custom integrations	After: one standard plug
● Shape Every AI app writes its own connector for every tool: N apps × M tools.	● Shape Each tool ships one MCP server; each app includes one MCP client: N + M pieces.
● Cost Duplicated effort; connectors break when either side changes.	● Cost Build once, works with any MCP-compatible app.
● Result Most tools were unavailable to most assistants.	● Result Assistants can read files, update tickets, query data across many systems.

MCP does for AI tools what USB-C did for chargers: one shape of plug, many devices.

MCP architecture

The host app contains MCP clients; each client talks to one MCP server; each server wraps one system



The pieces: the **host** is the AI application the user works in; the **MCP client** inside it handles the conversation with servers; an **MCP server** wraps a system and translates requests ("add today's figures to the latest sales sheet") into that system's own API calls; the **protocol** defines how messages, tool descriptions and results look. Servers can offer **tools** (actions, such as create a ticket), **resources** (data to read, such as files or records) and **prompts** (reusable templates).

MCP is about access, so it raises security questions a PM must own: which servers are trusted, what each may do, whether writes need user approval, and how to stop a malicious document or server from tricking the model into misusing another tool.

	RAG	MCP
Purpose	Improve what the model knows by supplying relevant information	Extend what the model can reach and do in other systems
How	Retrieve passages, add them to the prompt, generate	Standard client–server protocol exposing tools, resources and prompts
Typical use	Q&A over documents, enterprise search, support answers	Assistants and agents that read and update real systems
Relationship	Complementary: a RAG search can itself be exposed as an MCP tool, and an agent can retrieve with one tool and act with another	

TECHNOLOGY · AGENTIC AI

Agents: from answering to doing

What makes an AI system an agent, how the agent loop works, when agents beat simple workflows, and where they go wrong

A chatbot, asked for a recipe with tomatoes, writes a recipe. An **agent** given the goal "keep my kitchen stocked and plan dinners" checks what is in the pantry, plans the week's meals, notices you are out of onions, adds them to your grocery list and, if you have allowed it, places an order on a quick-commerce app. The difference is not a smarter model. It is that the system **plans and takes actions over several steps, using tools, towards a goal**.

Agentic AI refers to systems in which a model decides which steps to take and which tools to use to reach a goal, observing the results and adjusting as it goes. Autonomy is a **dial, not a switch**: an agent may suggest every action for approval, act alone on low-risk steps, or run end to end within tight permissions.

Workflow, generative AI, agent

Rule-based workflow

Fixed steps written in advance. 'When the inbox exceeds 10,000 emails, send an alert.' Reliable and cheap; cannot handle anything unforeseen.

Generative AI (single call)

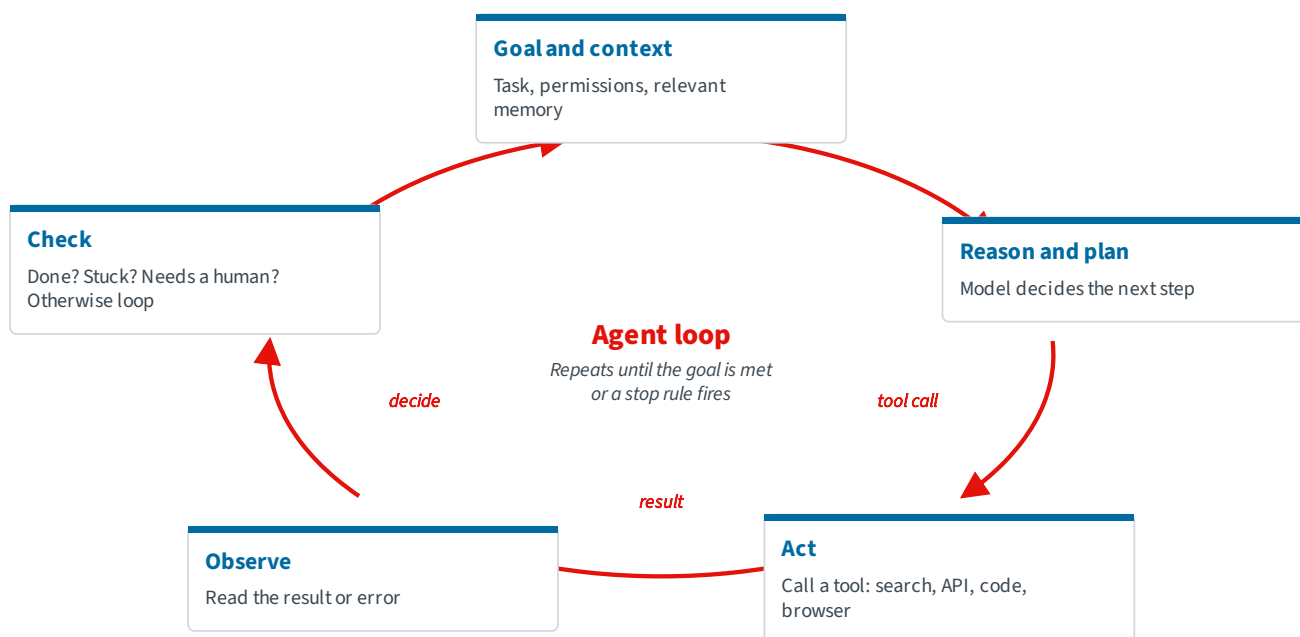
Responds to one prompt with one output. 'Write a recipe using tomatoes.' Reactive; a human drives each step.

AI agent

Pursues a goal in a loop: plans, calls tools, checks results, adapts. 'Clean up my inbox: sort by relevance, summarise, draft replies.'

The agent loop

How an agent works



Take a returns agent at a fashion e-commerce company. Goal: "resolve this customer's return request". It reads the chat, calls `get_order` to see the items and dates, calls `get_return_policy` (a RAG search), sees the item is within 30 days, calls `schedule_pickup`, and then asks a human to approve a refund above ₹5,000 before calling `issue_refund`. Five tools, several decisions, one goal.

Agents are typically **specialised**: given a narrow goal and a limited set of tools and permissions. Complex products increasingly combine several (**multi-agent systems**): one agent plans, others search, write code or check the work. Note one correction to a common description: agents do not automatically "learn from experience" like a model being retrained. They improve only if the product is designed to give them memory, feedback and updated instructions, and if someone re-tests them.

When should you use an agent?

Practitioners' advice, including Anthropic's widely cited guide *Building Effective Agents* (December 2024), is to **use the simplest thing that works**: a single well-prompted call, then a fixed workflow with a few model calls, and only then an agent. Agents earn their cost when the path cannot be scripted in advance.

Four questions before building an agent

Is the task open-ended?

If the steps are predictable, a workflow is cheaper, faster and easier to test. Agents suit tasks where the path depends on what is found.

Is being wrong cheap or reversible?

Filing a document in the wrong folder is easily fixed; a wrong medical dose or payment is not. High stakes: keep the AI advisory.

Can success be checked?

Code that must pass tests or a form that must validate gives the agent feedback. No check means errors go unnoticed.

Are tools and permissions ready?

The agent needs reliable APIs, least-privilege access, and approval steps for irreversible actions.

What goes wrong

- **Errors compound.** If each step succeeds 95% of the time, a 10-step task succeeds only about 60% of the time ($0.95^{10} \approx 0.60$). Reliability per step matters enormously; so do checkpoints.
- **Goal misalignment.** A vague goal like "maximise sales" could lead an agent to spam every customer with discounts. Goals need constraints, budgets and forbidden actions spelled out.
- **Latency and cost.** Each loop is one or more model calls; an agent can take minutes and many times the tokens of a single answer. Fine for a background research task, wrong for a checkout page.
- **Security.** An agent that reads the web or emails can be manipulated by injected instructions, and an agent with broad permissions can do broad damage. Least privilege, sandboxing, allow-lists and human approval for sensitive actions are the defences.
- **Accountability.** When an agent makes a mistake, who owns it? Logs of every step, clear escalation paths and a product decision on which actions need a human.

Agent vocabulary

Tool / function calling: the model requests an action; the app executes it. **Orchestration:** the code that runs the loop, passes results and enforces limits. **Memory:** information kept across steps or sessions. **Guardrails:** rules that block or check inputs, outputs and actions. **Human-in-the-loop:** a person approves or corrects at chosen points. **Task success rate:** share of tasks completed correctly end to end, the headline agent metric, alongside cost per task, time per task and escalation rate.

Why agents matter for a PM

For the **product**, agents open up multi-step, cross-app tasks without constant user input: a travel app that re-plans your connection when a flight is delayed; a shopping app that tells your list app when a wishlisted item is back in stock; a support system that actually resolves a return instead of explaining the policy. For the **PM's own work**, agents already update tickets and documents across tools, prepare meeting briefs, draft PRDs and A/B-test ideas from research notes, and, through coding agents such as Claude Code, Codex and Cursor, build working prototypes from a description.

The one-line rule

RAG gives the model knowledge; tools and MCP give it reach; an agent gives it a loop. Add each only when the simpler option fails, and put a human wherever an action is costly or irreversible.

In the interview. Expect "What is RAG and why use it?" (open-book exam; current and private knowledge; citations; fails when retrieval fails), "RAG or fine-tuning for a support bot?" (RAG for policy facts, fine-tuning only for tone or format), "What is MCP?" (USB-C for AI tools, host-client-server, tools and resources), "Design an AI agent for X" (goal, tools, permissions, where humans approve, how to measure task success and cost, what happens when a tool fails). A strong line: *"I'd start with a workflow that handles the top three return reasons deterministically, and use an agent only for the long tail, with refunds above a threshold always going to a person."*

Check yourself

1. Why is it wrong to call RAG a feature of the LLM? What is it instead?
2. Describe the two phases of RAG and name the step where access control is enforced.
3. Give two ways RAG can still produce a wrong answer, and one fix for each.
4. When would you choose fine-tuning over RAG? When long context?
5. In tool use, who actually executes the action: the model or the application? Why does that matter?
6. What problem does MCP solve? Name its three parts and three things a server can expose.
7. Why does a 10-step agent with 95% per-step accuracy fail about 40% of the time?
8. (From the LLM chapter) Why would you run a returns agent at low temperature?

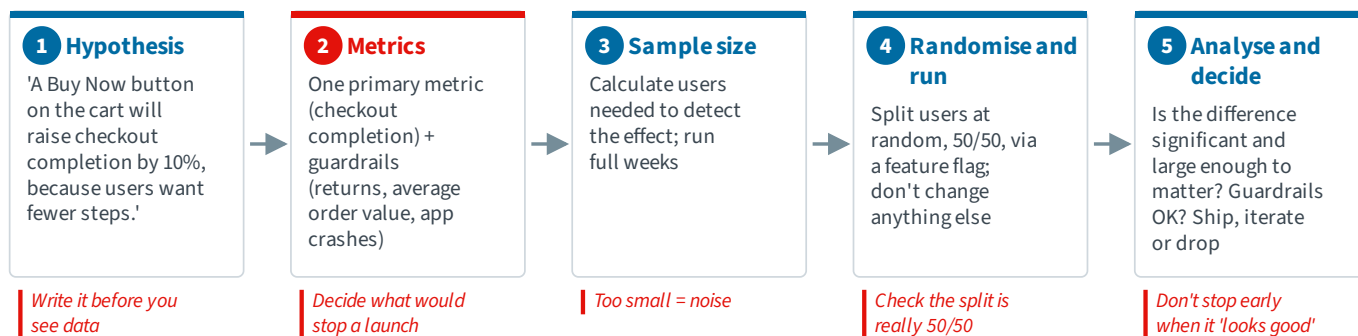
TECHNOLOGY • MEASURING WHAT WORKS

A/B tests: letting users settle the argument

How controlled experiments work, how big they need to be, and the traps that make a winning test lie

The design team at an e-commerce company is split. Half believe a "Buy Now" button on the cart page will cut drop-offs; the other half think it will confuse users who want to keep shopping. The meeting could go on for weeks. Instead, the PM proposes a test: for two weeks, half of shoppers (chosen at random) see the current cart, and half see the cart with "Buy Now". At the end, the variant's checkout completion is 14% higher than the control's, and the difference is far too large to be chance. The button ships to everyone. Nobody had to win the argument; the users settled it.

An **A/B test** is a controlled experiment: you randomly split users into a **control** group (A, the current experience) and a **variant** group (B, the change), and compare them on a metric you chose in advance. Because the split is random, the only systematic difference between the groups is the change, so any significant difference in outcomes can be credited to it. It is the product world's version of a randomised controlled trial in medicine.

Running an A/B test, step by step**How big must the test be?**

Flip a coin ten times and getting seven heads is not surprising. Flip it ten thousand times and 70% heads would be astonishing. Tests work the same way: small differences in small samples are often just noise. **Statistical significance** is the check that a difference is unlikely to be a fluke. By convention teams accept a result when there is less than a 5% chance that a difference this large would appear if the change actually did nothing (the "p-value below 0.05"). **Statistical power** (usually 80%) is the chance of detecting a real effect of the size you care about.

A handy rule of thumb for comparing conversion rates at those standard settings:

Rough sample size per group

Lehr's rule of thumb for 5% significance and 80% power

$$\text{Users per group} \approx 16 \times p(1 - p) \div \delta^2 = 16 \times 0.16 \div 0.0004 \approx 6,400$$

16 A constant from the chosen significance and power	×	$p \times (1 - p)$ Variability of the baseline rate; for 20% conversion: 0.2×0.8	÷	δ^2 Square of the smallest change worth detecting; 2 percentage points = 0.02^2
16		0.16		0.0004

Source: Standard approximation; use a proper calculator for real tests

The key intuition: **halving the effect you want to detect needs four times the users.** That is why small apps cannot test tiny tweaks, and why big apps such as Flipkart or Swiggy run hundreds of experiments at once.

The traps that make a winning test lie

- **Peeking.** Checking every day and stopping the moment the result looks significant inflates false wins dramatically. Fix the duration in advance.
- **Novelty and learning effects.** A new design gets clicks because it is new; the lift fades after two weeks. Or the reverse: users need time to learn a better flow. Run long enough, and look at the trend.
- **Too many metrics.** Test twenty metrics and one will look significant by chance. Name one primary metric beforehand.
- **Weekday effects.** Shopping on Sunday differs from Tuesday. Run whole weeks.
- **Broken randomisation.** If the split is 50.0/50.0 by design but 52/48 in practice (a **sample ratio mismatch**), something is wrong with assignment; the result cannot be trusted.
- **Network effects and interference.** In a marketplace, treating one group changes the other. If variant riders get priority orders, control riders get fewer, and both groups are affected. Delivery and ride-hailing companies use **switchback tests** (whole cities alternate between A and B by time slot) or **geo tests** (compare matched cities) instead.
- **Winning the metric, losing the business.** A bigger "Buy Now" button may raise conversion and also raise returns. Guardrail metrics catch this.

Contrasting cases. A/B testing is the wrong tool when you have too little traffic, when the change is a big strategic bet whose effect takes months (a new pricing model, a brand repositioning), when it would be unethical or illegal to give some users a worse experience (safety features, regulatory disclosures), or when the change must be tested one variable at a time but you want to compare several elements together (use **multivariate testing**, which needs much more traffic).

The one-line rule

Randomise, decide the metric and the sample size before you start, don't peek, and watch guardrails. A test that is too small or stopped early is worse than no test, because it gives false confidence.

In the interview. "How would you test X?" wants hypothesis → primary and guardrail metrics → unit of randomisation (user, session, city) → sample size and duration → how you would decide. Strong line: *"For a rider-incentive change I wouldn't split riders within a city, because treated riders take orders from control riders. I'd run a switchback by city and hour and compare delivery time and cost per order."* Also be ready for: "The test shows +3% conversion but -2% revenue per user. Ship?" (depends on which is the goal metric and the long-term effect; dig into which segment drove each).

TECHNOLOGY · AI PRODUCT QUALITY

Evals: the PRD of an AI feature

How to define and test 'good enough' for a system whose outputs you cannot fully predict

For a normal feature, a tester can check that the "Cancel order" button cancels the order. For an AI support assistant, there are thousands of ways a customer can ask about a refund and many acceptable answers, and the same question can get slightly different

answers each time. You cannot write down every correct output in advance. So how do you know it is good enough to launch, and how do you know a new model is better rather than just different?

An **eval** (evaluation) is a test for an AI system: you give it an input, grade the output with some grading logic, and measure how often it succeeds. A set of evals is how a team turns "good enough" into a number that can gate a launch.

Two kinds, and you need both

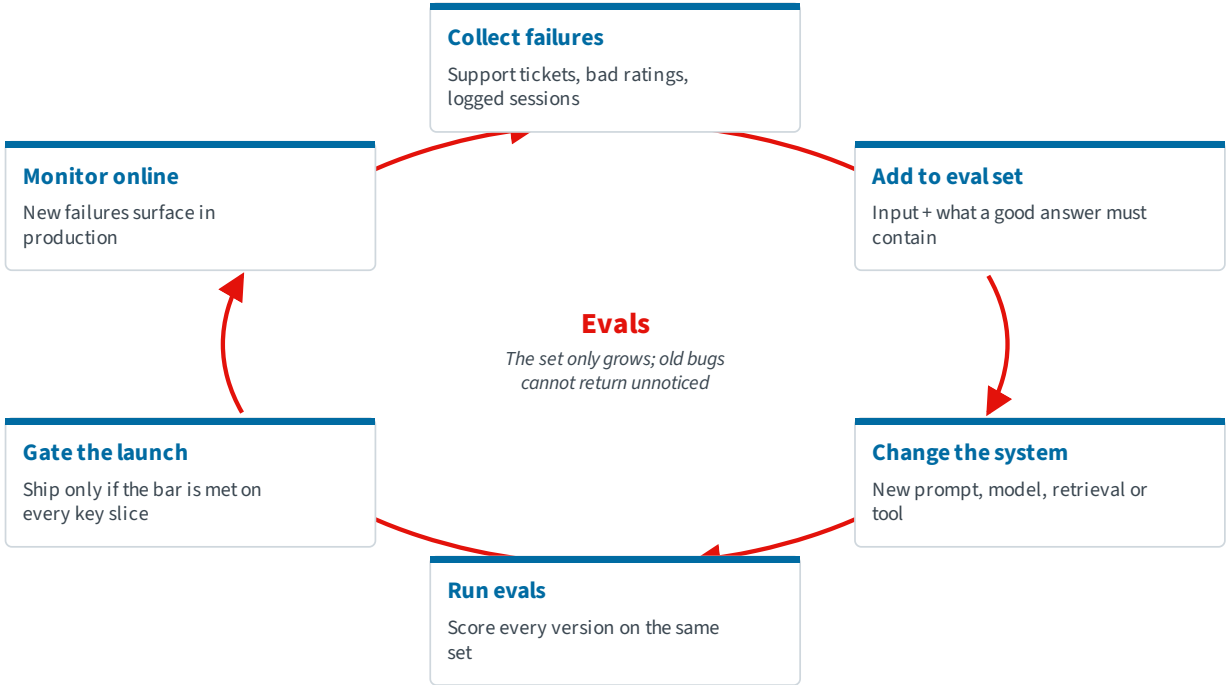
	Offline evals	Online metrics
When	During development, before shipping	In production, with real users
Data	A fixed, labelled set of test cases	Live traffic
Question answered	"Is the system good at the task?"	"Are users better off?"
Examples	Correct-answer rate on 300 real support conversations; hallucination rate on a labelled sample	Resolution rate, escalations to humans, CSAT, repeat contacts, A/B test results

A model can pass offline evals and still disappoint users (the test set did not reflect real questions), or look popular online while quietly giving wrong answers (users do not know the answer is wrong). Hence both.

Where the test cases come from

Build the eval set from **real failures**: your support queue, logged sessions, bug reports and complaints, not imagined scenarios. You do not need thousands to start. Anthropic's guidance on evaluating agents (January 2026) is that **20 to 50 tasks drawn from real failures** is a strong start. Every new failure you find becomes a new test case, so the set grows into a safety net that stops old mistakes from coming back.

The eval flywheel



Who grades the output?

Three kinds of grader

Code-based checks

Exact match, valid JSON, required fields present, test cases pass, cited source exists. Fast, cheap, objective; only works where 'correct' can be checked mechanically.

Human graders

Experts label outputs against a rubric. Most accurate for nuanced quality; slow and costly. A few hundred well-labelled examples go a long way.

Model as judge

Another LLM grades against a rubric. Cheap and scalable. Needs a clear rubric, an 'unknown' option so it does not invent a verdict, and regular checks against human labels.

A worked example. Before switching the model behind a support chatbot, the team runs 300 labelled conversations covering the top ten intents. The bar was set in advance: hallucination rate under 2%, and **no drop against the current model on any single intent**. The new model scores better overall but 8% worse on refunds. It does not ship for refunds; those failed conversations join the eval set so the same regression cannot slip through again.

Why a PM cares. The eval set is the closest thing an AI feature has to a PRD: it is where the quality bar actually lives. Without one you cannot tell an improvement from a regression. The rule of thumb in good AI teams: **no eval set, no launch**; if you cannot say which score would stop a launch, you do not have a bar. And **gate launches on quality metrics, never on engagement**: a chatbot that confidently gives wrong refund information can have excellent engagement.

TECHNOLOGY · AI PRODUCT QUALITY

Precision, recall and the threshold dial

Two ways to measure a model's mistakes, and why choosing between them is a product decision, not an ML one

A marketplace uses a model to flag fake listings. Every decision the model makes lands in one of four boxes.

The four outcomes of every yes/no prediction

Example: a model that flags listings as fake

True positive (TP)

Flagged, and it really was fake. A catch.

False positive (FP)

Flagged, but it was genuine. A false alarm: a real seller wrongly punished.

False negative (FN)

Let through, but it was fake. A miss: a buyer may get cheated.

True negative (TN)

Let through, and it was genuine. Business as usual.

Two metrics are built from these boxes:

- **Precision** = $TP \div (TP + FP)$. Of everything the model flagged, how much was right? Flag 100 listings as fake, 80 truly are: precision is 80%.
- **Recall** = $TP \div (TP + FN)$. Of everything that was actually fake, how much did the model catch? If 200 fake listings exist and the model catches 80: recall is 40%.

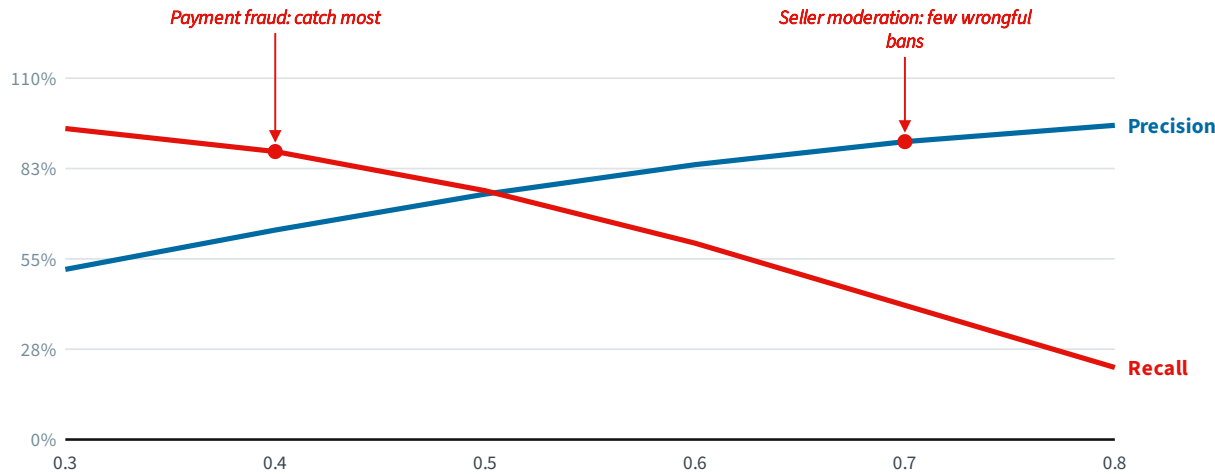
Why not just use **accuracy** (share of all decisions that were right)? Because when the thing you are looking for is rare, accuracy lies. If 1 in 100 payments is fraud, a "model" that never flags anything is 99% accurate and completely useless: its recall is zero.

The threshold: one dial, two metrics

Most models do not output "fake" or "genuine"; they output a score, such as 0.73. The **threshold** is the cut-off above which the product acts. Raise it and the model only flags cases it is very sure about: fewer false alarms (precision up) but more misses (recall down). Lower it and it catches more, but also flags more genuine cases. **Precision and recall trade off against each other** as you move the dial.

Moving the threshold trades precision for recall

Fake-listing model, illustrative



Calibration matters too: a score of 0.9 means "90% likely" only if the model is calibrated, and many are not. Check before you build rules like "auto-ban above 0.9".

Same maths, opposite answers

Which mistake hurts more?

Seller moderation: protect precision	Payment fraud: protect recall
● False positive Bans a real seller and takes away their income; damages trust with the supply side.	● False positive One genuine payment is blocked; the user verifies and retries.
● False negative A fake listing stays up a bit longer; can be caught by buyer reports.	● False negative Real money is lost; the fraudster tries again.
● Choice High threshold; send borderline cases to human review.	● Choice Lower threshold; use step-up checks (OTP) instead of hard blocks.

The engineer can set the threshold anywhere; only the PM can say which mistake hurts the user and the business more.

Why a PM cares. "Improve accuracy" tells the team nothing. "Protect precision: we cannot ban genuine sellers; route anything between 0.6 and 0.85 to human review" tells them exactly what to build. Moving the threshold is also the cheapest lever you have: it needs no retraining. A related single number, **F1**, balances precision and recall, but a PM should usually say which one matters more rather than hide the choice in an average.

Words to know

Confusion matrix: the 2x2 table of TP, FP, FN, TN. **Precision:** of flagged, share correct. **Recall (sensitivity):** of actual positives, share caught. **False positive rate:** of actual negatives, share wrongly flagged. **Threshold:** the score cut-off for acting. **Calibration:** whether a score of 0.8 really means 80%. **F1:** the harmonic mean of precision and recall.

Guardrails and human-in-the-loop

Deciding what an AI system may say and do on its own, and where a person must step in

Air Canada learned in 2024 that a chatbot's words are the company's words: a Canadian tribunal ordered it to compensate a passenger whom its website chatbot had misinformed about bereavement-fare refunds, rejecting the airline's argument that the chatbot was

responsible for its own words (Moffatt v. Air Canada, February 2024). Users forgive "I don't know". They do not forgive being confidently misled, and neither do regulators.

Guardrails are the controls that decide what an AI system may send to a user or do in the world. **Human-in-the-loop (HITL)** design decides where a person must step in.

Guardrails: stopping confident-but-wrong output

The main guardrails, and what each costs

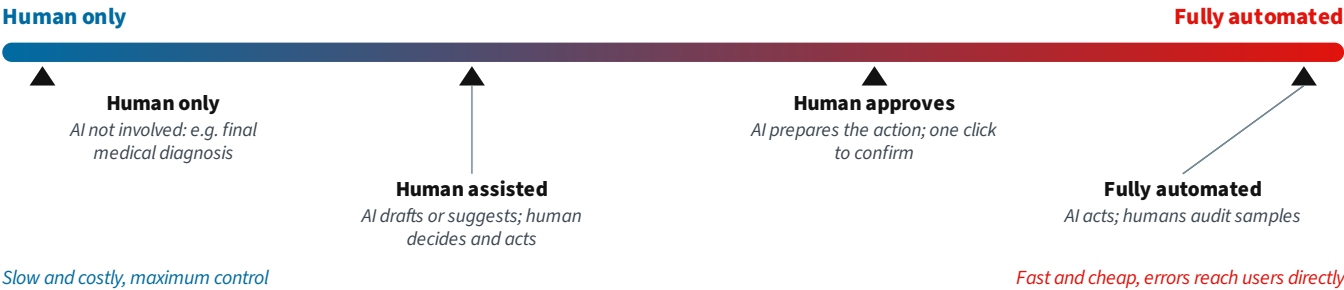
Grounding Answer only from retrieved sources, not from the model's memory. Costs: fewer questions answered.	Citations Show the source beside the claim so users can check. Costs: UI space; sources must be good.	Abstention If nothing relevant is retrieved, say 'I don't have that information' and offer a human. Costs: lower coverage.
Topic gates Route legal, medical, financial and other high-risk topics to a human whatever the confidence score. Costs: human time.	Input and output filters Block abuse, PII leaks, unsafe content and prompt-injection attempts. Costs: some false blocks.	Action limits Caps and allow-lists on what tools may do: refund ≤ ₹1,000, no deletes. Costs: more escalations.

The launch gate for a generative feature is usually the **hallucination rate**: the share of outputs containing a claim not supported by the sources, measured on a labelled sample. It must be measured directly, never inferred from complaint volume, because most users who are misled never realise it.

Guardrails cost **coverage**: fewer questions get answers, but the answers given can be trusted. Where to draw that line is the PM's call.

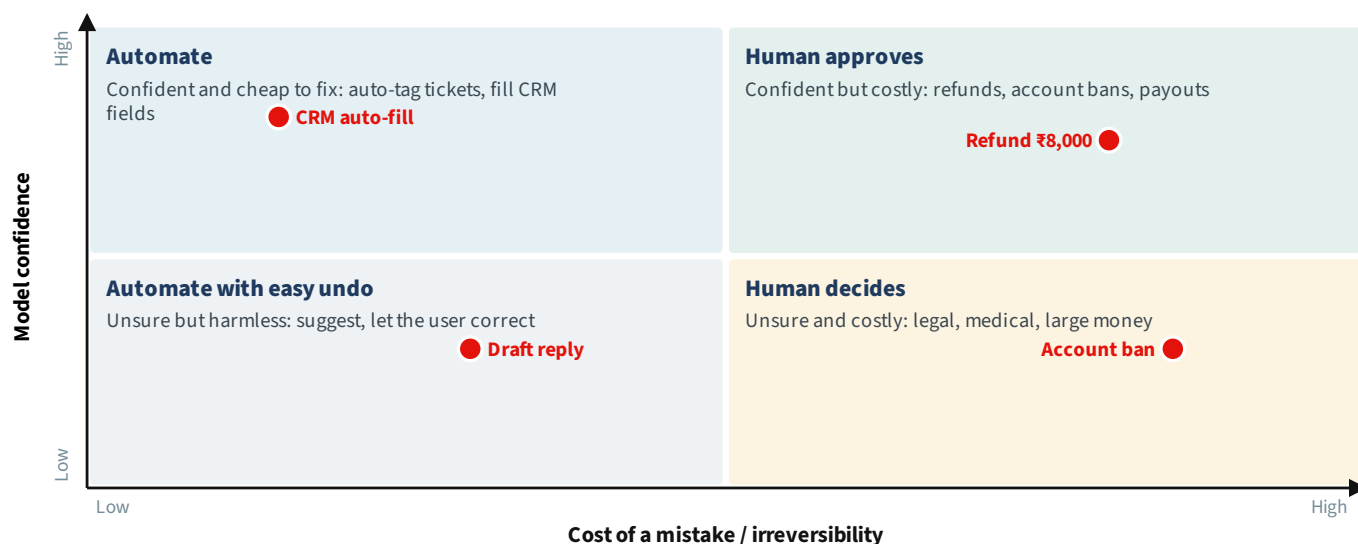
Human-in-the-loop: how much does the machine do alone?

The automation spectrum



Place the human gate on two axes: **confidence** (low-confidence cases go to a person) and **risk** (sensitive or irreversible actions go to a person regardless of confidence). The deciding variable is **reversibility**: **automate what can be undone; gate what cannot**.

Where to put the human



A worked example: an AI copilot for customer-support agents, with three automation levels chosen by reversibility. It **drafts replies**; the agent edits and sends, staying accountable for every message. It **auto-fills internal CRM fields** with no approval, because they are internal and easily corrected. It **never issues refunds on its own**, because refunds move money and cannot easily be undone. Same product, same model, three different answers, because the cost of being wrong differs.

One trap to design against is **automation bias**: when the model is usually right, people stop checking and start rubber-stamping. Countermeasures include showing the evidence behind a suggestion, occasionally sampling decisions for review, requiring edits on some cases, and tracking how often humans override the AI (an override rate near zero is a warning sign, not a success).

Why a PM cares. HITL design decides who is accountable when the system is wrong, a question that always comes up and is always a PM question. Raising it unprompted in an interview signals that you treat AI as **decision support with clear responsibility**, not as magic.

TECHNOLOGY · AI PRODUCT QUALITY

Cost per task: the unit economics of AI

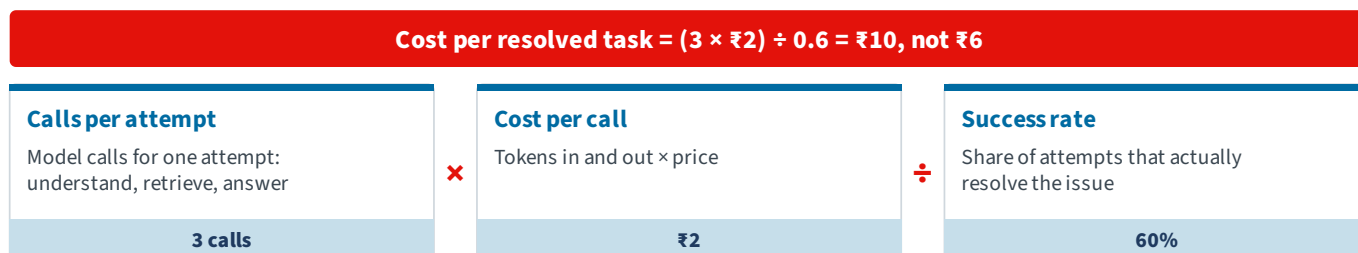
Why AI features have a real cost every time they are used, how to count it honestly, and how it changes pricing

Traditional software is expensive to build and almost free to run for one more user. AI features are different: **every use costs money**, in model calls, retrieval and compute, and that cost never falls to zero.

Cost per task is the full cost of an AI system delivering one completed unit of user value, not the price of one API call. The honest version is **cost per resolved task**, which counts retries, escalations and failures.

Cost per resolved task

Worked example from a support assistant



The quality–latency–cost triangle

Every AI feature trades three things against each other, and you cannot set all three independently.

Corner	What it means	How to measure
Quality	How good the output is	Your eval score, not user opinion
Latency	How long the user waits	p95, not the average; time-to-first-token for chat
Cost	What each completed task costs	Cost per resolved task, including retries and escalations

Why they move together: a bigger model has more parameters, so every token it produces takes more computation; more computation costs more and takes longer. Reasoning models add "thinking" tokens on top. Pushing quality up drags latency and cost with it, unless you get clever.

The levers for bringing cost down without losing quality

Model routing

Send easy requests to a small, cheap model; escalate only hard ones to a large or reasoning model.

Caching

Never pay twice for the same work: cache repeated answers and reuse long, fixed prompt prefixes (prompt caching).

Narrow the scope

A focused feature ('track my order') costs far less per task than an open-ended 'ask me anything'.

Trim the tokens

Shorter prompts, fewer retrieved chunks, capped output length.

Batch and defer

Non-urgent jobs (overnight summaries) can run in cheaper batch mode.

Distil

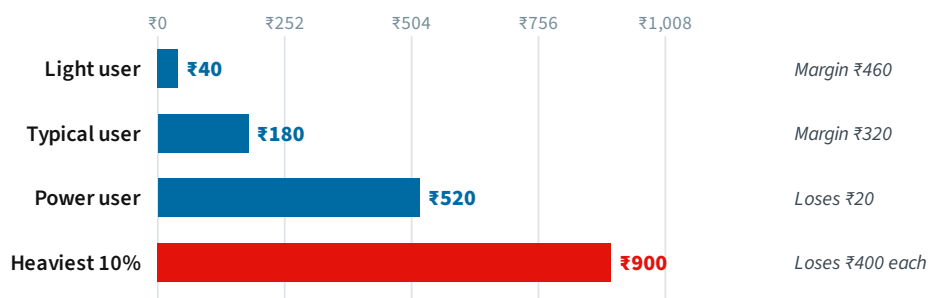
Train a small model on a big model's outputs for one narrow, high-volume task.

Why this breaks per-seat pricing

Andreessen Horowitz found in 2020 that AI companies often spent a quarter or more of revenue on cloud resources, with gross margins materially below comparable software businesses (a16z, *The New Business of AI*). Usage-linked costs create a new pricing problem.

Same price, very different cost to serve

An AI assistant at ₹500 per seat per month; monthly cost to serve by user type (illustrative)



A flat fee against usage-linked cost means **the more successful the feature, the worse the margin can get**, and your heaviest, most enthusiastic users can be your least profitable. The average margin hides this. Every fix is a product decision: route simple queries to a cheaper model; cache repeated answers; cap usage in the base tier; or move from seat pricing to **usage-based** or **credit-based** pricing, or to **outcome-based** pricing (charge per resolved ticket), which aligns price with the value delivered.

The one-line rule

Measure AI features by cost per *resolved* task and by quality on your evals, watch p95 latency, and price so that revenue rises with usage. Cost is a product constraint that shapes what you build, not a finance footnote.

In the interview. "If there is an AI product and a non-AI product, how does your pricing strategy differ?" is asked directly. A strong answer: "Non-AI software has near-zero marginal cost, so seat or flat pricing works. AI has a real cost per use that varies ten-fold between users, so I'd combine a base subscription with included usage and pay-as-you-go or credits beyond it, route easy work to small models, and track cost per resolved task by segment so we see loss-making power users early." Other forms: "Our AI feature is popular but losing money; what do you do?", "Should we use the best model available?" (only where the eval gain justifies the cost

and latency), "How would you launch an AI refund assistant?" (evals from real refund tickets, hallucination gate, abstention, human approval above a limit, cost per resolved refund).

Check yourself

1. Walk through the five steps of an A/B test for a new checkout button. What is a guardrail metric?
2. Using the rule of thumb, roughly how many users per group do you need to detect a 1-point change from a 10% baseline?
3. Why are switchback tests used in delivery marketplaces?
4. What is the difference between offline evals and online metrics? Why do you need both?
5. Define precision and recall with the fake-listing example. Why is accuracy misleading for fraud?
6. For seller moderation and for payment fraud, which way would you move the threshold, and why?
7. Place four actions of a support copilot on the confidence–reversibility matrix.
8. Three calls at ₹1.50 each and a 50% success rate: what is the cost per resolved task?
9. (From the RAG chapter) How would you build an eval that separates retrieval failures from generation failures?

TECHNOLOGY • PLATFORM ECONOMICS

Why the big get bigger

Network effects, two-sided pricing, switching costs and multihoming: the economic forces that explain how digital platforms grow, compete and sometimes dominate

Imagine you launch a food-delivery app in Pune with a genuinely better design than Swiggy and Zomato. Restaurants will not list with you because you have no customers. Customers will not download you because you have no restaurants. Riders will not join because there are no orders. Your better app is irrelevant. This is the central fact of platform economics: **for a platform, the product is largely the other people on it.**

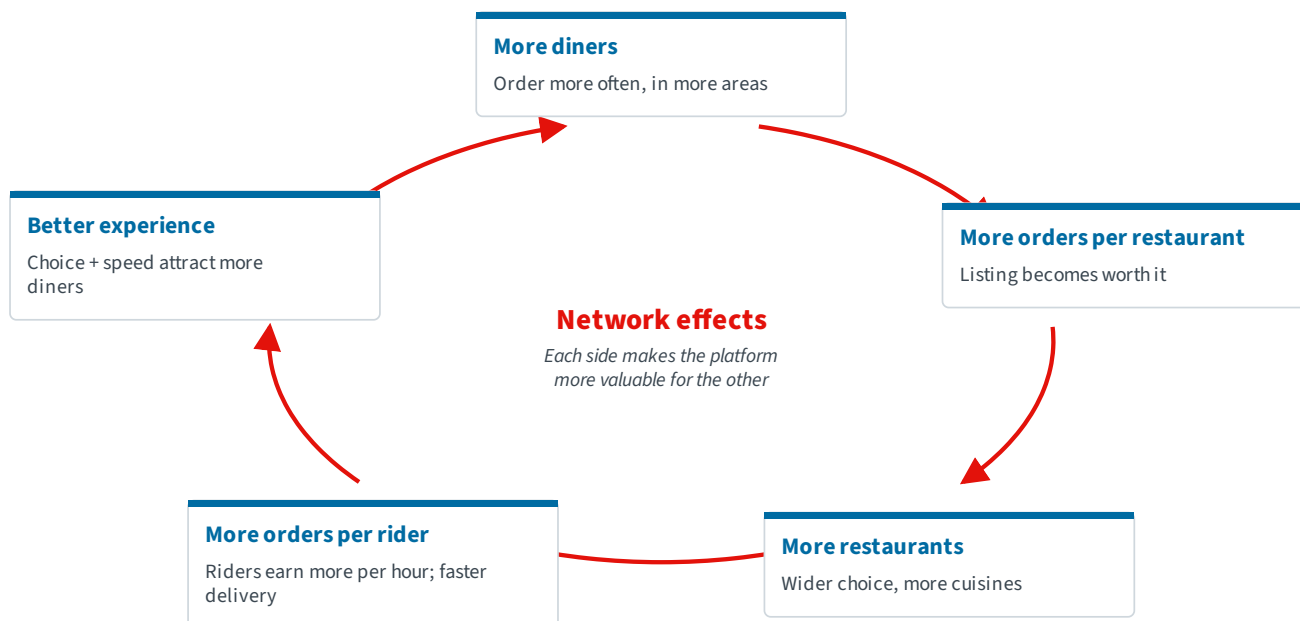
A **digital platform** creates value by connecting groups who want to interact: buyers and sellers (Flipkart, Meesho), riders and drivers (Uber, Rapido), diners, restaurants and delivery partners (Zomato, Swiggy), viewers and creators (YouTube), users and advertisers (Google, Instagram), app users and developers (Apple's App Store). Three forces shape how they grow and compete: **network effects, differentiation and switching costs, and multi-sided pricing.**

Network effects: more users, more value per user

A **network effect** exists when each user gets more value as more users join. A telephone is useless if you own the only one. WhatsApp is valuable because everyone you know is on it.

The cross-side network effect flywheel

The engine behind marketplaces from Airbnb to Swiggy



Network effects come in different kinds, and the kind decides how defensible a platform is.

Two dimensions of network effects



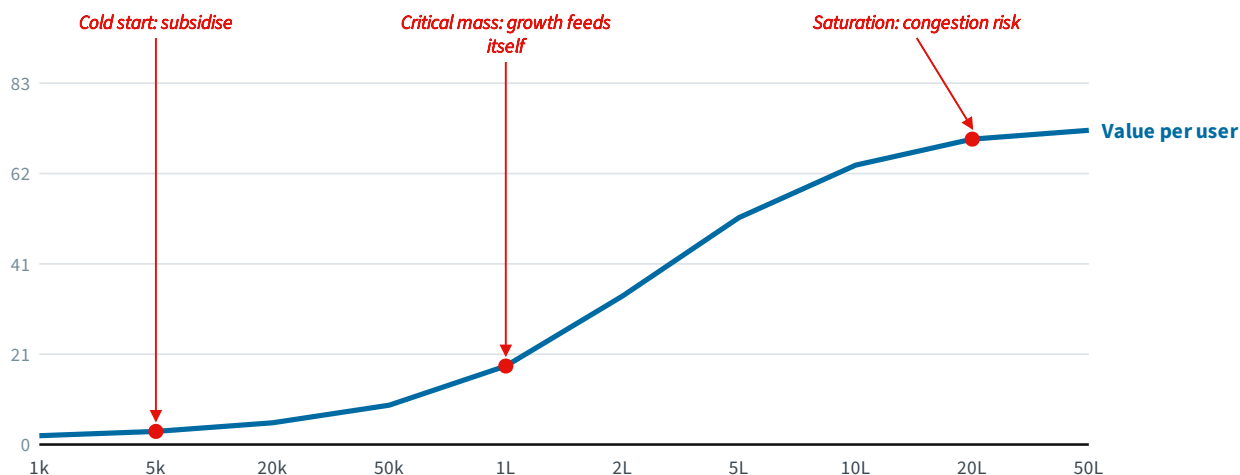
- **Same-side** effects: value rises as more *peers* join (Instagram is better when your friends are on it).
- **Cross-side** effects: value rises as more of the *other* side joins (more Airbnb hosts attract more guests, and vice versa).
- **Local** effects are confined to a geography or segment. They are more contestable: Uber faces a strong local rival in almost every country it enters, because a driver in Pune does nothing for a rider in Jakarta.
- **Global** effects travel: a new host in Goa makes Airbnb more attractive to travellers in London.

A third kind, **data network effects**, arises when more usage improves the product itself: more Google Maps users mean better live traffic data, which means better ETAs for everyone.

Network effects follow an **S-curve**. Early on, value grows slowly because there are too few users to matter; that is the **cold start**. Past a **critical mass**, each new user adds noticeably to everyone's value and growth feeds on itself. Eventually the benefit saturates; the millionth restaurant in Mumbai adds little. Network effects can also turn **negative** through congestion: too many sellers make search noisy, too many riders idle and earn less, too many members spam a group.

The S-curve of network value

Value to each user as the network grows (illustrative)



Differentiation: what makes users pick you

Sustainable differentiation lives **in the customer's perception**, not in internal process improvements. If Uber builds a new algorithm that improves cab availability but riders cannot tell the difference between Uber and Ola, Uber has not differentiated itself.

Two kinds of differentiation

Horizontal: different, not better	Vertical: objectively better
Idea Serves different tastes or identities.	Idea Everyone agrees it is better: lower search cost, more trust, better matches, faster delivery.
Examples Reddit's niche communities; a job board only for designers; a gaming-first chat app versus a work chat app.	Examples Faster delivery, verified sellers, easy returns, better search.
Effect Creates loyal segments; limits the size of each.	Effect Wins broad share, but rivals can often copy it.

Platforms usually win on vertical quality and trust, then cement the lead with horizontal community and identity.

Two levers dominate:

The cost of convergence. Copying a digital product's core features is cheap. Instagram launched Stories in 2016, a near-copy of Snapchat's signature feature, and quickly overtook it. If a rival can copy your advantage in a quarter, it is not an advantage. Late entrants win by differentiating on something hard to copy (supply, community, trust, a niche) rather than fighting a pure scale war.

Switching costs. Raise the real cost of leaving (time, money, convenience, lost benefits) by making staying genuinely better, not by trapping users with dark patterns.

Healthy switching costs a platform can build

Reputation and history A seller's 4.8 rating from 12,000 reviews, a driver's track record: not portable to a rival.	Workflow integrations Catalogues, APIs, payments, logistics, invoicing built around the platform: painful to rebuild elsewhere.	Loyalty and financial hooks Memberships (Swiggy One, Flipkart Plus), fast payouts, working-capital loans to sellers: benefits that accrue over time.
Social graphs Followers, groups, message history: your community lives here.	Personalisation and saved data Preferences, addresses, playlists, watch history that make the experience better the longer you stay.	

Pricing in multi-sided markets

On a platform, the right price for one side depends on the other side. Platforms set **asymmetric prices**: they usually **subsidise the side that is more price-sensitive or that attracts the other side**, and **charge the side that values access more**. Google search is free to users; advertisers pay. Most payment apps are free to consumers; merchants or other revenue lines pay. Food-delivery apps charge restaurants a commission and charge diners delivery and platform fees, with the balance shifting over time.

The split changes over the life cycle: heavy subsidies at launch to cross the cold start (the cashback wars of India's early UPI and wallet days, first-ride-free offers), then a gradual taper as network effects take over. Regulation can fix one side's price: in India, merchant fees (MDR) on most UPI payments have been set at zero since January 2020, which is one reason UPI apps monetise through lending, ads and other services instead of per-payment fees.

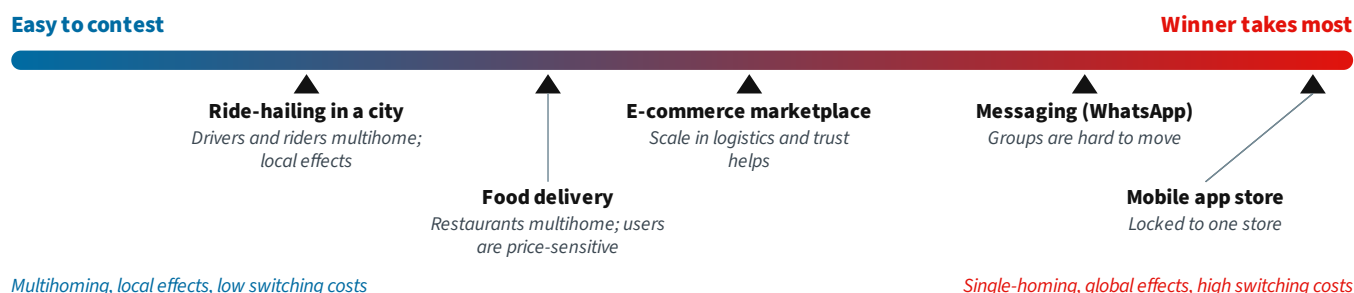
Competition: multihoming, bottlenecks and tipping

Multihoming means using several competing platforms at once. Many Indian restaurants list on both Swiggy and Zomato; many drivers keep Uber, Ola and Rapido open; shoppers compare prices across apps. Where users *and* suppliers multihome easily, rivalry stays active and take rates stay in check.

A **competitive bottleneck** arises when one side is locked in. iPhone owners effectively single-home on Apple's App Store, so developers who want to reach them have no alternative; that is how Apple could charge up to 30% commission for years, a level that would be hard to sustain with several competing app stores on the same phone. That power is now under regulatory pressure in several jurisdictions.

Expectations and tipping. Users join the platform they expect others to join. That can stall a launch (nobody wants to be first) or accelerate dominance once momentum builds and the market "tips" to one player. Announcing supply commitments, exclusive partnerships or big user numbers is partly about shaping expectations.

How contestable is the market?



Strategic issues every platform PM faces

The launching problem. How do you overcome the cold start? Options: subsidise one side; seed supply yourself (early Reddit founders posted content under many accounts; marketplaces often recruit the first sellers by hand); start with a **single-player utility** that is useful before any network exists (OpenTable gave restaurants reservation software first; Instagram's filters were worth using even with no followers); or launch in a tight niche and expand (Facebook started with one college). Expect to burn cash until critical mass.

Surplus and value capture. In a competitive market, value is split between consumers and producers. Platforms use **price discrimination** (auctions for ad slots, personalised offers, tiered seller fees) to capture more of it. Strong network effects raise users' willingness to pay, which increases total value, and a powerful, well-governed platform can claim a large share of it. Push too hard and suppliers leave or regulators step in.

Marketplace or reseller? A pure **marketplace** connects third-party sellers; it scales breadth with little capital and earns a take rate. A **reseller** or **hybrid** model buys inventory and sells it; it controls quality, speed and margin on selected lines but ties up capital. In India, rules on foreign direct investment allow foreign-owned e-commerce firms to operate only as marketplaces, not inventory-led retailers, which shapes how Amazon and Flipkart structure their businesses.

Platform leakage. Once a customer finds a good electrician through Urban Company, why not call him directly next time and skip the fee? Platforms prevent leakage by making on-platform transactions **strictly safer or better**: payment protection, escrow, insurance or warranty on the job, dispute resolution, verified professionals, loyalty benefits.

The one-line rule

A platform's value is the network, not the app. Win by crossing critical mass on the hardest side first, pricing each side for the value it gets and the value it brings, and making it genuinely better to stay and transact on-platform.

Contrasting case. Not every digital business has strong network effects. A calculator app, a single-brand D2C store or most SaaS tools gain little when other people join. Claiming "network effects" for them in an interview is a red flag; say "economies of scale" or "brand" instead.

Why a PM cares. Platform economics decides which side to prioritise in the roadmap (usually the scarce side), which metrics matter (liquidity: the share of searches that end in a transaction; fill rate; time to match), how to price, and where the defensible moat is. It also tells you when a feature war is pointless: if both sides multihome, a copied feature changes little.

In the interview. "How would you launch a new marketplace in a city?" (pick the hard side, seed supply, narrow the geography to reach density, single-player value, subsidise with an exit plan), "Why does Uber struggle to be profitable in some cities?" (local, cross-side effects; multihoming drivers and riders; price competition), "Is Instagram's moat network effects or something else?" (same-side global effects plus creator supply, data and habit), "Should Zomato raise restaurant commissions?" (multihoming, restaurants' alternatives such as ONDC and direct ordering, elasticity, leakage risk). A strong line: *"In a two-sided market, I'd first ask which side is scarce. Growth spend on the abundant side is wasted until the scarce side is fixed."*

Check yourself

1. Distinguish same-side from cross-side, and local from global, network effects, with one Indian example of each.
2. Sketch the S-curve and label cold start, critical mass and saturation. What is a negative network effect?
3. Why was Uber's internal algorithm improvement not 'differentiation'? What is the cost of convergence?
4. Name four healthy switching-cost levers, and one dark pattern you would avoid.
5. Why do platforms subsidise one side, and how does the subsidy change over time?
6. Explain multihoming and a competitive bottleneck using food delivery and app stores.
7. Give three ways to solve the cold-start problem.
8. (From the AI chapter) An AI feature makes a marketplace's search better for every user as more people use it. What kind of network effect is that?

TECHNOLOGY • EMERGING TECH

Blockchain: a ledger nobody can quietly rewrite

How a chain of blocks works, the four kinds of blockchain, where it has created real business value, and the questions that tell you whether you need one at all

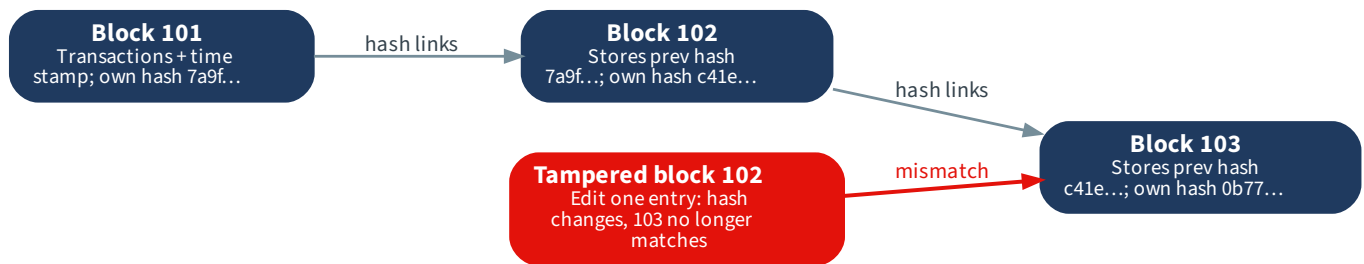
Picture a neighbourhood savings group of twenty families, a chit fund of the kind common across India. One member keeps the account book. Everyone has to trust that she records every contribution honestly and never quietly changes page 12. Now imagine instead that **every family keeps an identical copy** of the book, every new entry is read out and checked by the group before everyone writes it down, and each page carries a seal that depends on the page before it. To cheat, you would have to secretly rewrite the same page in more than half the books at once, and redo every page after it. Cheating becomes practically impossible, and nobody needs to be the trusted bookkeeper.

That is the idea of a **blockchain**: a **distributed ledger** shared across many computers (nodes), where records are grouped into blocks, each block is cryptographically linked to the one before, and the network agrees on each new block before it is added. It was invented to run Bitcoin, but the ledger idea applies wherever several parties who do not fully trust each other need one shared record.

How it works

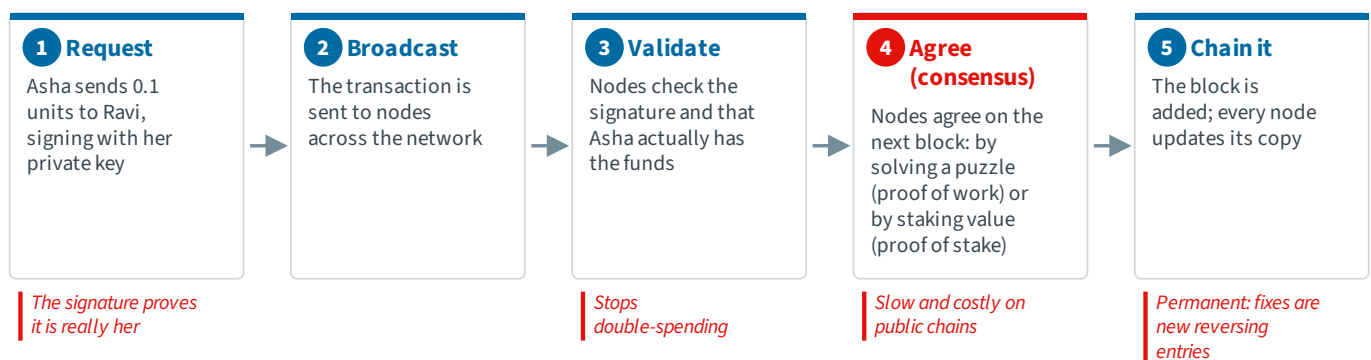
A chain of blocks

Each block stores the fingerprint (hash) of the previous one; change any old block and every later link breaks



A **hash** is a digital fingerprint: a short code calculated from the block's contents. Change even one character of the contents and the fingerprint changes completely. Because each block contains the previous block's fingerprint, the blocks form a chain; tampering with an old block breaks every link after it, and the thousands of other copies on the network immediately disagree.

How a transaction gets onto the chain



Three properties follow:

- **Open and distributed.** Participants share one ledger instead of each keeping and reconciling their own. A transaction is recorded once for everyone, removing duplicated effort.
- **Immutable.** No participant can tamper with a recorded transaction. If there is an error, a new transaction must reverse it, and both remain visible, like a correction entry in accounting.
- **Secure through keys.** **Public-key cryptography** gives each participant two keys. The **public key** is like your account number or letterbox address: anyone can send to it. The **private key** is like the only key to that letterbox: whoever holds it can authorise transactions and prove ownership. Lose it and the assets are gone; nobody can reset it.

Four kinds of blockchain

The two questions are: who may **read and join**, and who may **write and validate**?

Types of blockchain by permission



Where it has created business value

Most successful uses are **between organisations** that must share records but do not want any one of them to own the database.

Three well-documented enterprise results

Faster trade finance

HSBC and ING settled a soybean shipment from Argentina to Malaysia with digitised trade documents in 24 hours, versus 5–10 days on paper.

Food traceability

Walmart and IBM traced the source of a mango pack in 2.2 seconds, versus about 7 days by the old process, speeding recalls.

Shared KYC

Three banks in Singapore piloted verifying a customer once and reusing it across banks; KPMG estimated 25–50% cost savings.

Source: HSBC/ING (May 2018); Walmart/IBM Food Trust (2018); OCBC, HSBC, MUFG with Singapore IMDA (Oct 2017), KPMG estimate

Other applications: **cryptocurrency** (peer-to-peer digital money without a bank); **supply-chain tracking** (every hand-off recorded in a tamper-evident log); **smart contracts** (code on the chain that executes automatically when conditions are met, such as releasing payment when a shipment is scanned as delivered); **digital identity** (verifiable credentials the holder controls); **voting** and public records (auditable, hard to alter); and **tokenisation** of assets such as bonds or property.

The shortcomings

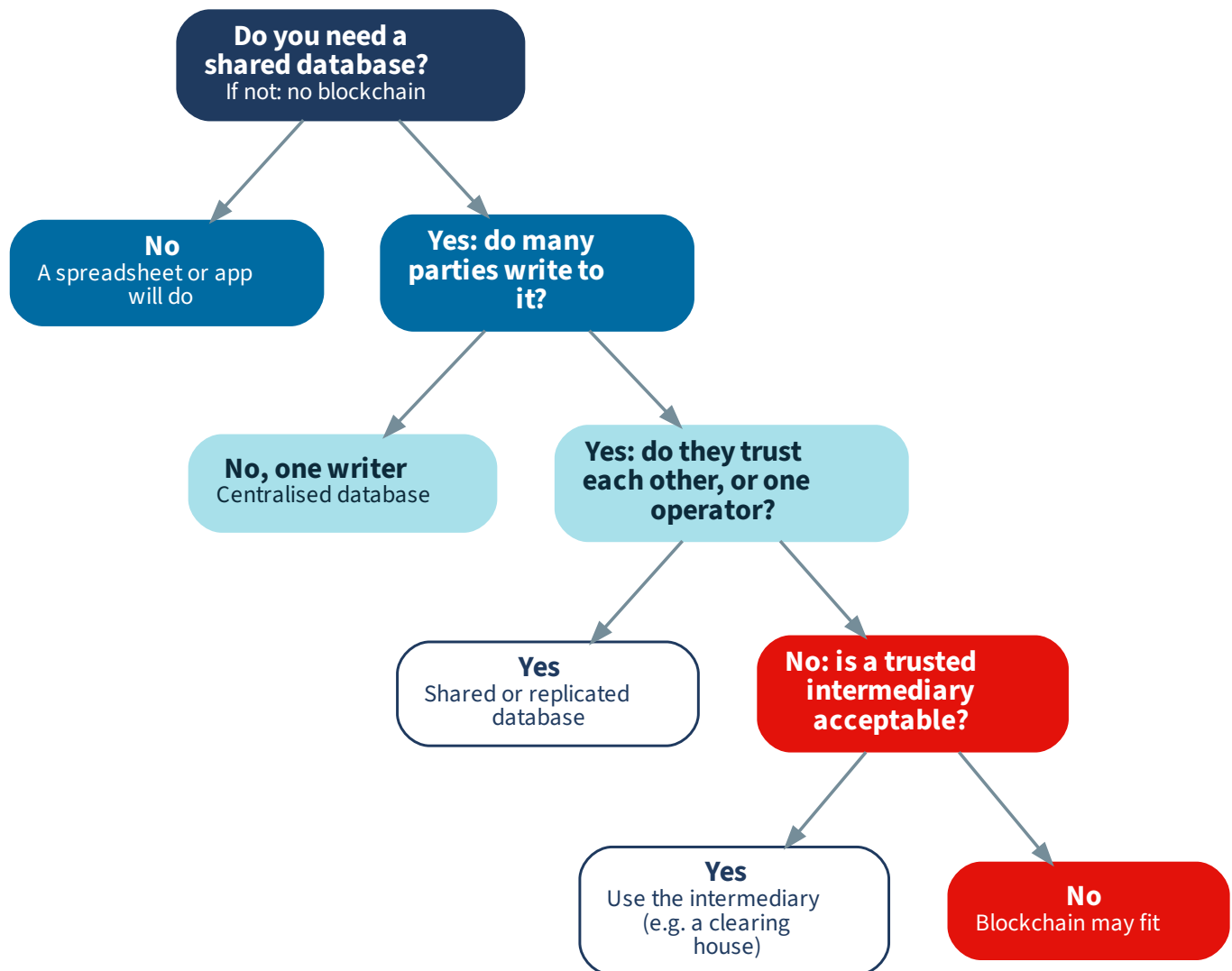
- **Energy use.** Proof-of-work mining consumes enormous electricity. Ethereum's switch to proof of stake in September 2022 cut its energy use by over 99%, showing it is a design choice, not a law of nature.
- **Throughput.** Bitcoin processes only about 7 transactions per second. UPI averaged about 9,000 per second in August 2026. Public chains are slow because thousands of nodes must agree.
- **User experience.** Wallets, seed phrases, gas fees and irreversible mistakes confuse mainstream users. Send to a wrong address and there is no customer care.
- **Regulation.** Rules vary widely and change often. India, for example, taxes gains on crypto assets at 30% with 1% TDS on transfers (since 2022).
- **Garbage in, garbage forever.** A blockchain guarantees that a record was not changed; it cannot guarantee the record was true when entered. If a supplier scans a fake label, the chain faithfully preserves the lie. (This is called the oracle problem.)

Do you actually need a blockchain?

Most proposals do not. A widely used test, adapted from the World Economic Forum and others:

Should this be a blockchain?

Answer in order; most projects exit early to a normal database



The one-line rule

A blockchain is a slow, expensive database whose one superpower is letting parties who do not trust each other share a record nobody controls. If there is a trusted operator, use a normal database.

In the interview. "Should our supply-chain product use blockchain?" (walk the decision tree; name who writes, who trusts whom, and the oracle problem), "Explain blockchain to a 10-year-old" (the shared notebook with seals), "Why is UPI not on a blockchain?" (a trusted central operator, NPCI, exists; UPI needs thousands of transactions per second, instant finality and reversibility for errors, all of which a public blockchain makes harder). Trap: treating "decentralised" as automatically better.

TECHNOLOGY · EMERGING TECH

The Internet of Things: when objects get sensors and a connection

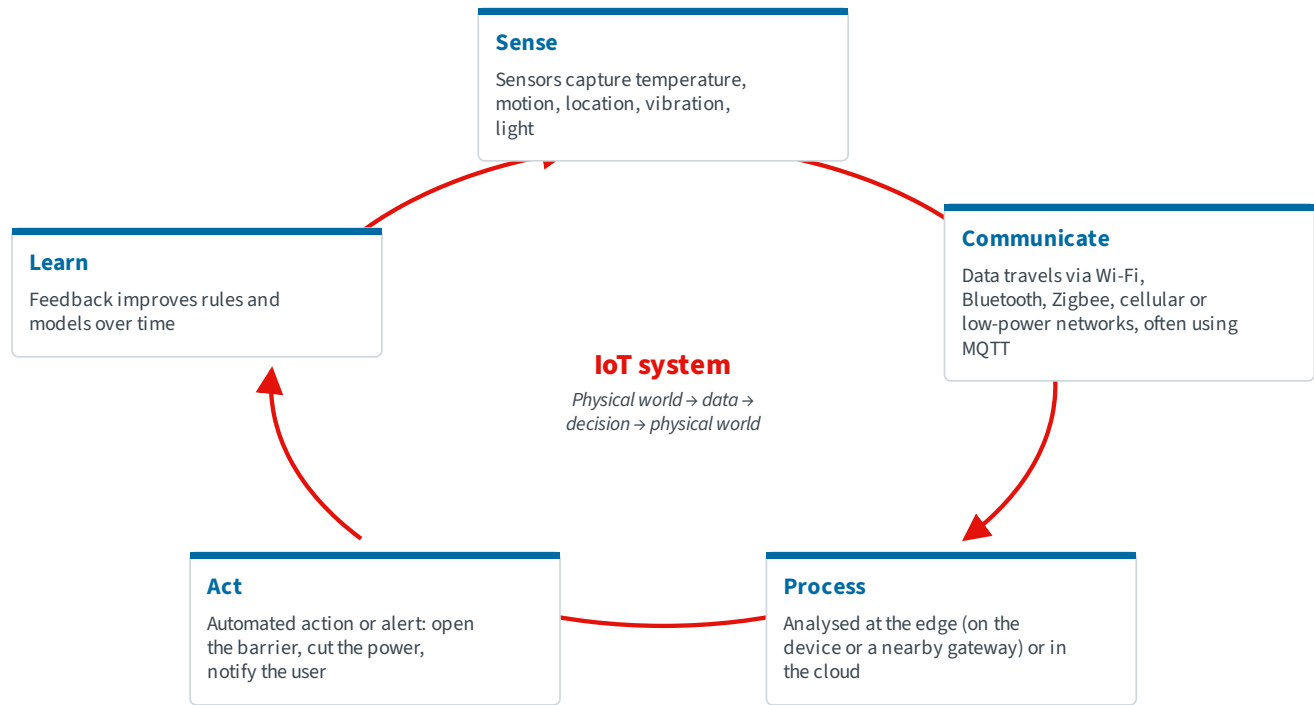
How connected devices sense, communicate, decide and act, from FASTag to factory machines

You drive through a toll plaza on the Mumbai–Pune Expressway without stopping. A small RFID sticker on your windscreen, **FASTag**, is read by a scanner above the lane, the toll is deducted from a linked account, and an SMS arrives before you have changed gear. FASTag was made mandatory on national highways in February 2021, and cash queues at most plazas shrank from minutes to seconds. The windscreen sticker, the lane reader, the payment switch and the SMS together are a small **Internet of Things (IoT)** system.

IoT is a network of physical devices with **sensors, software and connectivity** that let them collect data, exchange it, and act on it. It ranges from a fitness band or a smart electricity meter to thousands of vibration sensors in a steel plant or the traffic signals of a smart city.

How IoT works

The IoT loop



The connectivity choice is a product decision because it fixes range, battery life and cost.

Option	Range	Power use	Typical use
RFID / NFC	Centimetres to a few metres	None on the tag	FASTag, metro cards, warehouse tags
Bluetooth Low Energy	~10 m	Very low	Wearables, earbuds, smart locks
Zigbee / Thread	Tens of metres, mesh	Very low	Smart bulbs, home sensors
Wi-Fi	Home or office	High	Cameras, smart TVs, appliances
Cellular (4G/5G, NB-IoT, LTE-M)	Kilometres	Medium to low	Connected scooters, smart meters, fleet tracking
LoRaWAN	Several kilometres	Very low	Farm sensors, city infrastructure

MQTT is a lightweight messaging protocol designed for small devices on unreliable networks: devices publish short messages to topics, and anything subscribed receives them. **Interoperability** standards such as **Matter** (launched in 2022 by major smart-home companies) aim to let devices from different brands work together.

Edge versus cloud processing

Edge: decide on or near the device	Cloud: send data up, decide centrally
● Good for Split-second responses, poor connectivity, privacy (data stays local)	● Good for Heavy analysis, combining data from many devices, training models
● Example A factory machine shuts itself down within milliseconds when vibration spikes	● Example Predicting which of 5,000 machines will fail next month
● Cost Less computing power; harder to update	● Cost Latency, data charges, needs a connection

Most real systems do both: react at the edge, learn in the cloud.

Types of IoT

Four kinds of IoT

Consumer IoT Convenience and lifestyle. Smart speakers, lights, appliances, fitness trackers, glucose monitors, connected scooters like Ather's.	Commercial IoT Customer experience and security in business settings: smart retail offers, hotel room controls, asset tracking in warehouses.	Industrial IoT Predictive maintenance, safety and efficiency in manufacturing, logistics and energy: remote machine monitoring.	Infrastructure IoT Sustainability and safety at city scale: smart grids and meters, water leak detection, traffic signals, FASTag tolling.
---	---	---	--

Business value

- **Operational efficiency.** Amazon's warehouses use fleets of robots that bring shelves to pickers, speeding fulfilment and cutting labour cost.
- **Data-driven decisions.** Starbucks connects its coffee machines to monitor drink quality and push recipe changes remotely.
- **Customer experience.** Disney's MagicBand lets park visitors unlock hotel rooms, enter rides and pay with a tap of the wrist.
- **New revenue models.** John Deere sells precision-farming subscriptions built on data from connected tractors; connected-vehicle makers sell navigation and diagnostics packs. Hardware becomes a recurring service.

Coupled with AI, IoT enables **predictive** products (the machine that tells you it will fail next week) and personalisation from real-world behaviour.

The challenges

Security is the big one: cheap devices with default passwords are easy to hijack. In 2016 the Mirai botnet enslaved hundreds of thousands of cameras and routers and used them to knock major websites offline. Add **battery life**, **patchy rural connectivity**, **privacy** (a smart meter reveals when you are home), **interoperability** between brands, and **long hardware lifecycles**: a device in the field for ten years needs over-the-air updates and a plan for when the company changes its cloud.

The one-line rule

IoT turns physical events into data and data back into physical actions. The product is the whole loop, including the device's battery, its connection, its security and its updates, not just the app.

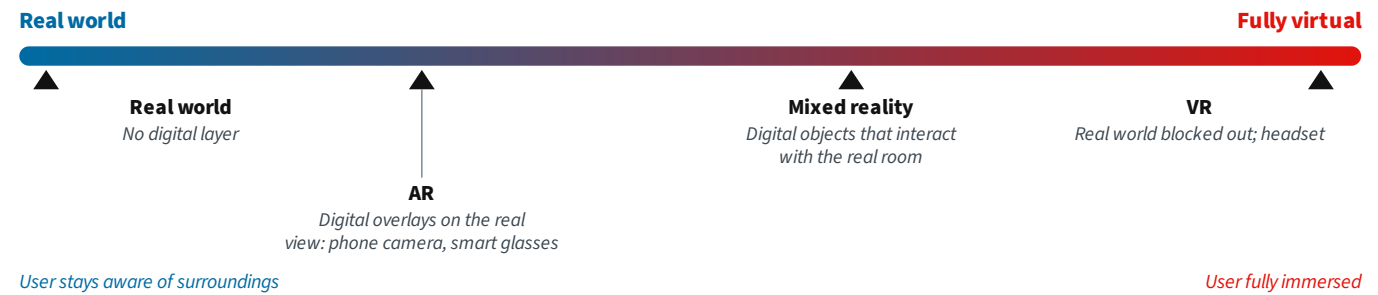
In the interview. "Design a smart water meter for Indian apartments" (who pays and why: residents want fair bills, societies want leak alerts; connectivity in basements; battery life of years; tamper detection; edge rules for leaks, cloud for billing; privacy), "How would you monetise a connected product?" (hardware margin, subscriptions for premium features, data insights for B2B, but with consent). Strong line: "For an IoT product I'd define the job at the level of the physical outcome, 'no leaks go unnoticed for more than an hour', and work backwards to sensing frequency, connectivity and battery."

AR and VR: adding to reality, or replacing it

How augmented and virtual reality work, where each fits, and the hardware realities that decide adoption

Open the Lenskart app, point your phone's camera at your face, and frames appear on it, moving as you turn your head. Open IKEA's app and a sofa sits in your living room at true scale. Walk out of a metro station, raise Google Maps in **Live View**, and arrows float on the street telling you which way to go. These are **augmented reality (AR)**: digital content layered onto the real world. Put on a Meta Quest headset and you are standing in a virtual cockpit or on a mountain; the real room disappears. That is **virtual reality (VR)**: a fully digital environment that replaces what you see.

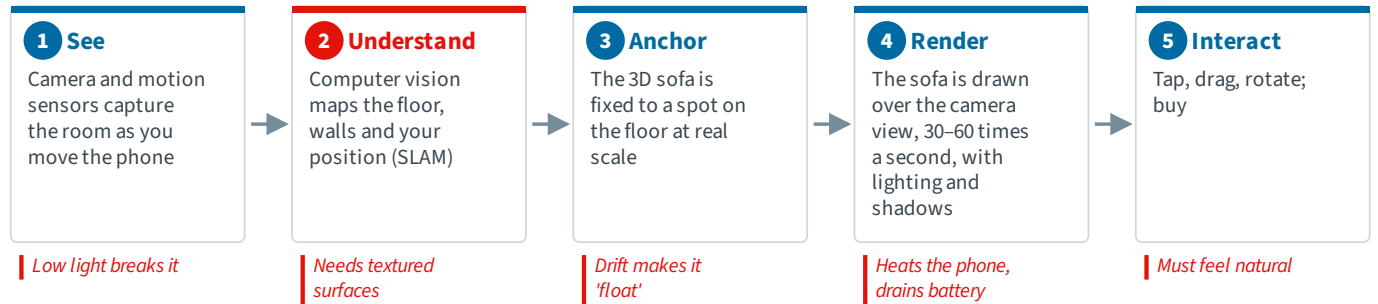
The reality spectrum



Together these are often called **XR (extended reality)**; "mixed reality" describes the middle, where virtual objects are anchored to and interact with real surfaces (Apple's Vision Pro and Meta's Quest 3 can blend both).

How they work

How phone-based AR places a sofa in your room



VR works differently. The headset shows a slightly different image to each eye to create depth, tracks your head (and often hands) in all directions many times a second, and redraws the world instantly as you move. The critical number is **motion-to-photon latency**, the delay between moving your head and the image updating: if it is too high (roughly above 20 milliseconds), the brain notices the mismatch between what the inner ear feels and what the eyes see, and many people feel sick. Spatial audio and haptic feedback (vibrations in controllers) add a sense of **presence**.

	Augmented reality (AR)	Virtual reality (VR)
Goal	Enhance reality	Replace reality
Environment	Real world + digital overlays	Fully virtual
User awareness	Remains aware of surroundings	Immersed; surroundings hidden
Hardware	Smartphones, tablets, smart glasses	Headset + controllers or hand tracking
Interaction	Touch, gestures, voice, glasses	Head, hand and controller tracking
Best for	Navigation, shopping previews, maintenance, labels in view	Gaming, training, simulations, virtual tours

	Augmented reality (AR)	Virtual reality (VR)
Examples	Pokémon GO, Snapchat filters, Google Maps Live View, IKEA, Lenskart try-on	Meta Quest, PlayStation VR, VR apps on Apple Vision Pro

Applications by industry

Industry	AR example	VR example
Retail	Furniture and eyewear try-on at home	Virtual showrooms
Healthcare	Vein visualisation, surgical guidance	Surgical training simulations
Education	Interactive textbooks, 3D models on the desk	Virtual labs and field trips
Manufacturing	Step-by-step assembly instructions in view	Safety and equipment training
Navigation	Walking directions overlaid on the street	Virtual building walkthroughs
Real estate	See furniture in an empty flat	Virtual home tours before visiting
Entertainment	Pokémon GO, face filters	VR games, immersive concerts

When to build which

A decision guide

Build AR when...	Build VR when...
● Users are interacting with the physical world.	● Full immersion improves learning or enjoyment.
● Location or context matters.	● Simulation is safer or cheaper than the real thing.
● They need real-time information overlaid on what they see.	● The experience is impossible physically.
● Examples Navigation, shopping preview, warehouse picking, maintenance support.	● Examples Pilot training, surgery practice, virtual tourism, gaming.

AR reaches billions of phones today; VR needs a headset, so it wins where immersion is worth the hardware.

AR challenges	VR challenges
Device compatibility across thousands of phones	Expensive hardware; Apple Vision Pro launched at US\$3,499 in 2024
Tracking accuracy; objects drift	Motion sickness
Battery drain and heat	Fatigue and discomfort in long sessions
Lighting and environment issues	Limited adoption: few people own headsets
Privacy: cameras always watching	High cost of creating 3D content

The one-line rule

Choose AR when the real world is the context and VR when leaving it is the point. Success depends on latency, comfort and device reach more than on visual wow.

Why a PM cares. Distribution decides AR/VR products: phone-based AR is available to hundreds of millions of Indian users today, while headsets remain niche, so most consumer wins in India are AR features inside existing apps (try-on, visual search, navigation). The metrics are practical: does try-on raise conversion and cut returns? How long before users feel tired? Does it work on a ₹12,000 phone in a dim room?

In the interview. "Should Nykaa invest in AR try-on?" (yes if it reduces uncertainty at purchase, the main driver of returns in beauty and eyewear; measure conversion and return rate by A/B test; ensure it works on mid-range phones and diverse skin tones), "Where does VR make business sense?" (training where mistakes are dangerous or expensive: pilots, surgeons, heavy machinery). Trap: proposing a VR headset product for a mass Indian consumer use case without addressing hardware adoption.

Check yourself

1. Explain hashes and why tampering with an old block is detectable.
2. What do the public and private keys each do? What happens if you lose a private key?
3. Place Bitcoin and Hyperledger Fabric in the permission matrix and explain the trade-off.
4. Walk through the 'do you need a blockchain?' tree for UPI. Where does it exit?
5. Describe the IoT loop using FASTag. What is processed at the edge and what in the cloud?
6. Why would you pick LoRaWAN over Wi-Fi for farm soil sensors?
7. What is motion-to-photon latency and why does it matter in VR?
8. Give one AR and one VR product idea for an Indian company, and the metric you would use for each.
9. (From the system-design chapter) A connected scooter maker must push a firmware fix to 3 lakh scooters. How would you roll it out safely?

TECHNOLOGY · BUILDING WITHOUT CODE

Low-code, no-code and vibe coding

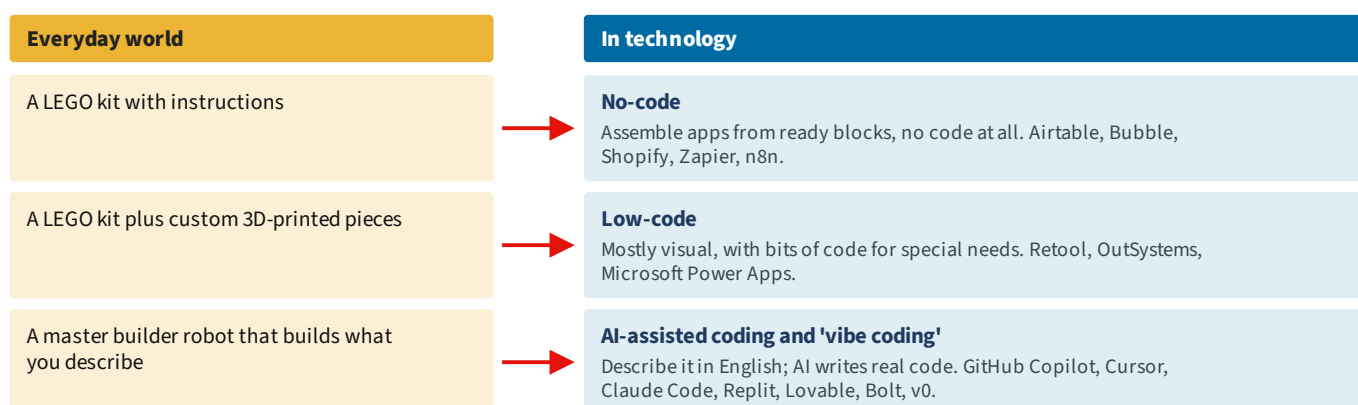
How PMs can now build working tools and prototypes themselves, which tool fits which job, and where the shortcuts become traps

You could build a house by learning to make bricks, mix mortar and frame walls from scratch. Or you could use a box of high-quality, pre-made LEGO blocks and have a good-looking house by evening. The first gives you total freedom and takes years; the second is fast but you are limited to the shapes in the box.

That is the idea behind **low-code and no-code (LCNC) platforms**: they provide ready-made building blocks for software (forms, databases, workflows, dashboards) that you assemble visually instead of writing every line of code. Since 2024, a third option has joined them: describing what you want in plain English and letting an AI write the code. For a PM, all three are levers for speed.

Three ways to build without (much) code

The LEGO analogy, extended



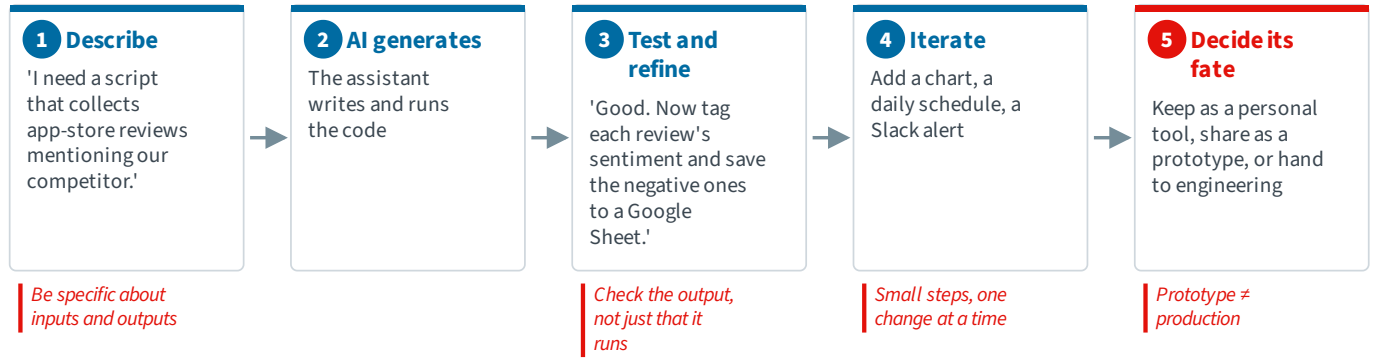
Category	Example use case	Tools
No-code	Track user feedback: an Airtable form and database, with an n8n or Zapier automation that posts each new entry to the team's Slack channel	Airtable, Bubble, Shopify, Zapier, n8n
Low-code	Sales wants a dashboard showing which customers use a new feature, pulling live data from the internal database, which a spreadsheet cannot do; built in Retool in a day	Retool, OutSystems, Power Apps, Appsmith

Category	Example use case	Tools
AI-assisted pro-code	Engineers building a complex feature use an AI pair programmer or coding agent to write routine code, tests and migrations	GitHub Copilot, Cursor, Claude Code, Codex

English is the new programming language

In February 2025, AI researcher Andrej Karpathy coined the phrase "**vibe coding**" for building software by conversation: you describe what you want, the AI writes the code, you run it, and you steer with more English rather than reading every line.

Vibe coding in practice



The newest frontier is **agent and workflow builders**: platforms where you configure an AI agent that reads a knowledge base, calls APIs (to check order status, say) and escalates to a human, through a visual interface. App builders such as Lovable, Bolt and Replit can produce a working web app with login and a database from a few paragraphs of description.

The PM's toolkit: which tool for which job

Picking the right tool

Internal tracker or form Airtable, Google Sheets + Apps Script, Notion databases	Automations between apps Zapier, Make, n8n: 'when X happens in one app, do Y in another'	Internal dashboards on live data Retool, Looker Studio, Metabase, Power BI
Clickable product prototype Figma for design; Lovable, Bolt, v0 for a working prototype	Data analysis and quick scripts An AI assistant writing SQL or Python you can run and check	Customer-facing product at scale Engineering team with AI-assisted pro-code; not a no-code tool

Speed versus control

Advantages and risks of LCNC and AI-built software

Advantages	Risks and limits
● Faster time to market MVPs, internal tools and AI features in days instead of months.	● Vendor lock-in Moving off a proprietary platform can mean a complete rebuild.
● Lower cost Less engineering time for standard apps and automations.	● Scale and performance ceilings Pure no-code tools can struggle at enterprise load.
● Agility PMs can build and change prototypes and internal tools directly.	● Black-box debugging AI-generated code nobody understands is hard to fix when it breaks.
● Democratised innovation Business users solve their own problems, freeing engineers for core work.	● Security and governance 'Shadow IT' built by citizen developers can leak data or skip access controls.

Use these tools freely for learning, prototypes and internal tools; bring engineering in before anything touches customer data or money at scale.

As Andrew Ng has put it: *"Engineers are 10x faster. Product managers haven't sped up at the same rate. Now they're the bottleneck."* When building gets cheap, the scarce skill becomes deciding **what** to build and **whether it worked**. The PMs who benefit most use these tools to shorten their own loops: synthesising research in an afternoon, testing three prototype variants with users in a week, and arriving at engineering reviews with something people can click rather than a document people must imagine.

The one-line rule

Build prototypes and internal tools yourself; let engineering own anything customer-facing, security-sensitive or meant to scale. A prototype that proves a point in three days is worth more than a spec that takes three weeks.

In the interview. "How do you use AI in your PM work?" (concrete examples: research synthesis, SQL for a quick analysis, a clickable prototype tested with five users, with the checks you do on the output), "Should we build this internal tool on a no-code platform?" (who uses it, how many, what data it touches, how long it must live, lock-in and security). Trap: claiming you would ship a vibe-coded app straight to customers.

TECHNOLOGY · TRUST

Building trust: security, resilience and governance

The three lenses that decide whether users will hand a product their time, data, money and beliefs

A decade ago, many users accepted trade-offs: convenience at the cost of privacy, speed at the cost of reliability. That balance has shifted. Users are more aware, regulators more watchful, and competition unforgiving. A single breach, a payment outage on salary day, or an AI that treats some users unfairly can undo years of brand building. **Security, resilience and governance** are no longer back-end concerns; they are front-line tests of whether a product deserves to exist.

Three lenses of trust

Each answers a question the user never asks out loud

Security: keeping promises

Can I trust you with my data, identity and money? Breaches are broken promises, not just technical failures. PM question: what are we promising, and can we keep it under stress?

Resilience: keeping them consistently

Will it work when I need it, on the IPL final or salary day? Users rarely notice resilience, but instantly feel its absence. PM question: can the product absorb shocks and still deliver its core value?

Governance: doing the right thing

Should this be built at all? Who could be harmed? Fairness, transparency, accountability. PM question: will this decision still look right when explained in public?

Security is not only encryption and compliance; it is designing experiences that make users *feel* safe without friction. UPI's rule that the PIN is only ever entered to *pay*, never to *receive*, is a security principle turned into a simple habit users can learn. Risk-based step-up checks (extra verification only for a new device or an unusual transfer) protect without taxing every user.

Resilience is the silent foundation. Latency spikes, outages and fraud attacks do not just frustrate; they erode credibility. PMs should prioritise fault tolerance, graceful degradation and recovery with the same seriousness as new features, and practise the incident playbook before it is needed (the system-design chapter covers the mechanics).

Governance grows in importance as products grow powerful, especially with AI. It is not about saying no to innovation; it is about innovation that sustains trust. In India it now has regulatory teeth: the Central Consumer Protection Authority's 2023 guidelines prohibit a list of **dark patterns** such as false urgency, basket sneaking, drip pricing and "confirm-shaming", and the DPDP Act requires clear consent and purpose limits for personal data.

The PM's takeaway

Treat security, resilience and governance as lenses, not checkboxes. For every decision, ask not just "will it work?" but "will it be safe, will it endure, and will it do right by the user?"

In the interview. Trust themes appear inside other questions: "Design a digital locker for documents" (security and consent first), "Launch an AI loan-approval feature" (fairness audits, explanations, human review, appeal path), "Your app was down for two hours on Diwali; what now?" (communicate honestly, compensate where fair, postmortem, resilience investment). Naming these lenses unprompted signals maturity.

TECHNOLOGY · MARKETING MEASUREMENT

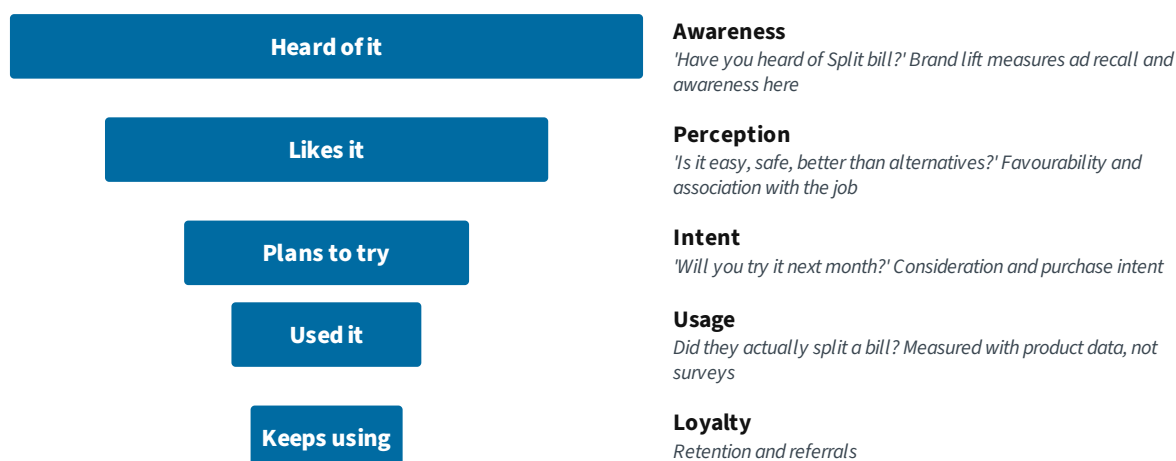
The customer funnel and brand lift

How to prove that a campaign changed what people know, think and intend, using the same logic as an A/B test

You launch a new "Split bill" feature in a payments app and spend ₹5 crore on a YouTube and Instagram campaign. Usage rises 12%. Was that the campaign, or the festive season, or a competitor's outage? Did people actually become *aware* of the feature, and do they now *think* of your app as the one for splitting bills?

A **brand lift study** measures how much a campaign changed people's **awareness** and **perception** of a product or feature. Awareness mainly helps the top of the customer funnel (getting considered); perception helps the bottom (getting chosen).

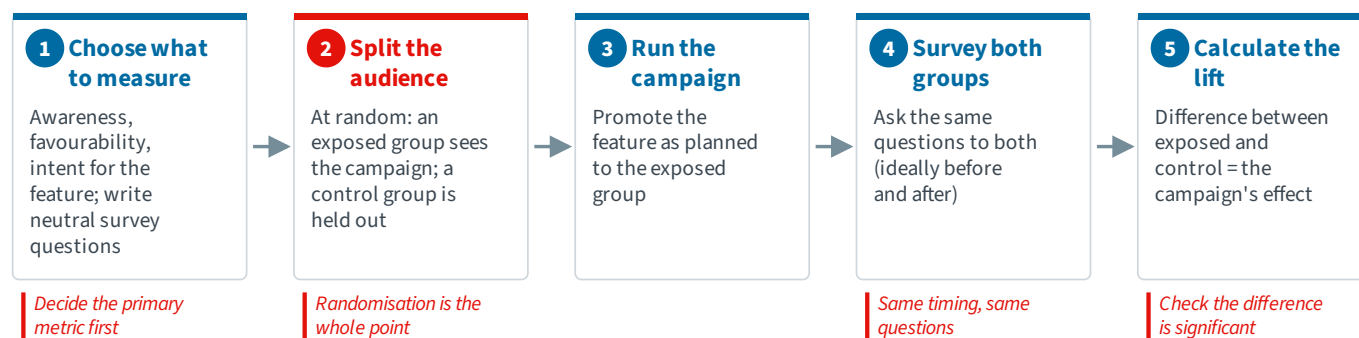
What a campaign can move, stage by stage



How a brand lift study works

It is a randomised experiment, like an A/B test, run on audiences rather than app users.

Running a brand lift study



A worked example. After the campaign, 40% of the exposed group know about Split bill, versus 20% of the control group. The **absolute lift** in awareness is 20 percentage points; the **relative lift** is 100% (awareness doubled). Always say which one you mean; they are easy to confuse in presentations. If you also surveyed before the campaign, compare the *change* in each group (a before-and-after in both groups), which removes pre-existing differences between them.

What it answers	Example question
Awareness	Are more people aware of the new feature after the campaign?
Perception	Do people have a better opinion of it: easy, safe, useful?
Intent	Are more people planning to try or buy it?
Engagement	Are people using it more, and saying positive things?

The benefits: understanding what the campaign actually did, making data-driven decisions about future campaigns, allocating budget to what works, and proving return on investment to stakeholders. Google (for YouTube), Meta and other ad platforms offer built-in brand lift studies that hold out a control group automatically.

Real-world brand-lift questions

YouTube Shorts

Rolling out Shorts globally: do people know Shorts exists as an alternative to TikTok-style video, and do they see it as fun enough to use? Track awareness and perception.

WhatsApp privacy features

After campaigns for disappearing messages and encryption: do users feel WhatsApp is safer and trust it with private conversations? Track perception (safety, trust) and intent.

Airbnb 'I'm Flexible'

Did ads and in-app nudges make travellers aware of flexible search, and do they now see Airbnb as a place to discover trips, not just book them? Track awareness and engagement.

Contrasting case and limits. Surveys measure what people say, not what they do; a lift in awareness does not guarantee a lift in usage. Pair brand lift with **conversion lift** (the same exposed-versus-control design, measured on actual sign-ups or purchases) or **geo experiments** (run the campaign in some cities and not in matched others). Sample sizes must be large enough, questions neutral, and the control group truly unexposed.

The one-line rule

Measure campaigns like experiments: a randomly held-out control group, the same questions to both groups, and the lift is the difference. Awareness and perception come first in the funnel, but usage is the result that matters.

In the interview. "How would you measure whether our launch campaign worked?" (define the funnel stage the campaign targets; brand lift for awareness and perception with a holdout; conversion lift or geo test for usage; watch for seasonality), "Awareness went up but usage didn't; why?" (message not linked to a job; friction in the product; wrong audience; timing). A strong line: *"I'd separate awareness from adoption: brand lift tells me whether people heard us, and a holdout on product usage tells me whether hearing us changed what they did."*

Check yourself

1. Distinguish no-code, low-code and AI-assisted pro-code with one use case each.
2. What is vibe coding, and what is its biggest risk in production?
3. Name two advantages and two risks of LCNC platforms. What is 'shadow IT'?
4. What question does each of the three trust lenses answer? Give an Indian product example for each.
5. Name three dark patterns and explain why they are a governance issue, not just a UX one.
6. Design a brand lift study for a new feature: groups, questions, and how you compute lift.
7. Exposed awareness is 30% and control is 24%. What are the absolute and relative lifts?
8. (*From the A/B testing chapter*) Why is a randomly held-out control group essential both in A/B tests and in brand lift studies?

INDEX

Every case in this book

#	Case	Type	Page
---	------	------	------